DOT-Design

Version Next, 2026-10-28

Overview

Describe a tool once. Reuse it in different deployments and compositions.

DOT-Design provides a portable description of software components and the `dotx` command-line tool to validate, deploy and connect them in the MARE 6G security ecosystem. A Dynamic Open Trust component (**DOT**) is a reusable definition, not a particular running container, VM or Kubernetes Pod.

For example, one Prometheus DOT can describe its API and supported deployment choices. A composition can then use it to scrape a selected metrics exporter, without creating a second Prometheus product definition for that use case.

Choose your first step

- **Use an existing tool:** Getting started takes you from CLI installation to a running Prometheus service, a verified API response and cleanup. Java and contributor tooling are not required.
- **Watch or present the ecosystem:** the 10-15-minute guided demo connects the concepts to executable evidence.
- **Integrate your own component:** start with the model below, then use the developer tour and the authoring and composition guide.

> The bottom-left version selector offers **Next**, the current development documentation, alongside immutable stable releases. Select **Next** to follow the latest walkthrough and examples.

The model in one minute

Question	Model	Plain-language meaning
What is this component?	`Dot`	Reusable identity and version, described by `dot.json`.
What can it do?	`Skill`	At least one high-level capability reference; not executable code or an API.
What interface does it implement or require?	`Contract`	An authoritative OpenAPI, AsyncAPI, Protobuf or GraphQL document. Complete declarations identify its raw HTTPS payload and SHA-256 integrity pin.
Which interaction can another DOT connect to?	`Anchor`	A stable `PROVIDES` or `REQUIRES` point, normally selecting an operation from a declared Contract rather than copying its interface metadata.
Where and how can it run?	`Runtime`, `Platform`, `Deployment`, `DeploymentManifest`	A platform is **where** it runs; a deployment selects **how** to launch it. Kubernetes may offer both manifest and Helm alternatives.

Question	Model	Plain-language meaning
Which service implements the interface?	`DeploymentService`	Logical services, named ports, Anchor bindings and supported configuration inside a selected deployment.
How do components work together?	`Composition`, `DotInstance`, `Wire`	A separate domain aggregate selects pinned DOTs and connects their Anchors; it does not copy or embed the reusable DOT definitions.

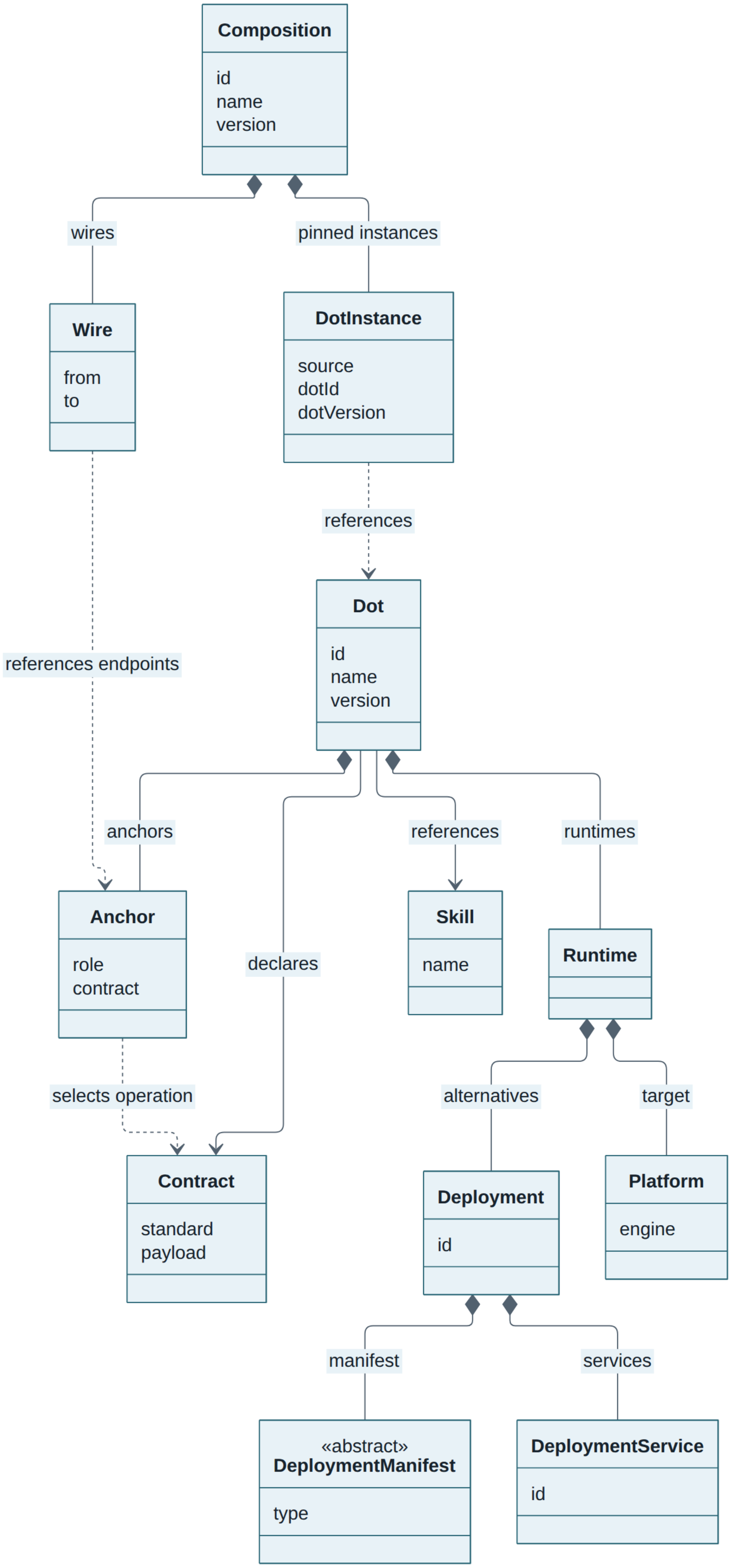

Composition
id
name
version
wires
pinned instances
Wire
from
to
DotInstance
source
dotId
dotVersion
references
Dot
id
name
version
references endpoints
anchors
references
runtimes
Anchor
role
contract
declares
Skill
name
Runtime
selects operation
alternatives
target
Contract
standard
payload
Deployment
id
Platform
engine
manifest
services
«abstract»
DeploymentManifest
type
DeploymentService
id

Figure 1. Simplified domain model: reusable DOT definitions and a separate Composition

Diamonds indicate embedded values; arrows indicate declarations or references. The dashed arrow applies to Contract-backed Anchors. Collection sizes, configuration, ports, binding helpers and audit metadata are omitted for readability; see the full generated class diagram for implementation detail. `Composition` is a separate DOT-Domain aggregate: its instances reference DOTs and its Wires reference their Anchors. Neither is embedded in the reusable Java `Dot` aggregate.

Read the runtime branch as **alternatives**, not a requirement to deploy everything: choose one supported deployment for each composition instance. An Anchor binding maps the logical Anchor to a named port of a deployment service; concrete reachable addresses are resolved for the running instance.

Groups, descriptions and metadata help classify or audit a definition. They do not change API semantics or grant runtime permissions. For field-level validation and the complete generated model, continue to DOT-Domain.

From one DOT to a working composition

A DOT describes a reusable component; a composition describes one use case. A separate `composition.yaml`:

1. Selects named DOT instances with pinned identities and versions.
2. Chooses a supported deployment for each instance.
3. Connects a provider Anchor to a consumer Anchor with a **wire**.
4. Supplies use-case configuration and Secret references; a reviewed lock pins the resolved definitions, Contracts and inputs.

For example, the PoC10 walkthrough connects Keycloak, APISIX, OPA, Oxigraph and a machine client. Realm configuration, gateway routes, authorization policy and credentials belong to that use case, not to separate copies of those products' DOTs.

Follow one API call

In the Prometheus quickstart, the official Contract describes the build-information operation. The `build-information` Anchor selects it, and the chosen deployment binds that Anchor to a service port. The verified response comes from the running Prometheus instance at `http://127.0.0.1:18909/api/v1/status/buildinfo`.

The Contract describes the interaction; the Anchor gives it a stable composable identity; the binding makes it reachable. None of these is the same thing as a successful live call.

> `dot.json` is the deployable DOT definition and the only source inspected by `dotx deploy`. Connections between deployed DOT instances are described in a separate composition document. A wire does not create an API, translate protocols automatically or prove production readiness. Concrete addresses, credentials and live policy results never become Anchor fields.

Documentation map

Read	When you need to
Getting started	Install the released CLI, deploy Prometheus with Docker Compose, verify its API, and clean up.
Contributor guide	Build the project, run shared local/GitHub checks, and generate documentation.
Guided demo	Follow a 10-15-minute live presenter route, a five-minute read-only overview, or a deeper onboarding walkthrough from catalogue validation to live evidence.
Developer demo: reuse and extend	Inspect a real DOT, prove an invalid wire is rejected, and map PoC changes to shared definitions, composition inputs and executable acceptance.
Architecture	Understand repository boundaries, sources of truth, runtime/deployment separation, and the composition resolution path.
DOT Anchors and Composition	Author Anchors, bind them to services, connect DOT instances, materialize configuration, handle security inputs, and operate a composition.
Verification and examples	Choose conformance, dry-run, source-adapted, or live evidence and find the executable MARE PoC scenarios.
DOT-Domain	Work with the Java aggregate, value objects, validation, Contracts, Runtimes, and deployment manifests.
Integration handoff	Implement a catalogue, composer, or SFC adapter without coupling to a specific service, language, framework, or storage technology.

Repository components

Component	Responsibility	Build ecosystem
`DOT-Domain`	Java domain model and validation library. It is not a web application.	Maven 3.10.0 / Java 27
`DOT-CLI`	Independent `dotx` CLI for authoring support, validation, inspection, deployment, runtime lifecycle, and the Composition v1 reference executor.	Go 1.27.2
`DOT-Repository`	Independently cloneable Git submodule containing reusable DOT definitions, reusable compositions, generated JSON Schema, and high-level Skill classifications. The filesystem is the catalogue index.	Git submodule

Component	Responsibility	Build ecosystem
`DOT-Testing`	Repository-level conformance, composition, interoperability, and end-to-end evidence. It is test infrastructure, not a Maven module.	Maven, Go, shell, container tools

Supported Contract standards

Standard	Anchor selector
OpenAPI	RFC 6901 URI-fragment JSON Pointer to an Operation Object
AsyncAPI	RFC 6901 URI-fragment JSON Pointer to an Operation Object
Protobuf	Fully qualified service method
GraphQL	Root-field schema coordinate

Contractless Anchors remain available for interactions that cannot be described by one of these standards. They must declare direction, protocol, and media type explicitly, and they do not pass the strict `interoperable` interface profile. An explicit SPARQL or metrics interface is not an invented OpenAPI document; a runtime-only broker does not own its applications' message Contracts.

Maturity and scope

The DOT-owned Anchor model, service-port binding, generated schema, repository resolution, conformance fixtures, `dot.mare6g.eu/v1` composition document, and `dotx compose` executor form the stable Composition v1 profile. The Java domain and the independently released CLI need not share a release number.

The reference executor is intentionally fail-closed. It does not claim to be a general multi-host orchestrator, trust-policy engine, active security scanner, or replacement for Kubernetes, Helm, Docker, Vagrant, or an SFC implementation. See the supported composition profile for the exact boundary.

First steps

After installing the CLI and checking out the catalogue, these commands run from the `DOT-Design` checkout without deploying anything. Topology rendering requires DOT-CLI `0.2.5` or a newer pinned checkout.

```
# Validate the real catalogue using verified local Contract assets
dotx validate DOT-Repository --schema DOT-Repository/Schema/schema.json --offline

# Inspect the reviewed five-DOT PoC10 resolved composition, not live traffic
composition=DOT-Repository/Compositions/keycloak-apisix-opa-sparql/composition.yaml
dotx compose validate "$composition" --locked
dotx compose topology "$composition" --locked --color auto
```

Use the guided demo for live execution and cleanup. Build toolchains, Java tests and the full local/GitHub CI route belong in the contributor guide.

Project links

- MARE project
- MARE proof-of-concept catalogue
- DOT-Design source repository

Getting started

Run your first DOT on Linux amd64 using the selected CLI release `v0.3.0` and the existing Prometheus definition. You need Bash, Git, GitHub CLI (`gh`), `curl`, `jq`, SHA-256 utilities, and Docker with Docker Compose. No Java, Go, Kubernetes, or Task is needed for this path.

This example runs Prometheus 3.15.1 on port `18909`. Allow time for initial image and Contract downloads. Use an isolated development host: the Compose manifest publishes unauthenticated HTTP on all host interfaces. Restrict access using your host firewall. Cleanup deletes the example's metrics data.

Install the released CLI

Authenticate with an account granted access to the private MARE repositories and releases. From a workspace directory:

```
gh auth login
gh auth setup-git
mkdir -p dot-quickstart-download
cd dot-quickstart-download
gh release download v0.3.0 --repo MARE-EU-Project/DOT-CLI \
  --pattern dotx-0.3.0-linux-amd64 --pattern SHA256SUMS
sha256sum --check --ignore-missing SHA256SUMS
mkdir -p "$HOME/.local/bin"
install -m 0755 dotx-0.3.0-linux-amd64 "$HOME/.local/bin/dotx"
export PATH="$HOME/.local/bin:$PATH"
dotx version
cd ..
```

Stop if checksum verification fails. Expected version: `dotx 0.3.0`. Add the `PATH` export to your shell profile for future sessions. The release also provides an SBOM and build/acceptance revision manifest.

Check out the catalogue

Only the independently cloneable catalogue is needed. Pin the acceptance revision recorded in this CLI release's manifest, then check your Docker access:

```
gh repo clone MARE-EU-Project/DOT-Repository
cd DOT-Repository
git checkout --detach eafc868fe41e8ed694e7032e72791f8b03ffcdd3
docker info
docker compose version
```

Run the following blocks in the same Bash shell from the **DOT-Repository root**. Run each block only after the preceding block succeeds. The selected descriptor embeds its Compose manifest; no neighbouring Compose file is inferred.

Select and validate

Choose an unused runtime name. For a different host port, run `export DOTX_HTTP_PORT=18919` before the selection block.

```
DOT_REPOSITORY="${DOT_REPOSITORY:-$PWD}"
DOT_BUNDLE="$DOT_REPOSITORY/DOTs/external/Prometheus-DOT"
DOT_NAME="${DOT_NAME:-dot-quickstart}"
DOTX_HTTP_PORT="${DOTX_HTTP_PORT:-18909}"
DOT_PROJECT_FILTER="label=com.docker.compose.project=$DOT_NAME"
export DOTX_HTTP_PORT
```

```
dotx validate "$DOT_BUNDLE" --schema "$DOT_REPOSITORY/Schema/schema.json" \
  --profile reusable
dotx inspect "$DOT_BUNDLE" --json
dotx deploy "$DOT_BUNDLE" --deployment docker-compose --name "$DOT_NAME" --dry-run
```

Validation checks the schema, official HTTPS OpenAPI Contract and SHA-256, Anchor selectors, and reusable deployment bindings. Inspection shows deployment choices. The exact ID `docker-compose` selects only that mechanism. Dry-run prints its commands; it does not prove that a service is running.

Deploy and verify

```
dotx deploy "$DOT_BUNDLE" --deployment docker-compose --name "$DOT_NAME" \
  --env "DOTX_HTTP_PORT=$DOTX_HTTP_PORT"
```

Wait up to 60 attempts (each HTTP attempt has a two-second timeout), then verify the service’s build-information API and version:

```
ready=false
for ((attempt=0; attempt<60; attempt++)); do
  if curl --fail --silent --max-time 2 \
    "http://127.0.0.1:$DOTX_HTTP_PORT/-/ready"; then
    ready=true
    break
  fi
  sleep 1
done
test "$ready" = true
curl --fail --silent --show-error --max-time 5 \
  "http://127.0.0.1:$DOTX_HTTP_PORT/api/v1/status/buildinfo" |
  jq -e '.status == "success" and .data.version == "3.15.1"'
```

Expected: `Prometheus Server is Ready.`, followed by `true` from `jq`. Check runtime status:

```
dotx ps
```

Expect a running workload. Open http://localhost:18909 (or your chosen port) to view Prometheus.

The `build-information` Anchor selects the official OpenAPI operation whose server prefix produces `/api/v1/status/buildinfo`. The `promql-query-api` Anchor selects query operations; both bind to the deployed service's `http` port. Its explicit contractless scrape-target requirement describes metrics ingestion; this example does not configure another DOT as a scrape target.

The pinned image's default configuration starts the service. The descriptor declares a Compose-managed `prometheus-data` volume. Data survives container restarts until the volume is deleted.

Clean up

> ⚠ For Compose, `dotx destroy` runs `down --volumes`: it deletes this instance's containers, network, and named volume, including collected metrics. Use the same `DOT_NAME` as deployment. Never target another user's instance.

```
dotx destroy "$DOT_BUNDLE" --deployment docker-compose --name "$DOT_NAME" &&
remaining="$(docker ps -aq --filter "$DOT_PROJECT_FILTER")" &&
test -z "$remaining" &&
remaining="$(docker network ls -q --filter "$DOT_PROJECT_FILTER")" &&
test -z "$remaining" &&
remaining="$(docker volume ls -q --filter "$DOT_PROJECT_FILTER")" &&
test -z "$remaining"
```

The final checks succeed silently. If interrupted, restore your original selection variables and repeat cleanup. Downloaded images remain cached.

Next: connect existing DOTs

For the existing Keycloak/APISIX/OAuth composition, use a full DOT-Design checkout with initialized submodules; see Contributor guide. From the **DOT-Design root**, with `dotx` on `PATH`:

This follow-up uses the source-built development CLI. Install the checked-out CLI using the contributor toolchain before continuing; released binaries do not necessarily implement the current commands or lock format.

```
composition=DOT-Repository/Compositions/keycloak-apisix-oauth/composition.yaml
task cli:install
dotx compose lock "$composition" --check
dotx compose inspect "$composition" --locked
dotx compose apply "$composition" --locked --dry-run
```

The three wires connect Keycloak discovery to APISIX, Keycloak's token endpoint to the OAuth client, and APISIX's protected operation to that client. The resolved composition resolves endpoints and configuration. Live acceptance additionally proves anonymous rejection and authenticated access; the harness prepares Kubernetes, native Secrets, and workloads. Follow Guided demo and Verification and examples for that path.

To author a component, continue with Authoring Anchors. For development toolchains and local/GitHub CI gates, use Contributor guide.

Troubleshooting

Symptom	**Action**
Release or clone returns 404/403	Check `gh auth status` and organization repository access, including SSO authorization if required. Run `gh auth setup-git` for HTTPS cloning.
CLI missing or wrong version	Check `command -v dotx`; put `$HOME/.local/bin` first on `PATH`.
Docker permission denied	Check `docker info` as your user. Ask the administrator for the intended Docker access, then log in again. Docker group access grants host-level power.
Compose unavailable	Install the Compose plugin/package for your Docker distribution and verify `docker compose version`. The legacy `docker-compose` executable is insufficient.
Port already allocated	Clean up the failed example, select a free `DOTX_HTTP_PORT`, and deploy again with the same name and explicit `--env` override.
Contract download or digest check fails	Check network/proxy access to the declared URL. Never bypass its digest. Private raw GitHub URLs require `MARE_READ_ONLY_TOKEN` or `GITHUB_TOKEN`; `gh` login alone does not supply those to `dotx`. Use `--offline` only with matching verified local Contract assets.

Symptom	Action
Readiness or version fails	Inspect `docker compose -p "$DOT_NAME" logs --tail=100` and `dotx ps --all`, check the selected image/port, then clean up.
Interrupted cleanup	Restore the original name and catalogue path, repeat cleanup, and inspect resources carrying `com.docker.compose.project=$DOT_NAME`.

Download the shared command blocks. Contributors rehearse them from DOT-Design with `task testing:quickstart:dry-run` or the opt-in `task testing:quickstart:live`.

Adoption, compatibility and editor setup

DOT connects reusable component descriptions, authoritative API Contracts, deployment choices and explicit composition wires. It does not replace OpenAPI, AsyncAPI, Protobuf, GraphQL, Docker Compose, Helm or Kubernetes. Start by reusing a complete catalogue bundle, not by inventing a new descriptor.

Choose a compatible set

The site opens **Next**: documentation for the current source checkout, not a promise that every command exists in the latest published binary. Use the version selector for historical release documentation.

Selected release baseline: CLI `v0.3.0` includes `dotx compose new` and the current model conventions for the supported Linux-amd64 profile. It does not certify every producer or catalogue entry. The original released CLI `v0.2.5` predates these features. For the official demo, use the submodules pinned by one clean DOT-Design commit, run `task testing:demo`, and retain its exact checkout revisions and binary SHA-256. Do not mix the latest release binary with the current catalogue. See the source-qualified demo route.

Artifact	Compatibility rule
Published CLI binary	Use its immutable release tag, checksum, acceptance manifest and matching catalogue revision. The released quickstart pins that catalogue explicitly; do not substitute current `main`.
Source-built CLI	Use the exact DOT-CLI and DOT-Repository submodules pinned by the DOT-Design checkout. `dotx version --verbose` identifies the revision and modified state. The source version label alone does not identify its capabilities.
DOT definition and Contract versions	These describe the component and interface, not the CLI release. Keep stable IDs, select exact DOT versions and review changed payload hashes and locks.
Composition v1	`apiVersion: dot.mare6g.eu/v1` selects the portable Composition input model. It does not imply support for every adapter, probe or future extension.

Current source supports textual Skill/Contract IDs and exact Contract standard spellings `OpenAPI`, `AsyncAPI`, `Protobuf`, `GraphQL`; `format` retains its separate uppercase vocabulary. The original published CLI `v0.2.5` predates these changes and the bundle-aware prerequisite check below. Do not silently rewrite IDs, spelling, manifests or reviewed locks to make mismatched artifacts appear compatible. Follow the pinned source route in the contributor guide for current catalogue work. See the compatibility and migration policy for intentional spelling/identity boundaries, release-version rules and preservation of existing files, Anchor IDs and reviewed locks.

Current source also distinguishes owned Contract declarations, explicit external Anchor references and provider-neutral OIDC requirements. Consumer bundles no longer carry producer Contract copies. An external reference requires its exact originating DOT in the selected catalogue; missing dependencies fail validation, including offline validation. See Anchors and composition for syntax, resolution and the deliberately narrow OIDC profile.

Edit with the actual model schema

Opening the DOT-Design root in VS Code associates `dot.json` with `DOT-Repository/Schema/schema.json`; opening the standalone DOT-Repository root uses `Schema/schema.json`. Both checkouts include workspace associations. JSON editing uses the built-in language service. For YAML compositions, enable the recommended Red Hat YAML extension; its workspace association uses the generated `composition-v1.schema.json`. No DOT-specific editor plugin is needed.

For other editors, associate those same **local** files with `dot.json` and `composition.yaml`, respectively. Use a matching pinned catalogue rather than a mutable remote schema. The generated schemas provide field descriptions, allowed values and structural validation without a Java runtime. Do not add an editor-only `$schema` property to DOT definitions: use editor configuration instead, preserving the portable document shape.

Editor validation is only the first check. It cannot replace bundle-local Contract integrity, Anchor selector resolution, catalogue identity checks or locked composition validation. Run `dotx validate` and the applicable local CI Tasks before committing. There is no automatic migration or lock repair.

For guided creation, author an owned OpenAPI interface with the same generated Contract model and existing verifier. The actual tool release is explicit; the wizard can generate provided Anchors and service-port bindings without copying a dependency Contract or inferring live acceptance. Raw-URL import can construct that declaration without hand-writing Contract JSON; its computed digest pins downloaded bytes, not upstream authority.

Check the selected deployment without starting it

Selected release or matching source. From the pinned DOT-Design checkout with the selected release or its source CLI installed:

```
dotx version --verbose
dotx inspect DOT-Repository/DOTs/external/Prometheus-DOT
dotx tool check --dot DOT-Repository/DOTs/external/Prometheus-DOT --deployment docker-compose
dotx tool check --dot DOT-Repository/DOTs/external/Prometheus-DOT --deployment docker-compose --json
```

From a standalone catalogue checkout use `DOTs/external/Prometheus-DOT` instead. For a standalone bundle outside a catalogue add `--schema /path/to/matching/Schema/schema.json`.

The command requires an **exact deployment ID**, validates the bundle offline with the ordinary validator, and checks only that adapter’s prerequisites:

Manifest	Prerequisites checked
`DOCKER_IMAGE`	Docker CLI and daemon
`DOCKER_COMPOSE`	Docker CLI/daemon and Compose plugin
`KUBERNETES_MANIFEST`	kubectl and selected cluster
`HELM_CHART`	kubectl/cluster and Helm cluster access
`VAGRANTFILE`	Existing Vagrant/libvirt/KVM readiness checks
`WASM_MODULE`	Wasmtime installation

Expected Compose output includes `Definition: validated offline`, `docker: ready=true` and `compose: ready=true`. Every applicable tool is checked even when another fails. No downloads, installation, cluster startup, workload mutation or cleanup occur. Missing or tampered local Contracts fail before tool checks. Warm a bundle's verified Contract assets through the ordinary online validation path separately if necessary; do not bypass a digest mismatch.

JSON has `formatVersion: 1`, `ready`, `validated`, selected `deployment` and `manifestType`, ordered `checks`, shared validator `diagnostics` and an explicit `evidence` scope. Check exit status as well as the report: 0 means checks passed, 1 means validation/tool failure, 2 means usage error. Ordinary JSON failures leave stderr empty; raw subprocess output and configuration values are withheld.

`ready` applies **only to these checks**. Instance inputs and permissions, available images, free ports, artifact reachability, storage and live API behavior still need their own checks. This is neither deployment authorization nor production certification. Continue with the first DOT or composition demo using that walkthrough's compatible artifact set.

Integrate without duplicating the model

All portable inputs are owned by DOT-Domain: consume its generated `Schema/schema.json`, `Schema/composition-v1.schema.json` and `Schema/models/*.schema.json`. Do not maintain a competing schema, struct model or copied Contract operation catalogue. Executor resolved compositions, locks and runtime state are CLI concerns rather than portable domain inputs.

DOT, Skill and Contract identities are case-sensitive textual IDs; legacy UUID-shaped strings remain accepted. Catalogue DOT IDs use `<name>@<tool-release>`; Contract IDs use `<origin-DOT.id>::<standard>::<name>` with canonical standard spelling. Every DOT must reference at least one meaningful Skill. Anchors use exact DOT-local kebab-case IDs (explicit legacy UUID strings remain readable); Composition ownership identities remain UUIDs. A DOT name or version change therefore changes its catalogue identity; update composition pins and regenerate reviewed locks. IDs are not a replacement for the raw Contract URL plus SHA-256. The integration handoff and executable acceptance profiles document cross-reference, conservative compatibility and runtime boundaries.

Independent onboarding trial

Use this checklist with three developers who did not implement DOT. The targets below are proposed usability goals, not measured results or acceptance claims.

Download the unfilled trial record for each participant. Keep release and current-source attempts separate; record pre-cached assets, maintainer setup, cold-download time, every intervention and cleanup. The template deliberately contains no enrolled participants or measured results. A maintainer's automated rehearsal does not fill these blanks.

1. Give each participant only repository access, a compatible Linux-amd64 host, Docker/Compose and the documentation entry point. Record exact revisions and whether images/Contracts were already cached.
2. Ask them to follow the released quickstart: install, find the DOT, validate, deploy, obtain the expected API response and clean up. Target: ten minutes of hands-on work, recording cold-download time separately.
3. Using the current-source route, ask them to inspect another deployment, explain one Contract/Anchor/wire, introduce an invalid reference, interpret the diagnostic and repair it. Record every undocumented intervention.
4. Check cleanup removed only their test-owned resources. Never remove another participant's workloads, volumes or forwards.
5. Keep redacted receipts, timings, failed commands and documentation gaps. Turn reproducible defects into regression tests; do not count coaching as independent success. No participants have been enrolled or trials performed by adding this checklist.

An independent implementation/conformance exercise is a separate next step: internal test coverage and a successful CLI demo do not establish adoption by another team. Licensing, governance and security-response decisions also need the project owners' approval; this guide does not grant new reuse rights.

Compatibility and migration policy

Select one tested artifact set

A product version alone is insufficient to identify the source-built ecosystem. Use either a released CLI with its release manifest and pinned catalogue, or a DOT-Design commit with its exact CLI/catalogue gitlinks. Record `dotx version --verbose` when reporting a problem. The CLI's release acceptance manifest identifies its test fixtures; it is not a floating recommendation for current `main`.

DOT-Domain is the authority for portable models. Its generated schemas and Go bindings must agree. Composition `apiVersion: dot.mare6g.eu/v1` identifies the input profile, not every runtime capability or CLI command. Locks and runtime state are executor artifacts, not portable domain models.

Current intentional boundaries

Area	Current-source behaviour	Upgrade requirement
Contract standard spelling	Only `OpenAPI`, `AsyncAPI`, `Protobuf`, `GraphQL` are accepted.	Released uppercase values such as `OPENAPI` are rejected, not normalised. Review an explicit document migration and affected locks.

Area	Current-source behaviour	Upgrade requirement
Identifiers	Legacy textual/UUID DOT, Skill and Contract keys remain readable. Current catalogue coordinates use `name@actual-tool-release` and `origin-DOT.id::standard::name`; Anchors use DOT-local keys; Composition UUIDs retain their ownership role.	Renaming a catalogue identity changes references and lock inputs. It does not adopt existing runtime state or establish API compatibility.
Required capabilities	A DOT needs at least one meaningful Skill reference.	Older empty scaffolds need reviewed capability declarations, not guessed IDs.
Anchor identifiers	DOT-local kebab-case IDs and exact legacy UUID strings are accepted. Composition references still pair an instance with its Anchor ID; Composition ownership stays UUID-based.	Java Anchor IDs and references change from `UUID` to `String`; recompile callers. Migrate declarations, bindings, wires and exposed interfaces together, then review new locks. Old UUIDs and display names are never lookup aliases.
Contract ownership	Dependency interfaces resolve an explicit originating DOT; owned declarations cannot borrow a different release's namespace.	Follow the catalogue's `IDENTIFIER-MIGRATION.md` and `CONTRACT-AUDIT.md`. Do not copy a dependency Contract or relabel it as consumer-owned.
CLI commands	`dotx compose inspect` displays the resolved composition; no separate command or authored document is needed to represent that derived view.	Rebuild the CLI and regenerate reviewed locks using the `resolutionDigest` field. Clean up running compositions with their original binary before upgrading; saved state is not migrated or adopted.
Java Composition API	Probe protocols/transports/response matching and materializer choices use typed enums instead of Strings; exclusive choices receive Bean Validation.	Recompile Java clients and use, for example, `AsyncAPIProtocol.MQTT` instead of `"MQTT"` in builders/setters. Valid v1 JSON spellings and reviewed lock inputs remain unchanged. Validate required-property presence before deserialization.
Reference and collection validation	ID-only views validate identity, root declarations cascade payload validation, and dangling service Contracts/ambiguous logical keys are rejected. Root ID-only Contract JSON remains ID-only on a Java round trip.	Repair invalid references and duplicate keys explicitly; no aliases, deduplication or lock repair is performed. Recompile Java callers of the old `setServices(Set)` signature for `setServices(Collection)`.

Area	Current-source behaviour	Upgrade requirement
Lifecycle safety	Unsupported stateful and changed Helm/Vagrant upgrades fail before mutation.	A valid historical deployment does not guarantee admission to an upgrade. Never weaken safety checks merely to reproduce an older invocation.
Service configuration keys	Environment/CDI names, config-file paths, and volume names/paths are unique within each service. Java and CLI semantic validation reject ambiguous keys; the generated JSON shapes are unchanged.	Repair duplicates explicitly rather than relying on aggregation to select a winner. The same key in different services remains valid. Schema-only `uniqueItems` is not property-key uniqueness; use full DOT validation.

The original published CLI `v0.2.5` predates these current-source model and authoring changes. These are not a claim of patch-level backward compatibility. The coordinated DOT-Design, DOT-Domain and DOT-CLI baseline is `0.3.0`; it promotes the reviewed `0.3.0-rc.1` portable model surface without changing its generated schemas or reviewed compatibility corpora. The quickstart selects that release and its exact catalogue pin; historical documentation retains its historical artifacts. This is a new minor version on the pre-1.0 line, not another `0.2.x` patch. Future breaking changes must likewise use a new minor version, rather than silently replacing an existing artifact. Publication does not make arbitrary CLI/catalogue combinations compatible.

Safe migration procedure

1. Record the current binary, repository revisions, compositions, reviewed locks and application-data recovery instructions. Preserve the original files.
2. Use a separate checkout for the target artifact set. Read its migration and Contract-ownership notes before changing IDs, enum spellings or dependencies.
3. Retain stable Anchor IDs for unchanged logical interfaces. `dotx new --force` creates a replacement scaffold; it is not an incremental editor.
4. Run online `dotx validate` against the pinned catalogue once to populate the verified Contract cache outside the DOT bundles. Then use `--offline` for definition validation, `dotx compose validate` and `dotx compose inspect` on the reviewed target inputs. A cold offline cache fails closed; it does not download Contracts or search for a local `contracts/` folder. Cached bytes are rehashed, and a digest mismatch requires investigation, not replacing the pin. Generate a new lock separately and review its differences; never repair the old lock to make it appear compatible with a changed graph.
5. Rehearse the supported lifecycle and actual API behaviour in an isolated, test-owned environment. Do not adopt live resources by renaming identities or editing private state. Stateful application migration remains application-owned.

`dotx compose new` already creates a template-based draft from a reviewed, adjacent-lock-verified composition, with a fresh ownership UUID. It preserves the selected DOT releases and wires; it does not infer a new graph, isolate fixed ports or shared inputs, migrate state, or create a lock automatically. See Create a composition from a reviewed example for preview, offline-cache requirements and explicit lock review.

There is currently **no automatic migration command** or incremental interface editor. Template-based creation is not an arbitrary graph-creation wizard or schema-version conversion. Retain manual review of interfaces, inputs and locks.

Compatibility gates

`task testing:models` is the deployment-free local coding gate: Java validation, serialization and coverage; the all-public-model compatibility corpus; generated schema/Go agreement; CLI race tests and coverage; offline catalogue validation and reviewed locks. It starts no runtimes and dispatches no GitHub workflows. Initial tool bootstrap may require network access. Live adapter and presentation qualification remain separate through `task ci:live` and `task testing:demo`; coding-only checks do not establish runtime acceptance or publish Antora. Existing workflows remain configured for later verification.

`task testing:compatibility` runs the historical Contract/identity boundary tests and the existing generated Composition-model compatibility checks. The frozen Contract declaration comes from the exact catalogue pinned by CLI `v0.2.5`: it is expected to fail the new spelling constraint until an explicit change. Tests preserve its original UUID and payload pin. This is not a claim that the whole historical catalogue is accepted unchanged.

`task testing:compose:acceptance` additionally exercises current compositions, reviewed locks and tamper rejection. Authoring serialization oracles protect field ordering, omission and payload bytes. None of these gates starts a production producer or proves independent-adopter acceptance.

Before a release, run the full applicable local workflow equivalents, require all mandatory remote jobs to succeed, verify reproducible packages and downloaded binaries, and retain the exact accepted artifact set. Queued, cancelled, skipped or running checks are not passing required gates.

After 1.0, backwards-incompatible portable document changes require a new public model/profile version and a migration path. Before 1.0, breaking product changes require a minor-version increment and explicit compatibility notes; patch releases remain backwards-compatible except documented fail-closed security/correctness fixes. Deprecation notes must identify the old behaviour, replacement and removal release before removal, unless an immediate safety fix requires rejecting the operation.

Guided demo

This guided demo explains how a reusable DOT becomes part of a checked composition. Start with the one-command guided experience, then explore the official ecosystem demo, choose the shorter 10-15-minute standards route, or the five-minute read-only stages later on this page. These paths connect authoritative Contracts, Anchors, deployment choices and explicit wires; they prove different things.

The read-only core path takes about five minutes on an existing checkout. It is read-only apart from temporary files created and removed by the CLI. The live presenter route uses Docker and Kubernetes and performs deployment, protocol probes, inspection and cleanup.

> ❗ Run the commands from the DOT-Design repository root. Keep evidence levels separate: a valid resolved composition proves deterministic resolution, a dry-run proves that the executor can render supported runtime actions, and only a live probe proves the observed protocol exchange.

Choose a path

For MARE use cases, start with the PoC composition walkthroughs: real PoC4 monitoring, PoC10 authenticated policy enforcement, then a labelled PoC6 CACAO/OpenC2 rehearsal using an NWDAF double. Its presenter tasks add a bounded post-verification inspection pause and automatic cleanup, with the same live assertions used by acceptance.

For a fully executable local presentation spanning Docker, Kubernetes, and OpenAPI/GraphQL/Protobuf/AsyncAPI wires, follow Docker/Kubernetes four-Contract composition. It includes the rehearsal, real probe outcomes, cleanup, and evidence boundaries.

Path	Time	Demonstrates	Runtime mutation
One-command guided experience	At your pace; actual startup time depends on the host	A real Prometheus API response first, then optional authenticated five-DOT composition	Disposable Docker resources; Kubernetes only if you continue
Official ecosystem demo	15-20 minutes of explanation, plus runtime waits	One DOT on two runtimes, secure policy enforcement and stateless recovery; optional four-standard deep dive	Disposable Docker and Kubernetes resources
Read-only overview	About 5 minutes	Repository validation, a reusable external DOT, 21 cross-standard wires, and a Kubernetes lifecycle rendering	None
Live presenter route	About 10-15 minutes after preparation	Six core concepts, one traced wire, four Contract standards across Docker and Kubernetes, and a real MARE use-case graph	Disposable Docker and Kubernetes resources

Path	Time	Demonstrates	Runtime mutation
New developer	About 15 minutes	The read-only overview plus selected resolved composition details, authoritative Contract facts, endpoint materialization, and evidence boundaries	None
Live acceptance	Depends on image downloads and host capacity	GraphQL, all four finite gRPC shapes, and MQTT, NATS, Kafka, and AMQP 0-9-1 delivery against standards-native test producers and real brokers	Disposable Docker resources

One command, immediate explanation, real feedback

```
task demo
```

Run from this checkout in an interactive Linux-amd64 terminal. This is the on-stage experience, not another qualification suite. This wrapper is available in the current **Next source checkout**, not the immutable Design `0.3.0` tag; it reuses the pinned `0.3.0` CLI and adds no new `dotx` command or portable model. It immediately explains what will happen, checks prerequisites without deployment, and reuses the exact Prometheus quickstart commands from Getting started. It then shows the verified build-information version and a clickable localhost URL while Prometheus is running. Open the URL, explain the result, and press Enter to clean up. Type `d` then Enter at the inspection pause for the underlying deployment command and Contract ID; detailed tool logs stay private rather than flooding the screen. Green result headings respect `NO_COLOR` and plain terminals.

Startup is not a silent wait. The display shows elapsed `mm:ss` time and the current phase: prerequisites, startup, API verification, inspection and cleanup. The composition lists the exact DOT releases from native locked resolution, then reports each DOT as `QUEUED`, `STARTING` and `DEPLOYED`, with `n/total` deployment counts and per-DOT elapsed time. While waiting, it prints a status line every 10 seconds; native rollout changes can appear sooner. Counts describe deployment progress, **not** an estimated percentage, ETA or production readiness. `DEPLOYED` means the deployment command completed; only the separate verified API heading means the live assertions passed. Timer updates pause during inspection so they do not interrupt your input; cleanup resumes visible progress. Raw tool logs, tokens, URLs from private errors and API bodies are not forwarded to the public progress display. Colors are optional; all statuses remain readable in plain text.

After Docker cleanup passes, press `q` then Enter to finish, or simply press Enter to continue with Keycloak, APISIX, OPA, Oxigraph and the OAuth test client. Kubernetes is needed only for this optional chapter. A separate read-only preflight checks Ready Docker-driver Minikube, exact cached images, namespace and port conflicts before it deploys anything. The node may use either Docker or containerd; the Docker **driver** and the node's **container runtime** are separate choices. Preflight detects the runtime from the validated Linux-amd64 node and inspects its actual image cache, not a substitute host cache. Unknown or missing runtime metadata fails closed. The short results display explains anonymous/invalid-token rejection, authenticated reads, forbidden writes and the controlled `200 → 403 → 200` policy transition. Type `d` then Enter for the native `dotx compose` `topology` graph and status command. Tokens and raw API payloads are never printed by the guided display. The protected localhost URL must reject anonymous browser access; it is not a broken demo or a public unauthenticated UI.

No long command needs to be copied. Enter requests cleanup at each inspection pause; Ctrl-C, EOF and the bounded timeout also trigger the original harness's ownership-safe cleanup. Inspection defaults to 600 seconds, configured by the existing `DOTX_POC_PRESENT_TIMEOUT` (1–1800 seconds); the optional chapter choice also finishes after 600 seconds or EOF, when no demo workloads remain. The command stops on prerequisite, API or cleanup failure. It does not claim success from a compatible graph, a running container, or an earlier receipt. Private diagnostic logs are retained under `DOT-Testing/e2e/target/guided-demo/run-*/`; do not publish raw logs. The terminal-driven CI rehearsal retains its public console transcript separately from those private tool logs. CI prepares the optional secure-API image cache offstage before exercising the complete interactive route; `task demo` itself still checks prerequisites without installing tools or building images on stage. Explicit cache preparation pulls only the selected digest-pinned images into their target runtime (Docker or containerd's CRI image service), then rechecks the cached identity and Linux-amd64 platform. Containerd verification also requires the requested manifest/index digest in its repository digests; that digest is not the image configuration ID. Read-only preflight never pulls or silently falls back to another runtime when an image or its metadata is missing.

The quickstart's port lease coordinates our test harnesses, not unrelated host processes. If another socket takes the selected port before Docker binds it, the run fails closed and cleans up its own resources. The guided display reports a host-port conflict without exposing the raw error; check listeners and retry, or set `DOTX_HTTP_PORT` to a free port. Never stop an unrelated listener to make the demo pass. A failed control run must be repeated, not counted as a successful signal or API test.

Prepare once offstage

For the complete route, use the existing qualification and preparation instructions and run `task testing:demo` before the presentation. That installs the aligned source CLI, prepares caches, qualifies the full matrix and cleans up. `task demo` does none of that installation, building or full-matrix testing on stage. It is **not** an offline guarantee: real deployment/probe paths may still require registry or Contract-server access. Keep the qualified source set, CLI and runtime contexts unchanged.

For a separate resilience rehearsal, use the OPA outage/restart qualification. It verifies fail-closed access, restart recovery and injected-failure cleanup offstage; the short interactive route never stops a dependency unexpectedly.

For the Docker-only first chapter, only Bash, Python 3, Task, the pinned source `dotx`, Git/Go (for provenance), jq, curl, Docker/Compose, timeout, awk, sha256sum, ss and flock are needed, with the selected Prometheus image and verified Contract already cached. Java, Kubernetes, Helm and the four-standard fixture builds are not prerequisites. The small display intentionally leaves deeper concepts until there is a working service to explain.

This first chapter has no wires. The optional composition is the existing source-adapted PoC10 slice, including an Oxigraph substitution and synthetic OAuth client—not acceptance of the whole producer PoC or every catalogue DOT.

Official ecosystem demo: reuse, compose, enforce, recover

For the simplest first experience, use `task demo` above; it starts with Docker only and offers composition after the first verified API. Use this route for the deeper ecosystem presentation. Budget 15-20 minutes of explanation for the first three chapters; provisioning, readiness and the intentional failed rollout add runtime-dependent waiting. The fourth chapter is an optional standards deep dive. For a short slot, show chapters 1 and 2. The shorter standards-focused route remains available below.

The story is: **one authoritative definition** → **explicit composition** → **observed behaviour** → **controlled recovery**. DOT-Domain and its generated Schemas define the portable models; `dotx` resolves and executes them. Docker/Kubernetes remain the workload engines. A wire connects declared Anchors; it does not magically translate protocols or implement missing producer functionality.

For questions about the rest of the catalogue, use the catalogue readiness guide and the generated report for the selected revision. Definition validity, declared interface completeness and live producer acceptance are separate; the rehearsed chapters do not qualify every catalogue entry.

Prepare and qualify one exact checkout

Run from the DOT-Design checkout root on Linux amd64, with repository access, Git, Go, Task, Python 3, Bash, jq, curl, `ss`, Docker/Compose, kubectl and a Ready local Docker-runtime Minikube node. Select context `minikube`. This advanced developer rehearsal installs the pinned source CLI. The selected release `v0.3.0` includes these model and authoring features, unlike the original `v0.2.5`. Nevertheless, do not replace the source-built binary after qualification, even with the same version: the exact checkout revisions and binary SHA-256 identify the frozen, qualified source set. This is the supported Linux-amd64 profile, not certification of every MARE producer or deployment. Put Go's executable install directory (`GOBIN`, or `$(go env GOPATH)/bin` when unset) on `PATH`; the installation phase verifies that plain `dotx` resolves to the aligned version and records any PATH warning in its printed install log. The simpler Getting started path still needs neither Go, Task nor Kubernetes.

Commit reviewed source changes before using these qualification commands. Choose unused namespaces and ports; keep the same overrides for preflight, rehearsal and presentation:

```
export DOTX_PORTABILITY_NAMESPACE=dotx-demo-portability
export DOTX_PORTABILITY_KUBERNETES_PORT=19083
export DOTX_POC10_NAMESPACE=dotx-demo-secure-api
export DOTX_POC10_FORWARD_PORT=19010
export DOTX_UPGRADE_NAMESPACE=dotx-demo-upgrade
export DOTX_UPGRADE_PORT=19081
task testing:demo
```

`task testing:demo` is the complete off-stage readiness command. Sequentially, it installs and verifies the aligned source CLI, runs the existing demo guards and locked validation/rendering, prepares all image caches with preflight, qualifies all seven live paths twice plus the four official presenter chapters, and exports sanitized fallback evidence from that exact successful receipt. It stops at the first failure; an older successful receipt is never substituted. Every lane is included regardless of a focused `DEMO` selector; namespace and port overrides still apply. Allow substantial runtime for builds, rollouts and cleanup; this is not a command to start during the talk. It needs the tools and already Ready Minikube listed above; it does not install system prerequisites. Successful qualification cleans up its test-owned workloads rather than leaving the presentation running. Start the chapter-specific presenter commands below when ready to show them. Keep sources and `dotx` unchanged throughout the run. The console explains five numbered phases and each live lane's purpose, reports elapsed time and outcome, and finishes with a results table, exact evidence and fallback paths, and the next presenter commands. Detailed tool output stays in per-step logs rather than flooding the terminal. The umbrella's own report and table live under `DOT-Testing/e2e/target/demo/run-*/`; preparation and live qualification retain their separate evidence scopes below. A failed or interrupted phase stops the run and names its logs; it never prints demo-ready success or exports a fallback from an older run.

After qualification, use `dotx version --verbose` and the exact root `summary.json` printed by this run to show the three checkout SHAs and binary SHA-256. Expect `Modified worktree: false`. Do not replace this identity evidence with the version label alone, an older receipt, or a newer unrehearsed checkout. Changes after qualification require a fresh rehearsal and fallback export.

The granular commands remain available for investigation:

`prepare` explicitly pulls only selected digest-pinned official images, builds and tests the standards fixtures from this checkout, and imports their provider image. It may start disposable image-test containers, but never applies a composition or provisions credentials. Fixture preparation shares the live harness's concurrency guard and label-scoped failure cleanup. Image caches and build cache intentionally persist. Registry access is needed; preparation does not promise offline operation. The command ends with read-only preflight. Use `DEMO=portability`, `upgrade`, `secure-api`, `monitoring` or `four-contracts` with either preparation or preflight for a focused lane; the default is all five.

`preflight` does not pull, build, deploy, adopt or delete resources. It checks the installed CLI provenance, local Contracts/locks, Ready Minikube, selected image caches and relevant namespace/port/runtime conflicts. It reserves nothing. The Compose comparison also needs the descriptor-declared port `18919` free.

`rehearse` runs the quickstart, monitoring, portability, secure API, upgrade/rollback, OAuth gateway and four-Contract paths **twice each**, followed by injected-failure and signal cleanup controls. It preserves every existing live assertion rather than replacing tests with a summary. All three checkouts must be clean; before/after revisions, tracked/untracked fingerprints and the binary SHA-256 must remain identical. Do not edit sources or reinstall `dotx` during it. This is qualification before the talk, not a command to start on stage.

`qualify` runs that same complete live rehearsal, then drives all four official chapters through a real terminal: wait for the verified inspection pause, inspect native status and topology, press Enter, and verify cleanup. It reuses the existing live harnesses and terminal driver rather than another deployment implementation. Application-result expectations are shared by both paths. Interrupted, timed-out and rejected-evidence steps remain visible as failures, even when the child command itself exited zero. The existing standalone `rehearse` Task remains useful when only noninteractive acceptance is wanted.

For the optional four-Contract chapter, use the prepared fixture path on stage:

```
export DOTX_DEMO_PREPARED=1
```

Preparation records source fingerprints, the installed binary hash, both Docker fixture image IDs and the imported Minikube provider image ID. `qualify` enables this mode automatically. With this variable set, the four-Contract live/presenter harness verifies that receipt before deploying and does **not** build, pull, import images or start Minikube. Missing preparation or changed sources/binary/images fail closed: re-run explicit preparation off stage and qualify again. Official digest-pinned images and probe-runner caches must still be available; this is not an air-gapped operation guarantee or a promise against registry outages. Do not retag images or switch Docker/Minikube contexts during the talk. Without this opt-in, the existing source-building live test path remains available; builds now have bounded five-minute timeouts.

The printed `summary.json` and adjacent `results.txt` consolidate step exits, logs and hash-linked child receipts/results. Receipt-backed live lanes require successful assertions, exact CLI identity and `cleanup=PASSED`; prerequisite, cleanup-control and log-only quickstart evidence stay separately labelled. Preparation has its own report under `DOT-Testing/e2e/target/preparation/` and explicitly claims **no live API acceptance**. Qualification reports live under `DOT-Testing/e2e/target/rehearsal/`. A missing/failed receipt or source change fails the complete rehearsal. These local evidence directories are not published as though they were current online service health.

Chapter 1: reuse one DOT on two runtimes

```
task testing:demo:portability:present
```

Say: both nodes use the same Prometheus release ID, official OpenAPI Contract and immutable image, with two selected deployment mechanisms. There are **zero wires** between these independent servers: this proves reuse, not replication/migration. **Show:** both build-information API versions match the selected image, `GOGC=75`, the declared default configuration and a real self-scrape `up=1`. At the verified pause, compare the runtime observations and the localhost endpoints in the table. Press Enter and require cleanup to pass before chapter 2. See the complete comparison and storage boundaries.

Chapter 2: compose and enforce a real API boundary

```
task testing:pocs:present:secure-api
```

Say: identity, gateway, policy engine, RDF service and client are independently reusable DOTs, connected by reviewed Anchors/wires and deployment-instance inputs. Existing Contracts remain authoritative where present; explicit interfaces are not misrepresented as vendor-official OpenAPI documents. **Show:** anonymous/invalid-bearer rejection, token acquisition and authenticated SPARQL access to stored RDF. The controlled policy change gives `200 → 403 → 200` and restores the original policy without changing the stored RDF data. Tokens stay out of printed results; native Secret references are inspected by name only. This is the source-adapted portable PoC10 slice, not acceptance of every producer or of the complete MARE security architecture. Enter verifies cleanup. See PoC walkthroughs and their remaining gaps.

Chapter 3: inspect a change and recover safely

```
task testing:demo:upgrade:present
```

Say: reuse the canonical OPA DOT; the change belongs to stateless composition configuration. **Show:** the native upgrade diff, successful stricter policy, deliberately invalid Rego, failed rollout and automatic last-known-good recovery. The recovered version is `1.1.0`, not the original `1.0.0`; a fresh API decision must pass with no pending transaction before the presenter pause appears. This does not demonstrate an OPA software-version upgrade, database migration or universal stateful rollback. Enter verifies cleanup. See the native lifecycle safety boundary and recovery.

Optional chapter 4: cross-runtime, four-standard interoperability

```
task testing:demo:docker-kubernetes-four-contracts:present
```

Show: real OpenAPI/HTTP, GraphQL/HTTP, Protobuf/gRPC and AsyncAPI/MQTT exchanges from the actual Docker consumer's network boundary to Kubernetes. A controlled HTTP-200 value mismatch is rejected, changes the consumer decision, and is then restored and freshly verified. Mosquitto is real; the provider and consumer are labelled standards-native fixtures, **not** production producer acceptance or MARE analytics. See the protocol and network details.

Close with evidence, not a blanket readiness claim

At every pause, the Mermaid graph is the **locked composition**. The new runtime table is a timestamped observation; a running container or available controller is not API acceptance. Historical probe records keep their own timestamps and age. The final application tables come from the separately completed live assertions. Use the printed cache/Kubernetes context for further inspection. Never substitute a fresh cache when recovering an interrupted run. Enter, Ctrl-C, EOF and timeout take the ownership-safe cleanup path; never delete unrelated resources or force namespace finalizers. Require the cleanup receipt, not just an empty terminal.

Keep matching rehearsal reports as a clearly labelled fallback. The umbrella `task testing:demo` automatically exports the sanitized audience summary and prints its path. When using the granular `testing:demo:qualify` task instead, export it manually using the exact printed receipt path:

```
task testing:demo:fallback RECEIPT=/absolute/path/to/summary.json
```

The exporter requires complete, clean-source live and presenter qualification and checks linked artifact hashes. It writes only `fallback/summary.json` and `fallback/README.md` beside that receipt: fixed step outcomes, source revisions and binary/receipt hashes. It does not copy logs, arbitrary API fields, host paths, runtime state, tokens or kubeconfig. Keep the parent run directory private. The exported result is historical rehearsal evidence, not live service health or an independent adopter/producer certification. Do not replay it as a claimed live result. For the catalogue-wide appendix, run `task testing:contracts:readiness` and consult the verification boundaries. Interface declarations, source-derived Contracts, runtime-only brokers, validation-only fixtures, stubs and producer doubles do not become production-certified through this demo. DOT-Registry and partner/SFC acceptance remain separate integration work.

The existing **Kubernetes and Helm runtime E2E** workflow accepts the optional `full_demo_readiness=true` manual input to run the same `task testing:demo` entry point. Ordinary pushes retain their focused runtime matrix; the long full-demo qualification is explicit. Uploaded reports are redacted evidence, not automatically a public live-service dashboard.

10-15-minute live presenter route

Use this route for an audience new to DOTs. The detailed stages below remain the reference walkthrough; this section is the talk track, not another harness. For a ten-minute slot, shorten the wire inspection and show the MARE graph as the closing example rather than starting a second deployment.

The three messages to leave with the audience:

- Reuse a product definition; put PoC-specific configuration in the composition.
- Keep existing interface standards authoritative; wires are not protocol translators.
- Distinguish a compatible resolved composition from a verified live exchange and production acceptance.

For technical Q&A, add the five-minute developer extension: inspect a real product’s deployment choices and official Contract provenance, demonstrate rejection of an invalid wire, and show where a PoC contribution belongs.

Before the audience arrives

Follow the mixed-runtime prerequisites and rehearse on the same Linux-amd64 checkout, installed CLI, Docker and local Minikube context that you will present. Install this checkout’s CLI for the exact pinned acceptance suite. Published `v0.3.0` includes native topology and the CLI operations used by the presenter harnesses; a release binary does not include later source fixes, and the version string alone does not identify the tested binary. Record the three checkout revisions as shown in that walkthrough.

For the Keycloak/APISIX/OPA/SPARQL follow-up, use `task testing:demo:preflight DEMO=secure-api` before deployment. This read-only gate checks the exact installed CLI revision, local Contracts, cluster readiness, cached images and namespace/port conflicts. Follow the secure-API preflight instructions to keep the same overrides in the presentation terminal. Select `DEMO=four-contracts` for the mixed-runtime fixture, or `DEMO=monitoring` for node-exporter/Prometheus. Select `DEMO=portability` or `DEMO=upgrade` for the additional comparison/recovery lanes. Without a selector, all five are checked. These are point-in-time prerequisite checks, not live acceptance or resource reservations. In particular, four-Contract preflight checks cached images, Docker resource conflicts and selected NodePorts, not end-to-end gateway routing.

Pre-download/build the images by completing the existing live acceptance task before the talk, and require successful cleanup:

```
task testing:demo:docker-kubernetes-four-contracts:live
```

The timing below is a speaking budget, not a deployment-time guarantee. Cold downloads, image builds and a slow cluster can exceed it. Keep the previously recorded, revision-matched rehearsal evidence available and label it as such if the live run fails; never describe a resolved composition or saved result as a new live pass. Use an isolated demo network: the example's plaintext NodePorts and anonymous MQTT are not safe public production defaults. Do not run competing acceptance or cleanup tasks during the presentation.

Minutes 0-2: the problem and the six concepts

Say: "We want to describe a component once and reuse it across deployments and MARE use cases. DOT-Design connects existing interface standards to explicit deployment and composition choices; it does not replace those standards."

Concept	Audience meaning	In this demonstration
DOT	Reusable software component description	Provider, consumer and Mosquitto use canonical catalogue definitions.
Contract	Authoritative description of an interface	OpenAPI, GraphQL SDL, Protobuf and AsyncAPI describe different interactions.
Anchor	Named provided or required interaction	The provider and consumer select the REST alerts operation from its Contract.
Wire	Connection between compatible Anchors	`rest-alerts` connects the provider's Anchor to the consumer's Anchor.
Deployment	How a DOT runs on a selected platform	Provider and broker run on Kubernetes; consumer runs on Docker.
Composition	Participants, connections and deployment-instance inputs	One locked graph selects those DOTs, deployments, wires and configuration.

Point out the ownership: `DOT-Repository` stores reusable DOT bundles and compositions; `dotx` in `DOT-CLI` validates, resolves and executes them; `DOT-Domain` defines the shared model. `DOT-Design` brings together documentation and executable acceptance harnesses. The synthetic demo graph stays in `DOT-Testing`; it is not a new production product.

Minutes 2-4: read the topology before deploying

```
dotx compose topology DOT-Testing/composition/docker-kubernetes-four-
contracts/scenario.yaml --locked --color always
```

Show: three DOTs and four logical wires. Match the diagram's `N` and `W` identifiers to the aligned DOT and connection tables. `CONTRACT` names the interface standard; the separate `PROTOCOL` column names the transport protocol. Explain provider versus consumer and the selected runtime, rather than reading UUIDs aloud. The separate `T4` connection identifies MQTT's broker transport endpoint; it is not a fifth Anchor wire. Arrows mean logical PROVIDES-to-REQUIRES, not packet direction: an HTTP consumer may initiate the request.

Expected observation: the graph is available without deploying anything. Parallel wires may share one arrow, but every wire remains in the table. This is a locked composition view, not a live health display.

Minutes 4-6: trace one wire from Contract to configuration

```
dotx compose inspect DOT-Testing/composition/docker-kubernetes-four-
contracts/scenario.yaml --locked --json |
  jq '.wires[] | select(.name == "rest-alerts") |
      {name, from, to, contractStandard, selector, operationPath,
       compatibility, materializer}'
```

Say: "The OpenAPI Contract owns `GET /alerts`. The Anchors select that operation; the wire names the two participants. The materializer puts the resolved operation URL into the consumer's declared `OPENAPI_YAML_URL` environment variable. We are not copying the API definition into the wire."

Expected observation: `OpenAPI`, the selected operation, provider/consumer Anchor references, `COMPATIBLE` and the environment-variable materializer. The template's `192.0.2.1` address is deliberately not a live target. The existing harness substitutes the actual Minikube address and locks that run-specific graph before applying it. Do not manually apply the template.

Limit: compatibility is checked against declared interfaces. It is not proof of reachability, delivery or application correctness. There is no automatic OpenAPI-to-gRPC-to-MQTT protocol translation.

Minutes 6-11: deploy, verify and inspect the live composition

```
task testing:demo:docker-kubernetes-four-contracts:present
```

This is the existing acceptance path with an inspection pause, not a parallel demo implementation. It deploys through `dotx`, checks readiness and executes probes from the actual Docker consumer's network namespace to Kubernetes. Presenter mode keeps build and deployment chatter in the printed, owner-only `details.log`. The terminal shows the numbered resolved composition, deployment, probe, inspection and cleanup stages; a failure still reports its stage, exit code and diagnostic-log location. Ordinary noninteractive acceptance remains verbose.

Standard	Explain the expected live result
OpenAPI	HTTP 200 plus the expected JSON alert; the running consumer derives `REVIEW_REQUIRED` from its wired endpoint.

Standard	Explain the expected live result
GraphQL	`Query.health` returns `true`.
Protobuf / gRPC	The selected health RPC returns `serving: true`.
AsyncAPI / MQTT	The broker delivers the exact uniquely correlated, schema-valid message after subscription acknowledgement.

Show: the verified pause appears only after all four live assertions and recorded status checks pass. Keep the first terminal at this pause. In a second terminal, copy the exact recovery/status command printed by the harness; it selects this run's private cache and Kubernetes configuration. Inspect the per-wire outcomes and redacted `result.json`, not access tokens or Secrets. The final live-results table separates `CONTRACT`, `PROTOCOL`, `OBSERVED` and `OUTCOME`. It is generated only after each successful probe is also present in recorded lifecycle status; `COMPATIBLE` and running workloads do not populate it. The observations summarize checked assertions, not raw API responses. The same four rows are saved in `result.json` and the plain-text `results.txt`. Before the pause, point out the controlled negative step: the owned fixture temporarily returns HTTP 200 with severity `low`, the native assertion records `RESPONSE_MISMATCH`, and the running consumer changes its decision. The harness then restores the critical alert and requires a newer `REVIEW_REQUIRED` receipt and successful native assertion. This distinguishes endpoint health from correct application data; it is not a claim of complete schema or policy validation. The graph remains the locked composition, not an inferred network map. Presenter lanes using the shared lifecycle helper, including the four-Contract lane, append timestamped runtime observations with `compose topology --locked --status`. Their results tables still provide the separate application assertions.

For an applied composition, run the following from the same private cache and Kubernetes configuration printed by its harness (do not use another run's state):

```
COMPOSITION=DOT-Repository/Compositions/keycloak-apisix-opa-sparql/composition.yaml
dotx compose topology "$COMPOSITION" --locked --status
```

Explain the layers separately: `APPLIED` is a recorded lifecycle phase; `RUNNING` describes a process, container or VM; `CONTROLLERS_AVAILABLE` describes selected Kubernetes controller status, not rollout or API acceptance. An empty selection is `NO_CONTROLLERS`; query failures are `UNKNOWN` and return nonzero. Historical wire probes show their original timestamps. `RECENT_RECORD` means recently recorded, never a fresh API call; `STALE_RECORD` requires a fresh probe before claiming current acceptance. The default record age boundary is five minutes, adjustable with `--probe-max-age 2m` (positive, at most 24 hours). This observation mode does not deploy, mutate configuration or run probes; it uses the lifecycle coordination lock. Mermaid export remains declaration-only.

State the boundary: Mosquitto is a real external broker, but provider and consumer are test-only DOTs with standards-native fixture Contracts, not vendor-official or MARE producer APIs. The consumer now executes the REST exchange and derives a synthetic review decision; explicit probes exercise GraphQL, gRPC and MQTT from its network namespace. The MQTT probe subscribes and publishes to prove broker delivery, not a provider application's business-event publication. This verifies interoperability mechanics across runtimes, not a production application or the whole MARE stack.

Minutes 11-13: relate the mechanism to a real MARE use case

Keep the live demo paused and show this separate, read-only catalogue graph:

```
dotx compose topology \
  DOT-Repository/Compositions/keycloak-apisix-opa-sparql/composition.yaml \
  --locked --color always
```

Say: "The same model expresses a PoC10-derived protected data service: Keycloak supplies identity, APISIX gates access, OPA decides policy, Oxigraph serves SPARQL and a client obtains a token and requests the protected capability." Point to the five wires, then explain that this graph has not been deployed by the four-Contract demo.

For a longer session, use the existing `task testing:pocs:present:secure-api` in a separate, rehearsed run. Its live checks require HTTP `401` for missing and malformed bearer tokens, authenticated POST `403`, and an authorized GET reading the stored `mare-poc10` value. See MARE PoC composition walkthroughs for prerequisites and commands. This is source-adapted evidence: Oxigraph substitutes for GraphDB, the client is synthetic, and SPARQL/OPA use explicit protocol interfaces rather than fabricated official OpenAPI Contracts. It does not accept the complete producer deployment, LDAP, telemetry or automated response.

Minutes 13-15: clean up and leave a reproducible next step

For deployment reuse, the Prometheus portability lane runs one canonical DOT and official Contract on Docker Compose and Kubernetes, with the same pinned image and fresh API/self-scrape checks. It compares two independent servers; it does not claim a wire, replication or data migration.

For a separate technical follow-up, the stateless upgrade lane reuses the canonical OPA DOT, changes a policy configuration through native `compose upgrade`, and verifies automatic last-known-good recovery after a deliberately broken revision. Rehearse it separately; do not fit its intentional rollout timeout into this 10–15-minute route or imply application-data rollback.

Return to the paused demo terminal and press Enter. Expect a receipt with `cleanup=PASSED` and `exit=0`; the harness checks its owned resources for leftovers. Ctrl-C also invokes cleanup and exits `130`. EOF, timeout and SIGTERM are non-success exits with cleanup, not successful acceptance. The inspection pause defaults to ten minutes; see the mixed-runtime walkthrough for its bound. If cleanup fails, retain the receipt and use that walkthrough's recovery instructions. Never delete unrelated namespaces, containers or volumes.

Close with: "A valid definition is not a running workload; a compatible resolved composition is not a successful exchange. We have shown the live results and their limits."

Direct attendees to Getting started to deploy real Prometheus, the reusable PoC walkthroughs to explore compositions, and Anchors and Composition to author their own integration. Keep the verification matrix available for questions; do not equate catalogue validity with production readiness or claim DOT-Registry REST API or joint SFC acceptance.

Audience FAQ

Why not just use Docker Compose, Kubernetes or Helm?

Those tools remain the native deployment mechanisms. DOT adds a reusable component description, authoritative interface references, named interactions, explicit cross-component wiring and a checked resolved composition. `dotx` delegates supported deployment actions rather than replacing those platforms. If an existing Compose file is all a use case needs, DOT's additional value is the shared catalogue and interoperability description, not a different container scheduler.

What is reused, and what changes between PoCs?

The product's canonical `dot.json`, Contracts and Anchor identities are reused. A composition selects participants and deployments and supplies use-case-specific configuration: routes, policies, scrape targets and credential references. There is no need to create another Keycloak DOT for every PoC. Runtime platform means *where* it runs; deployment mechanism means *how* it is deployed.

Does connecting two Anchors make incompatible APIs work together?

No. Wires connect declared compatible interactions; they do not translate OpenAPI, GraphQL, gRPC or MQTT automatically. An application-specific adapter, if required, must be implemented and described explicitly. The four-Contract demo verifies four separate standards-native exchanges, not one message automatically converted through four protocols.

Is a Contract the same as a protocol? Are all Contracts official?

No. OpenAPI is an interface-description standard; HTTP is a protocol used by the selected operation. Protobuf describes messages/services used by the gRPC profile; AsyncAPI describes messaging interactions and transport bindings. The topology table separates `CONTRACT` from `PROTOCOL` for that reason. Use vendor/project-official documents when available. Source-derived Contracts and test-fixture Contracts must be labelled, not advertised as official. An explicit protocol interface is not an invented official Contract; the topology labels that case `explicit interface`. A `-` on a transport-only `T` link does not create another Contract-backed Anchor wire.

Does validation mean a DOT is production-ready?

No. Base validation checks definitions and declared references; stricter interface checks assess declaration completeness. A locked compatible resolved composition checks the selected graph. A successful deployment establishes a runtime state. A live probe verifies a particular observed exchange. None alone proves full business functionality, resilience, security hardening, scale or acceptance of an unmodified PoC producer. Read the declared scope with each result.

What does the lock file guarantee?

It pins the composition's portable definition inputs and resolved composition so that unreviewed definition changes are detected. It does not freeze runtime health, network addresses, credentials or upstream service behavior. Contract payload digests verify the referenced bytes; a downloadable URL is not itself evidence that the live implementation conforms to the whole Contract.

Where do credentials and configuration belong?

DOTs declare supported service configuration. Compositions provide deployment inputs and secret references; secret values and concrete runtime state are not part of a reusable public interface description. The PoC10 rehearsal uses native Kubernetes Secret references. Do not display Secret contents, tokens, raw deployment state or private diagnostic logs during the talk. The four-Contract fixture uses plaintext/anonymous demo settings and is not an example of production security hardening.

What did this presentation actually demonstrate?

The official route verifies one Prometheus DOT on two runtimes, a source-adapted secure API with policy denial/restoration and unchanged stored RDF, and native stateless OPA upgrade/recovery. The optional standards chapter uses test-only provider/consumer components and real Mosquitto to verify four exchanges from the consumer’s network namespace. The shorter standards-focused route shows the PoC10 graph only unless its separate live task is run; a topology alone never accepts it. Producer doubles, stubs, validation-only fixtures and source adaptations remain labelled. DOT-Registry REST API, complete producer/PoC acceptance and joint SFC/partner acceptance are not claimed by these demonstrations.

How can an attendee try this afterwards?

Start with the real Prometheus quickstart, which needs neither a Java/Go development toolchain nor Kubernetes when using the supported released binary. Then explore reusable compositions and the authoring guide. The new topology and presenter features require the installed development checkout described above; do not assume an older release contains them.

The story in one diagram

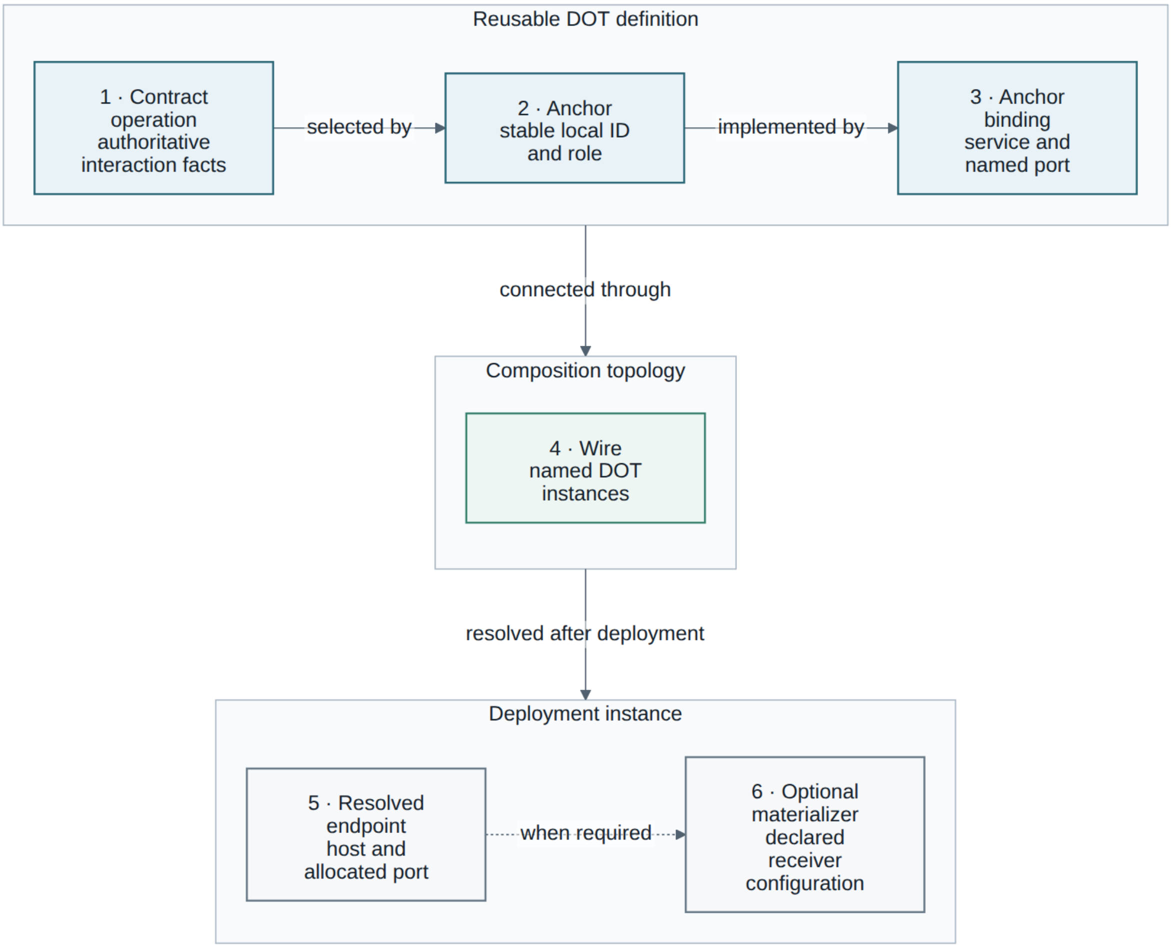

Reusable DOT definition
1 · Contract operation authoritative interaction facts
selected by
2 · Anchor stable local ID and role
implemented by
3 · Anchor binding service and named port
connected through
Composition topology
4 · Wire named DOT instances
resolved after deployment
Deployment instance
5 · Resolved endpoint host and allocated port
when required
6 · Optional materializer declared receiver configuration

Figure 2. From an authoritative Contract operation to a deployed connection

The Contract owns protocol semantics. An Anchor gives one selected Contract operation a stable logical identity. A deployment-service binding identifies the service port that implements the Anchor. A composition wire connects two named DOT instances and may materialize the resolved endpoint into configuration already declared by the consumer.

Prepare the checkout

For a first deployment with the released CLI, follow Getting started. This presenter path uses a full checkout from Contributor guide with initialized `DOT-CLI` and `DOT-Repository` submodules. The demo needs `dotx`, `jq`, and network access when URL-backed Contracts are not already cached.

```
git submodule update --init --recursive
task cli:install
dotx version
```

Source builds use `DOT-CLI/VERSION`, which may advance independently of the published release. The published release reports:

```
dotx 0.3.0
```

The independently released Linux-amd64 binary, its SBOM, release manifest, and checksums are also available from the DOT-CLI v0.3.0 release. Use `sha256sum --check SHA256SUMS` before executing a downloaded release asset.

Stage 1: validate the authoritative catalogue

```
dotx validate DOT-Repository
```

This recursively discovers every `dot.json`; there is no separate inventory. The command validates schema shape, identities, Skills, Contract payload integrity, selectors, service ports, Anchor bindings, repository placement, and catalogue-wide uniqueness. Contract-backed external and internal bundles also receive the strict reusable profile.

What this proves

The checked-out catalogue is internally consistent and every declared reference resolves. It does not deploy a workload or claim that a third-party service is reachable.

Stage 2: inspect a reusable external DOT

Keycloak is a useful example because its canonical DOT uses official, digest-pinned Contracts while offering multiple deployment mechanisms.

```
KEYCLOAK=DOT-Repository/DOTs/external/Keycloak-DOT
dotx inspect "$KEYCLOAK" --json |
  jq '{name, version,
       skills: (.skills | length),
       contracts: (.contracts | length),
       anchors: (.anchors | length),
       deployments: (.deployments | length),
       supportedDeploymentSystems}'
```

The current source catalogue produces this compact projection:

```
{
  "name": "Keycloak",
  "version": "26.8.1",
  "skills": 2,
  "contracts": 3,
  "anchors": 4,
  "deployments": 5,
  "supportedDeploymentSystems": [
    "DOCKER_COMPOSE",
    "DOCKER_IMAGE",
    "HELM_CHART",
    "KUBERNETES_MANIFEST"
  ]
}
```

What this proves

One reusable DOT root can describe several supported deployment choices without forking its identity for each PoC. Deployment-instance realms, routes, credentials, and endpoints remain composition inputs rather than copied DOTs. All five deployment choices target the same Keycloak 26.8.1 release in this one definition. Git preserves previous releases; there are no sibling version bundles.

Stage 3: resolve the cross-standard composition

The canonical Contract matrix connects six DOT instances with 21 compatible wires. Its committed lock pins every DOT, Contract, configuration input, and portable resolved composition.

```
dotx compose lock \
  DOT-Testing/composition/contract-standards/scenario.yaml --check

dotx compose validate \
  DOT-Testing/composition/contract-standards/scenario.yaml --locked

dotx compose inspect \
  DOT-Testing/composition/contract-standards/scenario.yaml --locked --json |
  jq '{apiVersion, name, version,
       dotInstances: (.dots | length),
       wires: (.wires | length),
       compatibleWires: ([.wires[] |
         select(.compatibility == "COMPATIBLE")] | length),
       standards: ([.wires[].contractStandard] | unique),
       interactions: ([.wires[].providerInteraction] | unique)}'
```

Expected summary:

```
{
  "apiVersion": "dot.mare6g.eu/v1",
  "name": "contract-standards-matrix",
  "version": "1.0.0",
  "dotInstances": 6,
  "wires": 21,
  "compatibleWires": 21,
  "standards": ["AsyncAPI", "GraphQL", "OpenAPI", "Protobuf"],
  "interactions": [
    "BIDIRECTIONAL_STREAM", "CLIENT_STREAM", "MUTATION", "QUERY",
    "REQUEST_RESPONSE", "SEND", "SERVER_STREAM", "SUBSCRIPTION"
  ]
}
```

The project gate asserts the complete result rather than only printing a summary:

```
task testing:compose:contract-standards
```

What this proves

Every provider and consumer selector resolves against its authoritative Contract; all 21 pairs are compatible; endpoints, interactions, probes, and materializers can be derived without copying standard-specific semantics into Anchors.

Follow one AsyncAPI wire

Inspect the Kafka wire to see how standard-derived facts and deployment facts remain separate:

```
dotx compose inspect \
  DOT-Testing/composition/contract-standards/scenario.yaml --locked --json |
  jq '.wires[] | select(.name == "asyncapi-kafka") |
      {name, compatibility, contractStandard, contractFormat,
       providerInteraction, consumerInteraction, channelAddress,
       connectionEndpoint, materializers, probe}'
```

The AsyncAPI Contract supplies the complementary send/receive operations, channel, message schema, Kafka binding, and correlation field. The composition selects the broker instance and materializes its resolved address and topic for the consumer. A resolved composition still does not claim that a message was delivered.

Stage 4: render a lifecycle without mutating it

The APISIX, Keycloak, and OAuth-client scenario demonstrates dependency order, configuration materialization, native Kubernetes Secret references, rollout readiness, and an exposed Anchor:

```
dotx compose lock \
  DOT-Repository/Compositions/keycloak-apisix-oauth/composition.yaml --check

dotx compose apply \
  DOT-Repository/Compositions/keycloak-apisix-oauth/composition.yaml \
  --locked \
  --namespace dot-guided-demo \
  --dry-run \
  --forward protected-metadata-through-gateway=19080
```

The rendering ends with:

```
Dry run complete; composition state was not written.
```

The command may create bounded temporary rendering files and removes them when it exits. It does not create Kubernetes resources, start a port forward, or write composition state. The rendered order is Keycloak, APISIX, then the OAuth client, matching the declared readiness dependencies.

What this proves

The reference executor accepts the locked inputs and can derive the exact platform-native actions. It does not prove rollout success, credentials, or HTTP behavior; those require the live acceptance layer.

Stage 5: connect claims to evidence

Claim	Executable evidence	Boundary
Catalogue correctness	`dotx validate DOT-Repository`	Static and Contract resolution evidence; no deployment
Cross-standard interoperability	`task testing:compose:contract-standards`	21 deterministically compatible wires; no live traffic
Live Contract probes	`task testing:compose:contract-probes:live`	Standards-native test producer and real brokers on disposable Docker resources
Runtime configuration portability	`task testing:compose:runtime-configuration`	Dry-run rendering across six adapters
Workload isolation	`task testing:compose:service-workloads:live`	Live repeated Compose, Kubernetes, and Helm instances
Portable-runtime isolation	`task testing:compose:runtime-isolation:live`	Live Vagrant/libvirt and WASI CLI instances
Released-product acceptance	0.2.4 downloaded-binary acceptance	Linux amd64, single-host supported profile

See Verification and examples for the complete test matrix and the distinction between model, resolved composition, dry-run, source-adapted, and live production evidence.

Optional: run the live Contract probes

⚠ This path builds images, starts disposable Docker workloads, and downloads pinned images when absent. Use the presenter path when time, network, or host capacity is uncertain.

```
task testing:compose:contract-probes:live
```

The harness builds a standards-native GraphQL/gRPC producer and an independent consumer, then deploys the canonical Mosquitto, NATS, Kafka, and RabbitMQ DOTs. It proves GraphQL query, mutation verification/cleanup, and GraphQL-over-SSE; all four finite Proto3 `NO_SIDE_EFFECTS` gRPC shapes through reflection; and correlated MQTT, NATS, Kafka, and AMQP 0-9-1 delivery. It records redacted evidence and destroys composition-owned resources on success, failure, or interruption.

Five-minute presenter script

1. Show the Contract-to-Anchor-to-wire diagram and explain the four ownership boundaries.
2. Run `dotx validate DOT-Repository` to establish that the catalogue is the checked source of truth.
3. Inspect Keycloak to demonstrate one reusable product identity with several deployment choices.
4. Print the 21-wire resolved composition summary and highlight the four Contract standards and eight interaction forms.
5. Inspect the Kafka wire to distinguish Contract semantics from its resolved broker endpoint.
6. Render the APISIX/Keycloak/OAuth lifecycle with `--dry-run`.
7. Finish on the evidence table and state the supported boundary explicitly.

Keep the published release notes and verification matrix open as fallback evidence if a local network or runtime is unavailable.

Supported boundary

The stable profile supports Linux amd64 and private single-host composition state. Multi-host orchestration, automatic cross-platform gateway provisioning, general stateful-data migration, AMQPS, AMQP 1.0, native arm64 and non-Linux executables, the DOT-Registry REST API, and joint SFC or partner acceptance are separate activities. Unsupported combinations fail closed rather than being silently approximated.

All eleven MARE PoCs have resolution or source-adapted evidence. That evidence is not presented as acceptance of every unmodified partner workload. Continue with Architecture for system boundaries or DOT Anchors and Composition for authoring and lifecycle details.

Developer demo: reuse and extend

Use this optional five-minute extension after the 10-15-minute presenter route, or during technical Q&A. It adds no new live deployment. Run from the DOT-Design checkout root with its installed `dotx`, initialized catalogue, Bash and `jq`.

The developer takeaway is: **reuse the product definition, describe the real interface, configure a composition, then prove the intended exchange.**

1. One real product, several deployment choices

Ask the audience: "If two PoCs use Keycloak with different realms and deployment platforms, which parts should they share?"

```
KEYCLOAK=DOT-Repository/DOTs/external/Keycloak-DOT
dotx validate "$KEYCLOAK" \
  --schema DOT-Repository/Schema/schema.json --profile reusable --offline

dotx inspect "$KEYCLOAK" --json |
  jq -e '{name, version,
          deployments: [.deployments[] |
            {id, platformEngine, type, supported}],
          anchor: (.anchors[] | select(.name == "admin-realm-read") |
            {name, role, contract})} |
         select(([.deployments[].platformEngine] | unique) ==
           ["DOCKER", "KUBERNETES"])'

jq -e '.contracts[] | select(.name == "Admin") |
       {id, name, version, standard, format, payload}' "$KEYCLOAK/dot.json"
```

Show: one `Keycloak@26.8.1` identity with Docker Compose and Kubernetes/Helm deployment choices in one root `dot.json`. Older releases remain in Git history, not nested version bundles. All deployment artifacts must match 26.8.1. PoC-specific configuration stays in composition inputs or named deployments, not a cloned product identity. The selected `admin-realm-read` Anchor refers to the Admin REST Contract and its `GET` selector. The root Contract declaration identifies the release-specific official raw document and SHA-256; source/provenance boundaries remain in the audit. No API definition is copied into the Anchor.

This bundle declares every owned Contract completely in `dot.json`, using `payload.url` for raw HTTPS bytes and `payload.sha256` for integrity. Contracts do not need a local directory. Validate online once before using `--offline`; the CLI then rechecks cached bytes without downloads. Composition inputs name the exact owner, not a copied wrapper. Official documents remain preferred; derived specifications and their acceptance limitations remain labelled.

Explain the boundary: an inspector reporting a supported deployment is not live acceptance of every deployment variant. The PoC10 rehearsal selects pinned native Kubernetes deployments `Keycloak@26.8.1` and `APISIX@3.19.0`, with composition-owned realms, routes and policy. It does not certify every variant or every Admin REST operation. Compare each resolved composition's `dotId`, `dotVersion` and `deploymentId` when explaining provenance.

2. Show the guardrail, not just the happy path

```
task testing:demo:developer-tour
```

The task repeats the short inspection, validates the existing two-DOT PoC7 validation-only graph, then attempts its existing deliberately rejected wire. Expect:

```
EXPECTED REJECTION (exit=1): scoring-api-to-collector mixes Contract-backed and
contractless Anchors.
The valid two-DOT graph has zero wires; no runtime was deployed by either validation
command.
```

The task itself exits zero only when the positive graph resolves and the negative graph returns that specific compatibility diagnostic. Missing credentials, a broken checkout, a missing CLI or an unrelated validation error are not counted as a successful negative test.

Say: "Putting two boxes on a diagram is not an implemented integration. We cannot make an undeclared collector interface compatible by drawing a wire." At the pinned source boundary, the collector prints flow data rather than calling the scorer. This control demonstrates interface admission, not a live trust-scoring result or a runtime security test.

The first current rejection is Contract-backed/contractless incompatibility; it does not prove that a later undeclared-configuration check ran. After implementing a real consumer, declare its supported endpoint configuration, Anchor and binding, then validate the resulting wire and test the actual call.

3. What a PoC developer changes, and what stays shared

Concern	Owning artifact	Developer action
Product capability and interfaces	The existing DOT bundle, root Skills/Contracts, Anchors and service bindings	Reuse it. Extend the shared definition only when the software genuinely supports an additional capability or interface.
API or messaging semantics	Official Contract, or an explicitly labelled source-derived Contract	Select an existing operation. Do not redefine request/response schemas or invent a consumer implementation in a wire.
Deployment choice	A selected runtime/deployment already declared by the DOT	Choose a supported deployment ID. Add a reusable variant to the product root if a new mechanism is genuinely needed; test that variant.
Realm, route, policy, scrape target or configuration file	Composition inputs and endpoint materializers	Configure the use case without copying or hard-coding the product definition.
Credentials and concrete runtime state	Secret references and the deployment instance	Keep secret values outside reusable public descriptors and presentation output.
Reproducibility and proof	Reviewed composition lock and executable acceptance harness	Inspect the resolved composition, review changed lock inputs, run dry-run checks, then verify the real workload and its cleanup. Commit definitions, not local runtime state.

The practical contribution sequence is:

1. Find the product under `DOT-Repository/DOTs/external/` or `DOTs/internal/`; check its README, source revision and declared gaps.
2. Use official interface documents where available. For MARE-owned code, export or derive the Contract from the implementation and review its limits.
3. Declare the provided/required Anchor and the selected deployment's service binding. Ensure the application really reads the declared configuration.
4. Add or update a reusable graph under `DOT-Repository/Compositions/`, with its README, non-secret inputs and reviewed lock. Put synthetic fixtures and live test drivers under `DOT-Testing`, not beside a new duplicate product.
5. Run `dotx validate DOT-Repository`, inspect the locked composition and deployment dry-run, then add a far-end behavioral assertion and success/failure cleanup.

`Skill` is a required high-level capability reference; it is not an API operation or a command executor. Anchors and authoritative Contracts describe the concrete interoperability boundary.

4. Show the broader potential without extending the live claim

Capability	Existing executable evidence	What to say
Four Contract standards, 21 selected wires	`task testing:compose:contract-standards`	Broader deterministic interface coverage than the four-wire live demo; a compatible resolved composition is not traffic.
GraphQL query/mutation/SSE; finite unary and streaming gRPC; MQTT, NATS, Kafka and AMQP 0-9-1	`task testing:compose:contract-probes:live`	A separate, longer test-producer/broker acceptance suite. Do not start it unexpectedly during a short presentation or claim partner-producer acceptance.
Configuration defaults and overrides across six adapters	`task testing:compose:runtime-configuration`	Docker image/Compose, Kubernetes/Helm, Vagrant and WASI rendering; not proof that every possible DOT runs on every adapter.
Repeated multi-service instances and selective cleanup	`task testing:compose:service-workloads:live`	A separate Compose/Kubernetes/Helm isolation test, not multi-host orchestration.

Choose one row in response to a technical question rather than reading the entire test matrix aloud. The examples demonstrate extension points and tested profiles, not automatic protocol conversion, arbitrary networking, complete PoC production readiness, a DOT-Registry REST API or joint SFC acceptance.

Continue with Anchors and Composition for authoring, Verification and examples for the evidence matrix, and the audience FAQ for short answers.

MARE PoC composition walkthroughs

Start here when presenting DOT composition against MARE use cases. Run commands from DOT-Design with its pinned submodules. These are integration slices, not complete official PoC outcomes. For generic cross-runtime/four-Contract mechanics, use the Docker/Kubernetes demo.

Presenter route: inspect, run, prove, clean up

For a 15-20 minute presentation, show PoC4 first, PoC10 second, and PoC6 only with its adapter/double boundary visible. Avoid new feature changes on the day. Each presenter command uses the same locked composition and live assertions as acceptance; there is no second set of DOT definitions or demo-only success path.

```
# Run from a real interactive terminal, one example at a time.
task testing:pocs:present:monitoring
task testing:pocs:present:secure-api
task testing:pocs:present:remediation
```

Each follows five stages: **inspect topology** → **deploy** → **verify** → **inspect the running result** → **clean up**. The terminal prints resolved DOT versions, deployments, wires and evidence boundaries. It pauses only **after** every API assertion passes. While paused, open the printed Prometheus targets page or inspect the redacted `result.json`; the secure API must still deny anonymous requests. Do not paste access tokens or Secret values into the presentation.

The topology is generated from the locked composition by `dotx compose topology`, using the MIT-licensed `mermaid-ascii` 1.6.1 Go library embedded in the binary. There is no Python renderer or separate executable to install. DOTs appear once, with connected Unicode boxes. Parallel wires share a connection labelled `W1,W2,…`; the connection legend lists every wire, Contract standard and protocol separately. `T` links show broker transport endpoints, not extra Anchor wires or invented application publishers. Aligned tables show full instance names, platforms, deployments and versions, plus each connection's provider, consumer, Contract standard, protocol and name. Long values wrap inside their columns without truncation, using the same width as the graph. Arrows mean logical PROVIDES-to-REQUIRES relationships, not packet direction. The graph reappears after successful live verification.

Interactive diagrams use cyan Kubernetes, magenta Docker and amber connection IDs. Set `NO_COLOR=1` or `DOTX_TOPOLOGY_COLOR=never` for plain output; `always` forces colour when capturing a demonstration (`NO_COLOR` and `TERM=dumb` still disable it). Logs are plain by default. `DOTX_TOPOLOGY_WIDTH=80` controls width (40-240 columns). The renderer tries vertical/compact layouts for narrow screens and fails clearly if it cannot show every connection; it never silently clips.

For all four Contract standards across Docker and Kubernetes, use `task testing:demo:docker-kubernetes-four-contracts:present`; see the mixed-runtime presenter walkthrough.

Press Enter to destroy owned resources and verify cleanup. Ctrl-C also cleans up and returns 130; SIGTERM returns 143. The default inspection limit is ten minutes (`DOTX_POC_PRESENT_TIMEOUT`, integer 1-1800 seconds). Timeout returns 124 and EOF returns 1, both after cleanup: neither is reported as a successful presentation. Headless launches are rejected before deployment, and `DOTX_E2E_KEEP=1` is forbidden in presenter mode.

```
# Safe headless previews: print the real graph and render without deployment.
task testing:pocs:present:monitoring -- --dry-run
task testing:pocs:present:secure-api -- --dry-run
task testing:pocs:present:remediation -- --dry-run
```

The noninteractive `testing:pocs:*:live` tasks remain unpaused. Use the presenter tasks above, not the full failure-injection suite, in front of an audience. For preparation, `task testing:pocs:present:controls` exercises Enter, EOF, timeout and real signals with live Prometheus on overridden port 19041. `task testing:pocs:present:rehearse` requires clean commits, drives all three presenter flows through a real terminal, inspects live status during each pause, then verifies cleanup. It preserves checkout/CLI fingerprints and redacted results under ignored `DOT-Testing/e2e/target/presenter/` and `poc-rehearsals/`. Keep those receipts and the exported PDF as an explicitly labelled fallback, not evidence that a new live exchange occurred during the talk.

Prerequisites and fixed inputs

Use Linux amd64, Docker, curl, jq and Git. Acceptance harnesses also require Bash, flock, Python 3 and Task; ShellCheck checks their regressions. PoC10 needs kubectl and a Ready local Docker-driver Minikube node with context `minikube`. PoC6 additionally needs Docker BuildKit and read access to the pinned private producer through Git/`gh`, or `POC6_SOURCE_DIR` at the exact revision below. Automatically acquired PoC6 source uses the private run directory's filesystem, not shared `/tmp`, and is removed after the run. A caller-supplied source checkout remains untouched.

```
git submodule update --init --recursive
task cli:install
dotx version
git rev-parse HEAD
git -C DOT-CLI rev-parse HEAD
git -C DOT-Repository rev-parse HEAD
task testing:pocs:dry-run
```

To display just the graph without running the acceptance harness:

```
dotx compose topology \
  DOT-Repository/Compositions/keycloak-apisix-opa-sparql/composition.yaml \
  --locked --color always
```

Install from the pinned CLI checkout: released 0.2.4 predates repository-wide composition validation. Dry-runs must pass before deployment. The three paths contain respectively 1, 5 and 2 compatible wires. Inspect their exact Anchor IDs, selected deployments and materializers; `COMPATIBLE` is not a live result.

1. PoC4: Node Exporter to Prometheus

```
Node Exporter -- metrics scrape wire --> Prometheus -- query API --> presenter
```

Graph: `DOT-Repository/Compositions/node-exporter-prometheus-monitoring/composition.yaml`. Producer context: PoC4 `c03eca71a9d87358603a1d89fa0eca58a9dabe03`. Node Exporter provides metrics; Prometheus requires the scrape endpoint. A `CONFIG_FILE` materializer inserts `ENDPOINT_HOST_PORT` at `/scrape_configs/0/static_configs/0/targets/0`. Prometheus's query Anchor uses its DOT-local Contract. OpenMetrics is not a fabricated OpenAPI operation.

```
dotx compose inspect DOT-Repository/Compositions/node-exporter-prometheus-
monitoring/composition.yaml --locked --json
task testing:pocs:monitoring:live
```

The task waits for readiness, verifies the exact resolved scrape target is healthy, checks `up{job="node-exporter"}` equals `1`, and verifies the exporter version through `node_exporter_build_info`. Expect `PASS`, then a receipt with `exit=0, cleanup=PASSED`. Override `DOTX_POC4_QUERY_PORT` if 19040 is busy.

This grants no host namespaces or host root filesystem mounts. It observes the exporter's container, not the testbed host. Full log/dashboard flow, MITM detection and mitigation remain outside this slice.

2. PoC10: authenticated, policy-controlled SPARQL

Presentation preflight

After preparing the local cluster and rehearsing once to cache images, run this read-only gate before presenting. It is scoped to this secure-API composition, not the four-Contract fixture or all catalogue deployments:

```
# Keep these overrides in the same terminal for preflight and presentation.
export DOTX_POC10_NAMESPACE=dotx-poc10-presentation
export DOTX_POC10_FORWARD_PORT=19010
task testing:demo:preflight DEMO=secure-api
task testing:pocs:present:secure-api
```

The preflight requires local Go only to inspect the installed binary's embedded build information. It requires clean CLI/catalogue checkouts and a binary built from the CLI's exact revision, rather than trusting the `0.2.5` version label. If mismatched, review/commit CLI changes and run `task cli:install`. It prints the CLI SHA-256, Design/catalogue revisions, verifies local Contract assets through offline catalogue validation, and resolves the reviewed composition lock.

It then checks Docker/Compose access, the `minikube` context, one Ready Linux-amd64 node without resource pressure, an unused namespace and TCP port, and every selected manifest image in the Docker-driver Minikube cache. All five selected images are digest-pinned at their rehearsed versions; tags remain in the reference for readability. Preflight rejects tag-only references and cached images whose reported platform is not Linux amd64. It prints the resolved image IDs without dumping image configuration. This is cache availability, not a remote registry/Contract URL check, image vulnerability scan, or API acceptance. The check is bounded and does not start tools, pull images, create Secrets, deploy, adopt resources or clean anything up. Read-only CLI resolution can use temporary files/cache; no composition lifecycle state is written.

If a prerequisite fails, correct only that prerequisite and rerun. Prepare the cluster explicitly with `dotx tool check kubernetes --start` if needed. To warm images, run the existing `task testing:pocs:secure-api:live` acceptance task before the talk; it may download images and must finish with verified cleanup. For an occupied port choose another `DOTX_POC10_FORWARD_PORT`. Never delete a foreign namespace to make preflight pass: choose a new name, or recover only a previous run whose ownership is recorded in its receipt. Preflight is a point in time check, not a reservation; the live harness repeats ownership checks.

What the live result proves

The harness now seeds a known RDF triple into the disposable backend, then reads that stored value through the authenticated gateway. Acceptance requires exactly one expected literal, rather than a constant `SELECT` expression that would work with an empty database. An authenticated INSERT attempt must be rejected with 403. Seeding is explicit out-of-band test setup in the owned namespace; it is not a claim that the public read-only interface authorizes writes.

The same live task also requires a reversible authorization transition:

1. Authenticated query of the seeded data: HTTP `200` and the expected literal.
2. Temporarily deny reads in this run's OPA module: the same authenticated query must complete with HTTP `403`, not a redirect or transport failure.
3. Restore the original module, read it back, then issue a fresh authenticated query: HTTP `200` with the same stored value.

The console prints these three stages. The redacted `result.json` contains `policyTransition.httpStatuses: [200, 403, 200]`, `restored: true` and `storedDataUnchanged: true` only after every assertion succeeds. The inspection pause happens after restoration, so the displayed composition remains usable.

Policy administration is an explicit operator-side demonstration action using OPA's Policy API, not a sixth Anchor wire, automatic remediation, or a production policy-management design. The harness requires ownership of the applied namespace and an exact match to the reviewed policy before mutation. A failed or ambiguous update triggers a restoration attempt; any failure still invokes the parent harness's owned-resource cleanup. It never changes another namespace's policy. In production, OPA recommends bundles for policy distribution; this local demonstration uses the API to make one bounded, observable transition.

Offstage dependency-outage qualification

```
task testing:pocs:secure-api:resilience:live
```

Run this separate preparation task offstage, not during the short `task demo`. It repeats a real OPA process outage twice, then injects a failure after OPA has stopped to verify restoration and cleanup. The existing canonical composition is reused; no duplicate DOT or deployment descriptor is introduced.

Each successful run verifies an authenticated stored-data query (HTTP `200`), zero OPA pods and an authenticated rejection (`403`), then a new OPA pod and the same stored value (`200`). The pinned APISIX OPA plugin fails closed with `403` when its decision service is unavailable; a redirect, `5xx` or transport failure does not pass. Pod-drain polling uses a 90-second deadline with individually bounded API requests; an in-flight request can extend the drain's wall-clock duration. Rollout has a 180-second limit. This process outage is distinct from changing a Rego rule.

The operator-side control verifies the namespace UID, Deployment UID, exact pinned image, replica count and resource version before conditional patches. It attempts restoration after ambiguous responses or injected failure, never adopts a replacement Deployment, and leaves final ownership-safe teardown to the existing harness. Private per-run `resilience-control.json`, `resilience-restoration.json` and successful `resilience.json` receipts retain only redacted assertions. The aggregate receipt under `DOT-Testing/e2e/target/poc-wave/` requires unchanged source/CLI fingerprints, two recoveries and verified failure cleanup.

This does not qualify Keycloak outage/token-cache behaviour, gateway failover, resource pressure, HA, durable backup/restore, stateful upgrades or the complete producer PoC. Those remain separate operational acceptance work.

Composition and ordinary live route

These arrows show logical PROVIDES-to-REQUIRES wires, not HTTP packet direction:

```
Keycloak -- discovery --> APISIX <-- SPARQL -- Oxigraph
OPA ----- decisions ---> APISIX -- protected capability --> Client
Keycloak ---------------- token -------------------------> Client
```

Graph: `DOT-Repository/Compositions/keycloak-apisix-opa-sparql/composition.yaml`. Producer context: PoC10 `e6a755145bbb71551916163209c0be6e9e996f9c`. Five instances exchange through five wires:

- Keycloak discovery to APISIX;
- Oxigraph SPARQL to APISIX;
- OPA policy decisions to APISIX;
- OAuth token endpoint to the client (`TOKEN_URL`);
- protected SPARQL capability to the client (`CAPABILITY_URL`).

Native Secret references supply credentials. Routes/policy are explicit, composition-owned inputs, not automatic translation. OIDC/OAuth Contracts come from canonical bundles. SPARQL and the legacy OPA workflow remain explicit protocol interfaces, not invented official vendor OpenAPI documents.

```
kubectl config current-context
kubectl get nodes
dotx compose inspect \
  DOT-Repository/Compositions/keycloak-apisix-opa-sparql/composition.yaml \
  --locked --json
task testing:pocs:secure-api:live
```

Expect a fresh namespace and native test Secrets, HTTP `401` for missing and malformed bearer tokens, authenticated POST `403`, and an authenticated GET returning `mare-poc10` from the real SPARQL engine. Every required client input is checked. The harness destroys its namespace and forward; the receipt must say `cleanup=PASSED`. Override `DOTX_POC10_FORWARD_PORT` if 19010 is busy.

Observation	Expected result
Anonymous request to the gateway	HTTP `401`
Deliberately malformed public test token	HTTP `401`; redirects/transport errors are not acceptance
OAuth client-credentials token	Acquired inside the client pod; never printed
Authenticated POST/write attempt	HTTP `403` from the protected route
Authenticated stored-data GET	Exactly one literal `mare-poc10`
Lifecycle	Recorded status, destroyed owned namespace/forward, `cleanup=PASSED`

During the presenter pause, inspect the printed private-cache `compose status` command and `result.json`. The five compatible wires are composition/configuration evidence; the four HTTP assertions above are the live end-to-end evidence. They are not five independent native Contract probes or full response-schema validation. Press Enter to clean up and retain the receipt. Re-run from the cleaned state to check repeatability; never treat a previous receipt as a new live exchange.

The invalid token is fixed public test data; real access tokens are never printed. `invalidBearerHttpStatus` in `result.json` records the checked 401. The bounded check ignores local curl configuration and does not follow redirects. It proves rejection of this malformed token, not expiry/revocation handling, issuer/audience isolation, signing-key rotation, or a security certification.

Oxigraph substitutes for GraphDB; a synthetic machine client uses client credentials instead of a password grant. This does not accept the full producer GraphDB/LDAP/Data Security Service stack, telemetry/events or automated response. HTTP, bootstrap defaults and the read-only policy are demo settings, not production security hardening.

3. PoC6: explicitly adapted remediation rehearsal

```
CACAO interpreter [adapter] -- MQTT/OpenC2 via Mosquitto --> actuator
actuator -------------------- remediation call ---------> NWDAF [DOUBLE]
```

Graph: `DOT-Testing/e2e/poc6-cacao-openc2/scenario.yaml`, not a catalogue example. Producer revision: `cd359b25df1b010324869d7e1c755067070e42dd`.

```
task testing:e2e:poc6-cacao-openc2:dry-run
task testing:pocs:remediation:live
```

A guarded interpreter adapter resolves the CACAO external broker URI. The unchanged pinned actuator receives OpenC2 over real Mosquitto/MQTT through the AsyncAPI wire, then calls the OpenAPI remediation endpoint. The exact job UUID must reach the far end. That endpoint is an **NWDAF double**, not real NWDAF remediation. Telemetry, AI and playbook selection remain unaccepted. Override `DOTX_POC6_INTERPRETER_PORT`/`DOTX_POC6_NWDAF_PORT` if 18261/18262 are busy.

Repeatability, cleanup and recovery

```
task testing:pocs:rehearse
```

This runs each slice twice, followed by injected failure and real SIGINT/SIGTERM cleanup controls. It records the CLI hash, three checkout revisions, tracked/untracked fingerprints, runtime image identities and per-run receipts under `DOT-Testing/e2e/target/`. Changed inputs invalidate the aggregate receipt. Dirty-worktree evidence does not claim those changes were already published.

The manual **Anchor live E2E** workflow's `first-wave` selection runs the same checks and rehearsal driver. Remote infrastructure and queue failures remain separate from local acceptance; no workflow success is claimed before it runs.

Harnesses refuse existing resources, lock fixed identities, use private caches and check owned leftovers. Anonymous Docker volumes are removed; unrelated resources are untouched. PoC6 test images remain cached. `DOTX_E2E_KEEP=1` records `SKIPPED` cleanup and cannot pass acceptance.

Kubernetes namespace creation records the UID from its create response. Cleanup rejects unknown or replaced ownership and uses a server-side UID precondition for namespace deletion. Lost create confirmation is a failed cleanup requiring review, not permission to adopt a same-named namespace.

Current-source named Helm restart with explicit `--forward` also reconciles a previously recorded forward when its custom release name differs from the DOT product name. Replacement requires an exact namespace/service/port mapping in the private forward registry; unrelated namespaces, services, mappings and unrecorded listeners are not adopted. This is a Next runtime correction, not a change retroactively added to the published `0.3.0` CLI binary. Keep the rehearsed source/binary set together.

If cleanup fails, preserve the printed run directory. Set `XDG_CACHE_HOME` to its `cache` directory and, for Kubernetes runs, set `KUBECONFIG` to that run's private `kubeconfig`. Do not follow a later global context change or share the saved kubeconfig. Stop if the original run files are missing; a new run's state does not authorize recovery. Confirm the namespace still has its recorded UID; stop and review a mismatch or an unconfirmed identity. Inspect `dotx compose status <composition>`, then retry `destroy` on the same graph only while the original ownership still matches. Inspect only recorded namespaces and resolved identities. Never strip finalizers or delete unrelated workloads. Select an unused port instead of killing another service. Review missing Contracts, inputs or stale locks before intentionally regenerating a lock.

Full-catalogue appendix

`DOT-Testing/interoperability/FIRST-WAVE.md` and `pocs.yaml` retain all eleven PoCs and remaining gaps. `DOT-Repository/CONTRACT-AUDIT.md` separates official and source-derived Contracts, protocol-only interfaces and runtime-only brokers. Validation-only fixtures, stubs, adapters and doubles remain labelled. Declaration completeness is not runtime acceptance or production certification.

This page publishes through `next`; stable-tag documentation is unchanged. Rehearsal evidence is a presentation fallback, not a substitute for a claimed current live result or a green remote workflow.

Docker/Kubernetes four-Contract demo

This `next` development walkthrough reuses three canonical catalogue definitions: the standards-native provider, consumer, and official Mosquitto broker. Named deployment variants select Kubernetes for provider/broker and Docker for consumer. No product descriptor is duplicated.

The composition is named `docker-kubernetes-four-contracts`. Its template lives at `DOT-Testing/composition/docker-kubernetes-four-contracts/scenario.yaml`: this is a synthetic test graph, unlike the reusable catalogue examples `keycloak-apisix-oauth` and `keycloak-prometheus-openapi-validation`.

> Provider and consumer are explicitly test-only components, not MARE analytics or vendor products. Real HTTP, GraphQL.js, gRPC-js and Mosquitto exchanges demonstrate interoperability mechanics, not producer acceptance, production security, or automatic protocol conversion. Probe clients use the actual consumer's Docker network namespace; its application process consumes REST data and derives a synthetic review decision. MQTT traffic is generated by the correlated delivery probe, not the GraphQL application. The MQTT runner uses the pinned official `mosquitto_rr` client in MQTT 3.1.1 mode, subscribing and publishing on the same Contract-selected channel. It waits for SUBACK before publishing, ignores retained messages, uses a clean session, and requires the exact uniquely correlated payload. This proves broker delivery, not an application-level request/response implementation.

What runs where

Wire	Authoritative Contract	Kubernetes endpoint	Docker-side assertion
`rest-alerts`	OpenAPI 3.1, `GET /alerts`	Provider, NodePort `31080`	HTTP 200 plus the expected fixture alert UUID and `critical` severity
`graphql-health`	GraphQL SDL, `Query.health`	Provider, NodePort `31400`	`/data/health` equals `true`
`grpc-health`	Protobuf, side-effect-free `AlertService.Check`	Provider, NodePort `31551`	Reflected response has `serving: true`
`mqtt-alert-delivery`	AsyncAPI 3.0, MQTT-bound alert channel	Mosquitto, NodePort `31883`	Schema-valid, uniquely correlated QoS 1 delivery

Anchors select Contract operations. Wires select participants and materialize resolved endpoints into the consumer's declared environment. Each wire declares `EXTERNAL_GATEWAY`: the selected DOT declares the NodePort Service, and Minikube provides the reachable node address. Docker normally isolates its bridge networks: the harness explicitly attaches only its disposable consumer to the existing `minikube` Docker network. Cross-runtime probes use `--network container:<consumer>` so they exercise that actual consumer route, not a different runner's network. The attachment is external to `dotx` state, and consumer destruction removes it; the harness verifies its removal. No host listener, firewall change, tunnel, or Minikube-container modification is made.

The REST provider returns one deterministic, read-only synthetic alert: `a4a70000-0000-4000-8000-`

`000000000099`, severity `critical`. The native `probe.openapi.expect` assertion compares the complete JSON array, requiring a declared successful status, JSON media type, and the expected identity/severity. Use the installed development CLI; the released `v0.2.5` binary predates this feature. A supplementary independent application assertion reads `OPENAPI_YAML_URL` from the deployed consumer and fetches it from that consumer's actual network namespace. It requires HTTP 200, JSON content type, exactly one alert, and the expected identity/severity. Redirects, wrong data, oversized bodies and a stalled response fail acceptance. The 10-second, 1-MiB-bounded Node.js client uses the already-built provider image; it is a test client, not a new DOT or an implicit GraphQL-to-REST translator. Only its fixed redacted summary is saved as `rest-body.json`; arbitrary response fields are not printed. The independent check also verifies the actual materialized consumer configuration; it is not needed to enable the native CLI response assertion.

Show application behaviour and a controlled failure

The Docker consumer now runs a small synthetic application, not `sleep`. It repeatedly fetches the wire-materialized `OPENAPI_YAML_URL`, validates the bounded response, and derives `REVIEW_REQUIRED` when any alert is critical; otherwise it derives `NO_CRITICAL_ALERTS`. It does not hard-code the demo UUID or emit a canned decision. Its owner-only, atomically replaced receipt lives inside the disposable consumer at `/tmp/dotx-consumer-result.json`. No new public endpoint or duplicate DOT definition is introduced.

The normal live and presenter commands automatically rehearse this sequence:

1. Observe a fresh consumer receipt: one critical alert, `REVIEW_REQUIRED`.
2. Change only the owned synthetic provider's test-control file to serve severity `low`. Its API still returns HTTP 200 and schema-valid JSON, so a readiness check alone would not expose the changed application behaviour.
3. Require `dotx compose probe` to exit 1 with recorded HTTP 200 and `RESPONSE_MISMATCH`, not a timeout, transport error, or arbitrary failure. Independently observe the running consumer change to `NO_CRITICAL_ALERTS`.
4. Restore the critical fixture; require a successful native assertion and a newer consumer receipt with `REVIEW_REQUIRED` before presenting results.

These steps run automatically through the existing live and presenter commands below; no separate control command or second demonstration is required.

The file `/tmp/dotx-rest-severity` is an explicitly test-only fault switch in the disposable provider container. The harness controls it only after owning the run's namespace, restores it before accepting the negative control, and destroys the owned composition on any unexpected failure. It never changes a real vendor DOT, Kubernetes context, or unrelated workload. `low` is valid for the Contract's string field; the rejection proves the reviewed composition's value assertion, not full response-schema validation or production policy.

Evidence includes `consumer-before.json`, `consumer-negative.json`, `consumer.json`, `rest-negative-status.txt` and `response-recovery.json`. Increasing receipt sequence numbers prove observation after each transition. `result.json` includes the recovered consumer decision and negative-control summary. Interrupted or failed checks cannot publish a successful result. The application and fixtures are synthetic: `REVIEW_REQUIRED` is a demonstration decision, not real security analytics, mitigation, or producer acceptance.

Prepare before presenting

Use the full checkout in Contributor guide, and run commands from its root. Requirements: Linux amd64, Go and Task to install this checkout's CLI, Git, Bash, jq, curl, GNU coreutils, `flock` (util-linux), Docker with Compose, kubectl, and a running local Minikube cluster using its Docker driver and `minikube` Docker network. The Kubernetes context must be `minikube`. The deterministic demo checks also require ShellCheck and Python 3. On Ubuntu, install the `shellcheck` package before running `task testing:demo:checks`. The introductory Prometheus quickstart still needs no Go or Kubernetes.

```
git submodule update --init --recursive
task ci:prepare
git rev-parse HEAD
git -C DOT-CLI rev-parse HEAD
git -C DOT-Repository rev-parse HEAD
go version -m "$(command -v dotx)"
docker info --format '{{.ServerVersion}}'
docker compose version
kubectl config current-context
kubectl get nodes --request-timeout=10s
minikube ip
```

`dotx 0.2.4` alone is insufficient: this example uses post-release development changes. Install the pinned submodule and keep the checkout fixed. Evidence records revisions, CLI binary SHA-256, root/CLI/catalogue working-tree patch hashes, and provider/consumer image IDs. Non-empty patches mean the commit alone does not describe the tested checkout. Broker/probe images use digest references; the test images are locally built from pinned Dockerfiles/dependency locks, not claimed published immutable images. The provider uses `imagePullPolicy: Never` after explicit loading; the harness verifies actual running image IDs. Rehearse with network access for metadata/dependency/image downloads; caching does not promise fully offline execution.

After a successful live rehearsal has built/loaded the fixture images and cleaned up, run the read-only check for this route:

```
task testing:demo:preflight DEMO=four-contracts
```

It checks the exact installed CLI revision, clean CLI/catalogue checkouts, offline Contract validation and locked composition, the Ready Linux-amd64 Minikube node, selected cached images, Docker identity/network conflicts, and existing Services occupying the four selected NodePorts. It does not build/pull images, reserve ports, attach networks, deploy anything or prove gateway reachability. The live acceptance below remains necessary. Use `DEMO=monitoring` for the node-exporter/Prometheus route, `DEMO=portability` or `DEMO=upgrade` for those lanes, or omit `DEMO` to check all five presenter routes.

For a stage run without fixture rebuilds, first run explicit `task testing:demo:prepare DEMO=four-contracts`, then export `DOTX_DEMO_PREPARED=1` before the live/presenter command. This checks the prepared source/binary fingerprints and Docker/Minikube fixture IDs and refuses drift. It performs no build, pull, image import or cluster start in that mode. It does not guarantee offline probe execution; see the qualification and sanitized fallback route.

> ⚠ Use an isolated demo machine/network. NodePorts are plaintext; MQTT permits anonymous access and disables persistence. Do not expose them to untrusted networks. These are demo defaults, not production authentication, TLS, retention, or availability settings. Never delete an unrelated namespace or Service.

Inspect without deploying

```
exec bash DOT-Testing/e2e/multi-runtime-demo/test.sh --dry-run
```

This resolves three DOTs/four wires, writes and verifies a lock, checks both platforms and all four standards, and renders deployment actions in a private directory under `DOT-Testing/e2e/target/multi-runtime-demo/`. The template uses documentation-only address `192.0.2.1`; never manually apply that template.

Run the working example

Presenter mode: inspect before cleanup

```
task testing:demo:docker-kubernetes-four-contracts:present
```

Run in a real interactive terminal. The composition-derived Mermaid graph is laid out by the Go `mermaid-ascii` library embedded in `dotx`: Unicode boxes, cyan Kubernetes labels, magenta Docker labels and amber connection IDs. Parallel wires share a labelled arrow and each retains its own legend entry. The node inventory shows selected runtimes/deployments/versions; a `T4` link identifies the MQTT transport endpoint, not another Anchor wire or an application publisher. Arrows mean logical PROVIDES-to-REQUIRES relationships, not packet direction. The graph is printed again after all four real probes and recorded status checks pass.

Only then does the harness pause. Use the printed, shell-quoted recovery/status command in a second terminal to inspect the exact live composition. `resolved-composition.json`, `topology.txt`, `topology.mmd`, probe logs, `status.txt`, `results.txt` and `result.json` remain in the printed private run directory. The result summarizes checked fixture assertions, not real MARE analytics. `rest-body.json` holds the separate REST data assertion. Presenter mode shows concise numbered progress and an aligned results table with separate Contract, protocol, observed assertion and outcome columns. Each row requires both completed probe and recorded status evidence. Detailed build/deployment/cleanup output remains in owner-only `details.log`; a failure prints the original failing stage, exit code and its log location. Do not share raw private logs. Normal live acceptance keeps its verbose output. Both normal and presenter runs save `result.json` and `results.txt`; summaries are published only after all probes, the REST data assertion and recorded status checks pass. A failed recheck removes earlier result summaries rather than leaving stale green output. Cleanup is reported separately in `evidence.json`; an assertion summary alone never proves successful cleanup. See the audience FAQ for the model's value, reuse, standards and evidence boundaries. The graph deliberately omits endpoint hosts, configuration and credential values. `topology.mmd` can also be rendered in Antora. `NO_COLOR=1` disables colours; `DOTX_TOPOLOGY_COLOR=auto|always|never` controls them (`NO_COLOR` or `TERM=dumb` takes precedence). `DOTX_TOPOLOGY_WIDTH=80` sets a 40-240-column target; narrower screens use vertical/compact layouts. No connection may disappear to fit. The saved `topology.txt` is always plain text. Rendering needs only the `dotx` binary: no Python, Node, browser or separate `mermaid-ascii` executable. The Go module pins the upstream 1.6.1 source revision and verifies dependencies through `go.sum`; no renderer is downloaded at runtime.

```
# Read-only topology preview; Docker/Kubernetes need not be running.
dotx compose topology DOT-Testing/composition/docker-kubernetes-four-
contracts/scenario.yaml --locked
dotx compose topology DOT-Testing/composition/docker-kubernetes-four-
contracts/scenario.yaml --locked --color always --width 80
dotx compose topology DOT-Testing/composition/docker-kubernetes-four-
contracts/scenario.yaml --locked --format mermaid
```

Press Enter to clean up normally. Ctrl-C exits 130, SIGTERM 143, EOF 1 and timeout 124, all through the existing ownership-checked cleanup path. Inspection defaults to 600 seconds; set `DOTX_POC_PRESENT_TIMEOUT` to an integer from 1 to 1800 to change it. Headless presenter launches and `DOTX_E2E_KEEP=1` are rejected before runtime mutation. CI and the normal live task never pause, even when presenter environment variables are inherited.

`task testing:demo:present:rehearse` drives all five terminal outcomes against the real composition, verifies status while resources are still running, and requires verified cleanup. It is an acceptance preparation task, not the talk.

Noninteractive acceptance

```
exec bash DOT-Testing/e2e/multi-runtime-demo/test.sh --live
```

Equivalent Task: `task testing:demo:docker-kubernetes-four-contracts:live`. The harness locks the real Minikube address, builds/loads images, creates an isolated namespace and state cache, deploys through `dotx compose apply --locked`, checks readiness, and probes each wire with a 45-second timeout. It verifies recorded status, destroys the composition, removes its namespace, and checks for owned Docker leftovers.

Expect `PASSED` lines for `rest-alerts`, `graphql-health`, `grpc-health`, and `mqtt-alert-delivery`, plus exit zero and `cleanup=PASSED`. Inspect the printed run directory’s `resolved-composition.json`, `render.txt`, `status.txt`, and `evidence.json` to explain the locked composition, endpoint materialization and observations. Evidence JSON contains no passwords/tokens or response bodies. Each run retains its evidence; the top-level `evidence.json` is only the latest result, possibly an injected failure.

Rehearse the complete presentation

Present Prometheus first, authenticated Keycloak/APISIX/OAuth second, then the cross-runtime standards example:

```
task testing:demo:dry-run
task testing:demo:mqtt-suback:live
task testing:demo:prepare
task testing:demo:qualify
```

The full rehearsal now repeats seven live paths twice: quickstart, monitoring, Prometheus portability, secure API, OPA upgrade/rollback, OAuth and this fixture. Use the integrated ecosystem route for explicit preparation, preflight and evidence interpretation. `qualify` additionally rehearses the four official terminal inspection pauses and Enter cleanup. The narrower `rehearse` Task retains noninteractive coverage. The quickstart covers default/overridden ports and failure cleanup. OAuth verifies HTTP 401 without a token, token acquisition, protected access, native Secret references, status, and cleanup. The separate MQTT regression uses the actual pinned client and a loopback-only fault-injection peer: delayed, denied, or absent SUBACK, stale retained messages, and incorrect correlated payloads must behave correctly. This peer is not a replacement for the full composition's real Mosquitto acceptance. Its ephemeral credentials and Keycloak development mode are not production defaults. The final mixed-runtime run deliberately fails after deployment: exit 42 and successful cleanup are required. Two further live runs receive actual SIGINT and SIGTERM after deployment and network attachment. They must exit 130/143, record cleanup success, and pass independent namespace/container/network checks. Run those checks separately with `task testing:demo:interruption-cleanup`.

The rehearsal prints one per-run `summary.json` under `DOT-Testing/e2e/target/rehearsal/run-*/`, with a separate log for every step, exit codes, durations, and before/after checkout and CLI fingerprints. Tracked patches and untracked-file content fingerprints are recorded for all three checkouts; no source or credential contents are copied into the receipt. Changed inputs during rehearsal fail acceptance. The convenience `latest.json` points to that per-run receipt, including failures; it is not a green badge. A dirty checkout remains explicitly labelled by its recorded status.

This is pre-presentation acceptance, not the short live talk itself. A per-user live-run lock rejects overlapping instances before deployment, including across checkouts using the same runtime/cache environment. `DOTX_DEMO_LOCK_DIR` is an override for isolated lock tests only; do not override it to run concurrent copies of this composition.

Recover safely

Normal failure, Ctrl-C and termination attempt cleanup. Do not use `kill -9` or delete state while cleanup runs. After interruption, use the exact printed path:

```
RUN_DIR=/absolute/path/printed/by/the/failed/run
(
  test -f "$RUN_DIR/kubeconfig" && test -f "$RUN_DIR/scenario.yaml" || exit 1
  XDG_CACHE_HOME="$RUN_DIR/cache" KUBECONFIG="$RUN_DIR/kubeconfig" \
    dotx compose status "$RUN_DIR/scenario.yaml"
)
```

Keep the saved kubeconfig private; it contains credentials. These commands use the run's frozen Kubernetes context, not whichever context is currently selected in another terminal. If either required file is missing, stop: do not substitute the default cache/context or guess a namespace. Execute the commands one at a time and stop on an unexpected error.

Only after the original harness has stopped and ownership is confirmed, request cleanup separately (using the same `RUN_DIR`):

```
(
  test -f "$RUN_DIR/kubeconfig" && test -f "$RUN_DIR/scenario.yaml" || exit 1
  XDG_CACHE_HOME="$RUN_DIR/cache" KUBECONFIG="$RUN_DIR/kubeconfig" \
    dotx compose destroy "$RUN_DIR/scenario.yaml"
)
```

Inspect the namespace in that run's `render.txt`; delete only that test-owned namespace if it remains. An apply failure may already have rolled back with no applied state; independently verify resource absence. Never prune Docker, remove Minikube, or delete unrelated volumes. Find the owner of an occupied NodePort before proceeding. Fix Docker user permissions before rerunning. Gateway timeouts require checking node IP, routing/firewall, readiness and Services; resolution does not prove reachability. Contract download failures require repository access or verified bundle-local assets, never bypassing SHA-256 checks.

After a manually cancelled GitHub job

The live workflow queues subsequent pushes rather than cancelling a rehearsal mid-cleanup. Manual cancellation or loss of the runner can still interrupt teardown. An occupied-name/network preflight failure is not a request to delete whatever was found, and rerunning the job alone cannot repair those leftovers.

First confirm that no job or local demo still owns the resources. Correlate the cancelled attempt's private run path and timestamps with the container's exact name, image and `dotx.runtime.identity` label, the dedicated network's `dotx.composition.id` label and attached containers, and the namespace's workloads. Inspect mounts before considering removal. If the runner checkout no longer contains the run's state, do not use a new run's cache for recovery: an operator must verify each exact leftover independently. Remove only confirmed test-owned resources, verify their absence, and rerun the failed job on the fixed revision. Leave the shared Minikube network/container, cached images, unrelated volumes and namespace finalizers untouched. Successful recovery is not successful live acceptance; the subsequent workflow must still finish its probes and cleanup.

Catalogue readiness appendix

```
task testing:contracts:readiness
jq '.dots | group_by(.status) | map({status: .[0].status, count: length})' \
  DOT-Testing/e2e/target/catalogue-contract-readiness.json
```

Audited release-qualified snapshot: 74 valid definitions, 29 strict interface-profile passes and 45 gaps. Splitting mixed tool releases changes bundle counts, not product readiness; it does not resolve interface gaps. Use the generated report for the actual checkout. Broker-only and negative fixtures do not necessarily need a business API. Official/source-derived Contracts, source-adapted PoCs, validation-only fixtures, stubs and producer doubles remain distinct evidence levels. See catalogue `CONTRACT-AUDIT.md` and Verification and examples. This demonstration does not promote the entire catalogue to production-ready.

Stateless upgrades and recovery

This optional technical-demo lane demonstrates native `dotx compose upgrade` with the existing external OPA DOT's Kubernetes deployment. The OPA image and DOT ID/version stay unchanged; the **composition version and policy input** change. This is a configuration revision, not an OPA software-version upgrade. There are no copied product descriptors, persistent volumes or application-data migrations in this example.

The focused test graph lives under `DOT-Testing/composition/opa-policy-upgrade/`, not the reusable production catalogue: one revision deliberately contains invalid Rego. Its three composition documents and locks resolve the same canonical `DOT-Repository/DOTs/external/OPA-DOT` bundle. This lane has one DOT and an exposed Anchor, not multiple wired DOTs; use the four-Contract and secure-API lanes for those claims.

Prerequisites and preview

Use the DOT-Design checkout root on Linux amd64, with its exact pinned CLI installed (`task cli:install`), Task, Bash, jq, curl, GNU coreutils, `ss`, Docker, kubectl and a Ready local Linux-amd64 Minikube node. Select context `minikube`. Building that pinned CLI requires Go; follow the contributor installation instructions. Go is not needed by the installed binary during this live lane. The canonical OPA image is digest-pinned; Kubernetes must have it cached or be able to obtain it. Use an isolated demo cluster: this OPA API intentionally has no production authentication policy and must not be exposed publicly.

```
task testing:demo:upgrade:dry-run
```

This checks all three locks and renders their native deployment commands without deploying anything. It does not parse Rego or prove upgrade/rollback acceptance: the invalid policy is intentionally a runtime-negative fixture. The native upgrade diff requires an applied baseline, so the live lane runs it after apply.

Run, inspect, and clean up

```
# Optional overrides; keep them in the same terminal.
export DOTX_UPGRADE_PORT=19081
export DOTX_UPGRADE_NAMESPACE=dotx-opa-upgrade-presentation
task testing:demo:upgrade:present
```

Choose a fresh namespace and a free port; the harness refuses existing resources and does not adopt them. `DOTX_UPGRADE_ROLLOUT_TIMEOUT` defaults to `90s` per Kubernetes rollout. Allow a few minutes: the broken revision intentionally waits for a rollout failure before native recovery.

The task automatically executes this sequence:

Stage	Native lifecycle operation	Required observation
Baseline `1.0.0`	Locked `compose apply`	A viewer’s GET authorization decision is HTTP `200`, `result: true`.
Review	`compose upgrade --dry-run --json`	Exactly one `UPDATE` for the existing OPA instance, with different input digests.
Target `1.1.0`	Locked `compose upgrade` from `1.0.0` and its exact lock	Viewer decision becomes `false`; operator decision remains `true`.
Broken target `1.2.0`	Locked `compose upgrade` from `1.1.0`	The exact negative fixture fails parsing with the running OPA binary; its deployment fails rollout and the native command reports automatic rollback.
Recovered `1.1.0`	Native status plus fresh API requests	Last-known-good version remains `1.1.0`, no upgrade transaction is pending, viewer remains denied and operator remains allowed.

OPA’s decision API returns HTTP `200` for both boolean decisions; `false` is an authorization decision, not HTTP `403`. In the separate secure-API composition, APISIX enforces that decision and returns HTTP `403` to its caller.

Say: "A valid DOT and lock do not guarantee valid application configuration. The runtime rejected this policy, and the CLI restored the last verified configuration before reporting failure. We then tested the recovered service."

The normal lane calls native CLI commands directly. It does not implement a second upgrade or rollback engine. Recovery is to the last successful `1.1.0` revision, not an undo of the previously successful promotion from `1.0.0`. The baseline and target locks are required for both changes.

After successful recovery, presenter mode holds the running OPA instance for inspection. It prints the private run directory and a status command using that run’s cache and frozen kubeconfig. Inspect `upgrade-diff.json`, redacted `result.json`, API decision receipts and status snapshots. The saved `failed-upgrade.log` contains the expected failed rollout and native recovery; that failure is not silently treated as a successful upgrade. Routine native apply/upgrade output stays in private `apply.log`/`upgrade.log`; the terminal shows ordered progress stages and a final observed-decision table.

Press Enter to clean up. EOF, Ctrl-C and the bounded presenter timeout also invoke owned-resource cleanup. Successful acceptance requires a final `cleanup=PASSED` receipt. For noninteractive rehearsal use:

```
task testing:demo:upgrade:live
```

With Python 3 installed, `task testing:demo:upgrade:rehearse` uses the existing terminal-driven acceptance driver to verify the pause, inspect native status, send Enter and require verified cleanup. It records a summary and before/after checkout fingerprints; it fails if inputs change during rehearsal.

Boundaries and recovery

This proves one stateless native-Kubernetes configuration update and automatic recovery, not database rollback, zero downtime, all deployment adapters, policy distribution security or production certification. The exposed API Contract is the bundle's ecosystem-derived PoC1 normalization, not an official comprehensive OPA OpenAPI document. The live checks verify the actual boolean decisions.

Current upgrade admission rejects changed stateful deployments and changed Helm/Vagrant deployments whose storage safety cannot be established. A pending transaction after interruption requires the native `--resume` or `--rollback` route documented in Composition lifecycle. Do not rewrite its state file, delete unrelated resources or bypass these checks.

If cleanup is interrupted, inspect the printed run's evidence and use **both** its `XDG_CACHE_HOME` and `KUBECONFIG` for status/destroy. Never substitute a new cache or a changed global cluster context. Do not force namespace finalizers.

The existing **Anchor live E2E** workflow offers an explicit `opa-policy-upgrade` selection with the same harness and private evidence upload. The default PoC `all` matrix remains the established twelve scenarios; this optional technical lane is selected separately. Dry-run CI checks the fixtures through the existing presentation-validation job.

One DOT, two runtimes

This optional presenter lane answers: "Can we reuse the same DOT without copying its product definition when the deployment runtime changes?"

The catalogue composition `prometheus-docker-kubernetes-portability` selects the existing `configuration-docker-compose` and `configuration-kubernetes-manifest` profiles of the **same** Prometheus DOT: ID `Prometheus@3.15.1`, upstream release `3.15.1`. Both resolve its official OpenAPI Contract, use the same digest-pinned upstream image, and declare the same default self-scrape configuration and `GOGC=75`. No product descriptors, Contracts or default configurations are copied for this demonstration.

> These are two independent servers with **zero wires**. This demonstrates deployment and configuration portability, not replication, data migration, or cross-runtime communication. Use the four-Contract demo for actual Docker-to-Kubernetes exchanges, and the secure-API composition for authentication and authorization between several products.

Prepare and preview

Run from the DOT-Design checkout root on Linux amd64. Install its exact pinned CLI using `task cli:install`; Go is needed for installation, not by the running binary. Follow the contributor checkout/tool guidance. The released `v0.2.5` binary predates this lane's cross-runtime repetition and restrictive-umask config-mode fixes; the version string alone is insufficient. Requirements:

Task, Git, Bash, jq, curl, `ss`, GNU coreutils, Docker with Compose, kubectl and a Ready Linux-amd64 Minikube node with Docker runtime. Select context `minikube`. No Java, Kubernetes cluster provisioning or credentials are needed once those local prerequisites are installed and ready.

```
task testing:demo:portability:dry-run
task testing:demo:portability:prepare
```

The first command validates the reviewed lock and renders the two selected deployments without deploying anything. Temporary default-config files are materialized in an owned preview cache and removed on exit. The second command is **explicit preparation**, not read-only preflight: it pulls only the selected digest-pinned image into Docker and the Minikube node cache. It does not deploy the composition. Preparation requires registry access and does not imply live API acceptance or universally offline operation.

Present the comparison

```
# Optional: choose a fresh namespace and unused Kubernetes forwarding port.
export DOTX_PORTABILITY_NAMESPACE=dotx-prometheus-portability-presentation
export DOTX_PORTABILITY_KUBERNETES_PORT=19083
task testing:demo:portability:present
```

The Compose profile's declared localhost port is `18919`. Both host ports must be unused. Never delete an unknown workload to free them. The task refuses existing namespaces and Compose project resources; it does not adopt them.

The task then:

1. Resolves the same canonical DOT/Contract for both instances and requires the same immutable image pin. It reads the image's own `--version` output in a disposable, network-disabled container to establish the expected API version.
2. Applies both native profiles through `dotx compose apply` and records status.
3. Verifies the actual container/pod image identities, bounded readiness, official build-information version, runtime `GOGC`, default configuration and a real healthy self-scrape on **both** servers.
4. Displays the locked topology, a live comparison table and inspection URLs, then holds the verified services for bounded inspection.
5. Destroys only this run's resources and verifies no Compose containers, networks, volumes, namespace, forwards or composition state remain.

The default selected image currently reports Prometheus `3.15.1`; the assertion derives the version from its pinned bytes rather than assuming that a version label or two matching server responses prove provenance. The runtime receipts record actual image IDs and observation timestamps. The terminal table reports `GOGC=75` and `up=1` for each independently verified server.

Say: "The DOT identity and API Contract stay the same. The selected deployment mechanism, resource identities and endpoint change. That is reuse of one product definition, not two products that happen to have similar names."

While paused, open the displayed localhost URLs at `/targets`, or inspect the redacted `result.json` and each runtime’s receipt in the printed private run directory. The printed native status command includes the run’s cache and frozen kubeconfig; keep both when inspecting or recovering that run.

Press Enter to clean up. EOF, Ctrl-C and presenter timeout also invoke cleanup, but return a nonzero status. A successful demonstration requires the final `cleanup=PASSED` receipt. Native deployment details stay in `apply.log`; early Compose process failures retain bounded startup diagnostics before cleanup.

For noninteractive acceptance and an automated inspection/Enter rehearsal:

```
task testing:demo:portability:live
# Requires Python 3; also exercises a Kubernetes port override.
task testing:demo:portability:rehearse
```

Storage and acceptance boundaries

Data is disposable. Native Compose destroy removes this test project’s named TSDB volume. The selected Kubernetes profile has no persistent volume. This comparison does not move history between servers, prove persistence/backup semantics or certify every Prometheus API and deployment variant.

The CLI admits one native Kubernetes instance of this DOT alongside isolated Docker instances. It still rejects two native Kubernetes instances with fixed object names, including when a Docker instance separates them in the document. Do not bypass that guard or infer arbitrary same-namespace object isolation.

If interrupted, inspect the receipt and use the recorded cache/kubeconfig for native status/destroy on the same composition. Never substitute a new run’s cache, delete unrelated resources or strip namespace finalizers. The **Anchor live E2E** workflow offers this lane explicitly as `prometheus-portability`; dry-run CI exercises the same script. Runtime and fixture acceptance remain distinct from production certification.

Contributor guide

This page covers building DOT-Design and running its shared local/GitHub gates. To use an existing DOT without installing development toolchains, start with Getting started.

Prerequisites

Tool	Required for	Project baseline
Java and Maven	`DOT-Domain`, schema generation, and Java tests	Java 27 (`27.0.0+35-open` in `.sdkmanrc`); Maven 3.10.0 from `PATH`
Go	Building and testing `dotx`	Go 1.27.2
Graphviz and fonts	Reproducing the legacy full-Maven-build SVG export, not running services	Local baseline: Graphviz 14.1.2 and Ubuntu `fonts-noto-core` / `fonts-croscore` 20201225-2build1. Missing tools or different font fallbacks can change the SVG even when the generated model source is identical.
Task	Repository command aliases	Optional but recommended
Docker, Kubernetes tools, Helm, or Vagrant with libvirt/KVM	Only the matching deployment adapters and live scenarios	Install only what you intend to execute
Antora, the Antora PDF extension, Asciidoctor PDF, and Mermaid CLI	Building the HTML/PDF documentation site and Mermaid diagrams	CI uses Node.js 26 and Ruby 4.0; `task docs:check` reports the locally installed tool versions; see Build the documentation

Check out the repository

```
git clone https://github.com/MARE-EU-Project/DOT-Design.git
cd DOT-Design
git submodule update --init --recursive
```

The source repository and its two submodules may require access granted by the MARE-EU-Project organization. `DOT-CLI` and `DOT-Repository` remain separately cloneable; the generated Schema and high-level Skill classifications are tracked inside `DOT-Repository` and require no additional checkout.

When updating submodules, commit and push the child repositories first, then push the parent branch with a missing-submodule-commit check:

```
git -c push.recurseSubmodules=check -c push.followTags=false push
```

This refuses to publish a parent whose child commits are unavailable remotely. Release tags are published separately after qualification; a branch push must not publish prepared tags implicitly.

Verify the domain model

The domain module uses plain JUnit tests and does not start a service:

```
mvn -pl DOT-Domain test
```

Run the complete Java build with:

```
mvn clean install
# equivalent Task alias
task mci
```

The build generates CycloneDX JSON and XML software bills of materials during the Maven `verify` phase.

The full Maven build also regenerates diagram exports. Release packaging requires clean committed sources, including those exports. If rendering changes a tracked SVG, diagnose the source, renderer and fonts; do not discard the difference blindly or weaken the provenance guard to make packaging pass.

Install and test dotx

`DOT-CLI` is an independent Go module and is not part of the Maven reactor:

DOT-Design `VERSION`, its Maven artifacts and the pinned `DOT-CLI/VERSION` share the same version number. Run `task testing:version:alignment` to verify the checkout. Independent repositories and publication jobs do not imply independent version numbers; source revisions still identify unreleased changes. Antora keeps current development content under `Next`, preserving stable tags.

Install the standalone release with Go 1.27 or later. For the private module, authenticate Git first as described in Getting started and bypass the public Go proxy for this organization in this command only:

```
GOPRIVATE='github.com/MARE-EU-Project/*' go install github.com/MARE-EU-Project/DOT-
CLI/cmd/dotx@v0.3.0
dotx version
```

The GitHub release also provides a checksummed Linux-amd64 binary, build metadata, and a manifest that pins the exact DOT-CLI source plus the DOT-Design and DOT-Repository acceptance revisions. For source development or the complete cross-repository suite, build the checked-out submodule instead:

```
task cli:build
task cli:test
task cli:install
dotx --version
```

`task cli:install` installs the pinned checkout into `GOBIN` or `GOPATH/bin` and installs Bash completion. Keep that directory on `PATH`; project tasks, test harnesses, examples, and CI all invoke the same `dotx` command rather than a repository-local binary.

Run GitHub workflow gates locally

GitHub Actions and local pre-push validation share the root Taskfile test bodies. Actions still provisions clean toolchains and publishes artifacts, but the build, validation, conformance, coverage, runtime, and documentation commands are not CI-only shell fragments.

```
task ci:prepare
task ci:deterministic
task ci:compatibility-matrix
```

`ci:prepare` installs the pinned DOT-CLI checkout and verifies that `dotx version` matches the root `VERSION`. `ci:deterministic` runs every deterministic non-live release workflow gate, including Java and Go tests, schema drift, official Contract validation, repository and Anchor conformance, Composition v1, lifecycle recovery, upgrade, all PoC dry-runs, external-DOT rendering, coverage, Antora, and PDF generation. Individual equivalents are available as `ci:build`, `ci:contract-conformance`, `ci:repository-conformance`, `ci:coverage`, and `ci:docs`.

On a Linux host with all runtime prerequisites, run the mutable workflows too:

```
task ci:live
task ci:all
```

The live gate requires Docker, Minikube, Kubernetes, Helm, Wasmtime, Vagrant/libvirt, writable KVM access, Rust 1.98.1 with the `wasm32-wasip2` target, and network access for the first official WASI HTTP sample build. The shared task fetches the exact pinned `sample-wasi-http-rust` revision into an ignored test cache, builds it with Cargo, and passes it to the runtime test, so local and GitHub executions cannot silently omit the WASI HTTP assertions. Set `WASI_HTTP_COMPONENT` only to reuse an already built component. The live gate performs runtime mutation and cleanup; `ci:deterministic` is the normal pre-push default.

For a Vagrant runtime on Ubuntu, install or reconcile the complete provider boundary and then open a new login session so the `libvirt` and `kvm` groups are active:

```
dotx tool install vagrant
dotx tool check vagrant
```

The installer selects `qemu-kvm` when the distribution provides it and falls back to `qemu-system-x86` on releases such as Ubuntu 26.04. The readiness check also verifies the pinned `vagrant-libvirt` plugin, the `qemu:///system` connection, the default libvirt network, and `/dev/kvm` access.

Editor assistance from the domain schemas

After changing DOT-Domain, generate schemas with the Java build, then run these from the DOT-Design checkout:

```
task cli:domain:generate
task cli:domain:check
task cli:domain:generation:test
```

The read-only check covers all generated Go domain bindings, embedded validation schemas, the authoring output projection and Composition artifacts. It reports changed, missing or orphaned generated files without repairing the checkout. Fix drift by regenerating and reviewing it, not by editing generated Go files. The authoring projection derives field names, types and order from the domain schemas/generated bindings; frozen outputs protect readable JSON and existing omission behavior. Node and quicktype remain build-time tools, not dependencies of the installed CLI.

Core model and field descriptions are generated from DOT-Domain's Jackson annotations. Editors can show these descriptions alongside the same types, required fields and exact enum spellings used by validation. Do not edit generated schemas to change model documentation.

For a DOT-Design workspace, a VS Code JSON association can be configured in your local user/workspace settings (not in `dot.json`):

```
{
  "json.schemas": [
    {
      "fileMatch": ["**/dot.json"],
      "url": "./DOT-Repository/Schema/schema.json"
    }
  ],
  "yaml.schemas": {
    "./DOT-Repository/Schema/composition-v1.schema.json": "**/composition.yaml"
  }
}
```

The YAML association requires an editor YAML language extension. In a standalone DOT-Repository checkout, use `./Schema/` instead. Keep associations editor-side: adding `$schema` to a portable Composition is not supported. Editor hints do not replace `dotx validate`, semantic validation, reviewed locks or live acceptance.

Validate a DOT

`dotx validate` reads `dot.json` at the project root, falling back to `dot/dot.json`. The selected file is the unique DOT deployment definition; unreferenced Dockerfiles, Compose files, Kubernetes manifests, Helm charts, and scripts are not inferred.

```
dotx validate DOT-Repository/DOTs/testing/contract-standards/provider \
  --schema DOT-Repository/Schema/schema.json
dotx validate DOT-Repository/DOTs/external/Keycloak-DOT \
  --schema DOT-Repository/Schema/schema.json --profile reusable
dotx inspect DOT-Repository/DOTs/testing/contract-standards/provider --json
```

Base validation checks the JSON Schema and DOT-level Anchor, Contract, service-port, CDI, and sensitive-default relationships. Every declared Contract must be a complete URL/SHA-256 declaration in the owning `dot.json`. The validator never searches repository-level Contract directories or sibling DOTs. A bundle can therefore be copied and validated independently with an explicit schema path. A URL-backed definition declares one structured payload:

```
"payload": {
  "url": "https://raw.githubusercontent.com/vendor/project/v1.2.3/openapi.yaml",
  "sha256": "0123456789abcdef0123456789abcdef0123456789abcdef0123456789abcdef"
}
```

Raw Contract text and the former flat `payloadUrl`/`payloadSha256` shape are rejected. Validation downloads the exact bytes, rejects non-HTTPS or mutable raw-GitHub branch URLs, verifies the digest, parses the declared standard/format, and resolves Anchor selectors. Missing IDs, incomplete declarations, digest mismatches, and unresolved selectors always fail validation. Use `--profile reusable` before publishing a stable reusable DOT. That profile additionally requires at least one pinned Contract, rejects unused Contracts, and requires every deployment to bind each Contract-backed provided Anchor exactly once through a service that declares the same Contract. Explicit contractless Anchors retain base-validation evidence only; they are never silently promoted to Contract-backed interoperability.

For an external product, prefer a complete root Contract declaration pointing to an official versioned vendor raw file. A full Git commit or release-version tag is accepted for `raw.githubusercontent.com`; `main`, `master`, and other mutable branch names are rejected. When no official usable raw machine-readable Contract exists, catalogue declarations may point to a clearly attributed derived/adapter document hosted in a secret GitHub Gist. Use the HTTPS host `gist.githubusercontent.com` with path `owner/gist-id/raw/40-character-revision/filename` and the SHA-256 of the exact raw bytes. Unversioned Gist URLs, HTML Gist pages and Gist query parameters are rejected. A secret Gist is unlisted, not private; anyone with its URL can read it without authentication. The CLI does not send GitHub tokens to Gist downloads. Keep the original source, license and authority in the Gist's `PROVENANCE.json`; hosting does not make a derived document official or prove live acceptance. Existing unresolved license notices remain unresolved.

All current catalogue declarations now include their URL-backed metadata. No local Contract directory is required or used. Compositions select the exact owning DOT with `source:`

`repository://name@release`, retaining the originating Contract ID. Run `dotx validate DOT-Repository` online once, then use `dotx validate DOT-Repository --offline`. The shared digest cache lives under `$XDG_CACHE_HOME/dotx/contracts/sha256/` (normally `~/.cache/` on Linux). Every cache read rechecks SHA-256; a cold offline cache fails without downloads or writes, and corruption is never silently repaired. Payload availability is not producer acceptance. Shared Contract IDs still require identical metadata and bytes; root ID-only declarations cannot fall back to local or sibling files. For private raw GitHub URLs, `dotx` reads `MARE_READ_ONLY_TOKEN`, falling back to `GITHUB_TOKEN`, and sends it only to `raw.githubusercontent.com`; credentials must never appear in `payload.url`. Inspection provides a normalized, machine-readable view without deploying anything.

Validate and resolved composition a composition

The canonical example connects Keycloak, APISIX, and an OAuth client:

```
dotx compose lock \
  DOT-Repository/Compositions/keycloak-apisix-oauth/composition.yaml --check

dotx compose validate \
  DOT-Repository/Compositions/keycloak-apisix-oauth/composition.yaml

dotx compose inspect \
  DOT-Repository/Compositions/keycloak-apisix-oauth/composition.yaml

dotx compose inspect \
  DOT-Repository/Compositions/keycloak-apisix-oauth/composition.yaml --json

dotx compose apply \
  DOT-Repository/Compositions/keycloak-apisix-oauth/composition.yaml --dry-run
```

All five commands above are non-mutating. `lock --check` verifies the committed content-addressed input graph; `validate` answers whether references and compatibility can be resolved; `inspect` shows the exact instances, endpoints, security status, and materializers; `apply --dry-run` renders supported runtime actions without writing composition state.

Read Validation, resolution, and lifecycle before a live apply.

Verify the authoritative repository

Recursively discover and validate every definition. External/internal paths record ownership; Contract-backed external/internal bundles receive the strict reusable profile:

```
dotx validate DOT-Repository
dotx repository inspect \
  contract-standards-provider@1.3.0 \
  --repository DOT-Repository --json
```

The same command is the required DOT-Repository GitHub Actions gate for every change below

`DOTs/` or `.gitmodules`. The filesystem is authoritative; no separate inventory or validation wrapper is maintained.

Run `task testing:contracts:portability` to copy every bundle into an isolated temporary directory and validate it offline against the explicit catalogue schema. This is also part of `ci:repository-conformance`; it prevents a valid repository checkout from concealing a DOT’s missing local Contract asset.

Run the composition conformance gate

```
task testing:conformance
```

This gate combines Java domain tests, Go tests, repository-wide schema validation, Contract checks, composition validation/resolution, and dry-run scenario checks. It does not claim that external partner systems or production networks were exercised.

Run only the versioned composition production profile with:

```
task testing:compose:acceptance
```

This gate writes revision-pinned, secret-free JSON evidence below `DOT-Testing/e2e/target/`.

Build the documentation

Install the documentation toolchain once:

```
task docs:setup
```

Then generate the Antora HTML site and PDF export:

```
task docs:check
task docs:build
```

Generated artifacts are written under `DOT-Docs/build/site` and are intentionally ignored by Git. To build and serve the site locally:

```
task docs:run
```

Open `http://localhost:8000`. Do not edit files under `DOT-Docs/build`; update the AsciiDoc sources instead.

Mermaid sources are stored as Antora attachments and rendered locally by `task docs:diagrams`. The renderer produces SVG for the HTML site and high-resolution PNG for PDF export; both outputs are committed so a clean checkout can resolve every Antora resource. Each source includes an accessible title and description, and each AsciiDoc image macro supplies alternative text.

`task docs:build` regenerates the diagrams before building either output. The toolchain is version-pinned and does not send architecture content to an external rendering service. Edit the `.mmd` source, never the generated SVG or PNG, then run the full documentation build to verify both renderers.

Where to continue

- Guided demo for a presentation-ready walkthrough from catalogue validation to cross-standard resolution and optional live probes.
- Architecture for system and ownership boundaries.
- DOT Anchors and Composition for the model overview and task-oriented composition guides.
- Verification and examples for evidence levels and MARE PoC scenarios.
- DOT-Domain for Java model details.
- Integration handoff for catalogue and composer implementers.

Architecture

DOT-Design separates reusable component description from deployment-instance topology. That separation allows a DOT to be authored and validated once, then instantiated in multiple compositions without embedding addresses, credentials, or orchestrator-specific connection objects in the DOT aggregate.

The diagrams use a consistent visual language: blue identifies authored or persistent artifacts, teal identifies validation and resolution activities, amber identifies decisions or incomplete evidence, green identifies successful outcomes, and red identifies rejection. Labels and line styles carry the same meaning when the documentation is printed without color; dashed lines indicate optional, indirect, or evidence-only relationships.

System context

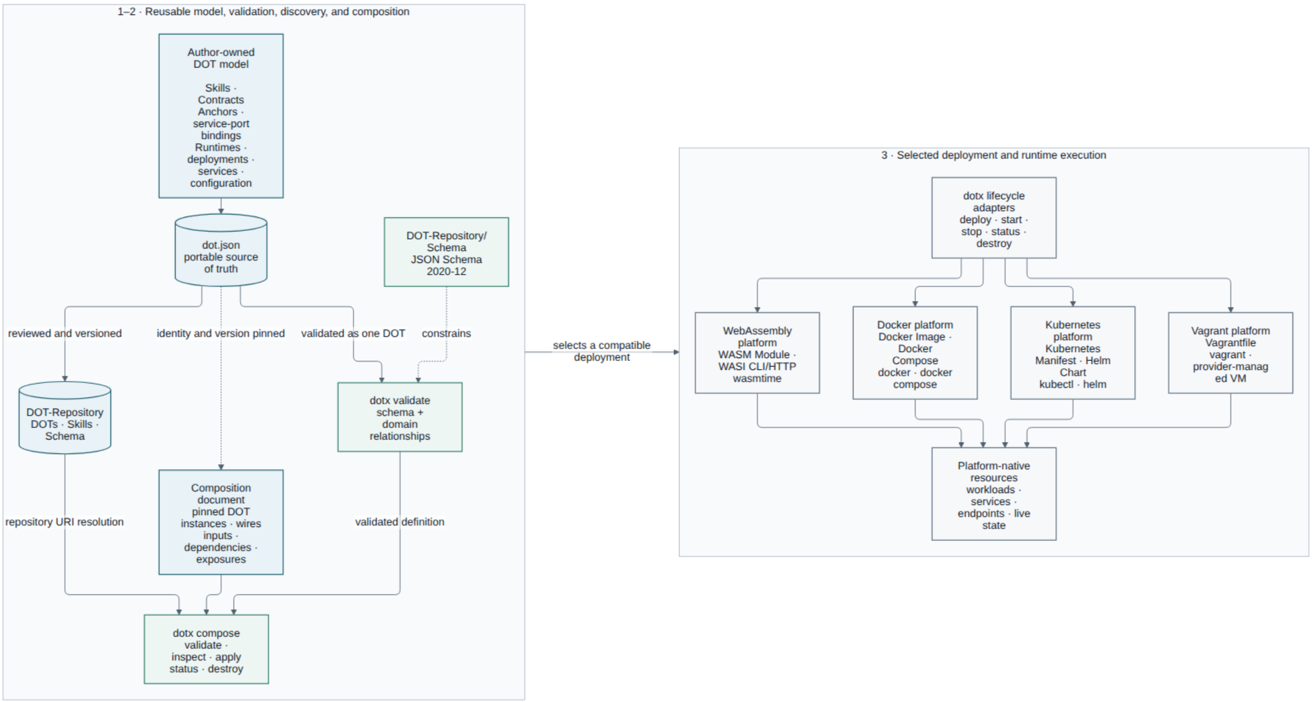

1–2 · Reusable model, validation, discovery, and composition
Author-owned DOT model
Skills · Contracts Anchors · service-port bindings Runtimes · deployments · services · configuration
dot.json portable source of truth
DOT-Repository/ Schema JSON Schema 2020-12
reviewed and versioned
identity and version pinned
validated as one DOT
constrains
DOT-Repository DOTs · Skills · Schema
dotx validate schema + domain relationships
Composition document pinned DOT instances · wires inputs · dependencies · exposures
repository URI resolution
validated definition
dotx compose validate · inspect · apply status · destroy
selects a compatible deployment
3 · Selected deployment and runtime execution
dotx lifecycle adapters deploy · start · stop · status · destroy
WebAssembly platform WASM Module · WASI CLI/HTTP wasmtime
Docker platform Docker Image · Docker Compose docker · docker compose
Kubernetes platform Kubernetes Manifest · Helm Chart kubectl · helm
Vagrant platform Vagrantfile vagrant · provider-managed VM
Platform-native resources workloads · services · endpoints · live state

Figure 3. DOT-Design system context and runtime support

`DOT-Repository` is the authoritative catalogue of reusable definitions. `dotx` is the reference validation, resolution, deployment, and composition client. Platform-native systems remain responsible for workload execution and live state.

The runtime portion deliberately distinguishes **where** a workload runs from **how** it is deployed. The domain model supports four `PlatformEngine` values and six compatible manifest mappings:

- Docker accepts `DOCKER_IMAGE` and `DOCKER_COMPOSE`;
- Kubernetes accepts `KUBERNETES_MANIFEST` and `HELM_CHART`;
- Vagrant accepts `VAGRANTFILE`; and
- WebAssembly accepts `WASM_MODULE`.

The current `dotx` lifecycle implements Docker Image, Docker Compose, Kubernetes manifest, Helm chart, Vagrantfile, and WebAssembly execution. The WebAssembly adapter uses Wasmtime with an explicit `WASI_CLI` or `WASI_HTTP` world, verifies the module's SHA-256 before execution, and grants only declared guest environment and directory capabilities.

Source-of-truth boundaries

All portable model definitions originate in `DOT-Domain`, including the separate `Composition` aggregate. The existing generation pipeline is Java/Jackson/Jakarta constraints → Victools JSON Schemas → quicktype Go input bindings. `DOT-Repository/Schema/` stores the generated artifacts; the CLI embeds the same Composition schema. There is no independent CLI input model or hand-authored production schema to keep in sync. Resolved compositions, locks and runtime state are executor representations, not alternative domain definitions.

Concern	**Authoritative source**	**Explicitly excluded**
DOT identity and supported capability	`dot.json`: `id`, `name`, `version`, Skills, and Contracts	Composition-local instance names and live state
Contract semantics	OpenAPI, AsyncAPI, Protobuf, or GraphQL Contract payload	Repeated protocol, media, schema, or security fields on a contract-backed Anchor
Logical composition point	`Dot.anchors[]`	Hosts, URLs, credentials, placement, health, and policy decisions
Deployable endpoint declaration	`DeploymentService.ports[]` and `anchorBindings[]`	Allocated addresses, forwarded ports, and certificates
Runtime configuration surface	`DeploymentService.config`	Operator-supplied values already applied to one deployment
Multi-DOT topology and readiness	Composition document wires and `dependsOn`	Changes to either participating `dot.json`
Runtime state	Platform-native resources plus private `dotx` composition state	Contract or Anchor metadata

Repository architecture

```
DOT-Design
 ├──── DOT-Domain       Java model and validation
 ├──── DOT-CLI          independent Go CLI and reference executor
 ├──── DOT-Repository   DOTs, high-level Skills, and generated Schema (submodule)
 ├──── DOT-Testing      conformance, composition, interoperability, and E2E evidence
 └──── DOT-Docs         Antora sources, UI, playbooks, and generated diagrams
```

The Maven build contains the `DOT-Domain` Java module. `DOT-CLI` has its own Go module and lifecycle. `DOT-Testing` is deliberately outside both module hierarchies. `DOT-Repository` is the only admitted source for valid deployable DOT definitions used by shared tests. It also owns the generated JSON Schema and the optional high-level Skill classification catalogue, avoiding separate repositories for assets that evolve with the DOT catalogue.

DOT aggregate

`Dot` is the aggregate root. Its primary relationships are:

```
Dot
 ├─── Skills[]                  human-readable capabilities
 ├─── Contracts[]               authoritative interaction definitions
 ├─── Anchors[]                 stable logical composition points
 └─── Runtimes[]
       ├─── Platform            where execution occurs
       └─── Deployments[]
             ├─── DeploymentManifest   how deployment occurs
             └─── DeploymentService[]
                   ├─── Contract references
                   ├─── ServicePort[]
                   ├─── AnchorBinding[]
                   └─── DeploymentConfig
```

`Skill` and `Contract` are independently identified domain entities. Root `Dot.contracts` carries authoritative Contract declarations (with legacy UUID-only catalogue references accepted during migration), while deployment services continue to reference those Contracts by ID only. Anchors, Runtimes, Deployments, services, bindings, and configuration declarations are embedded values owned by the DOT aggregate.

The class-level `@ValidAnchor` constraint delegates aggregate traversal to `AnchorValidator`. It enforces Anchor identity, Contract membership, selector syntax, and service-port binding consistency without placing cross-reference logic in `Dot`.

Contract, Anchor, binding, and wire

These four concepts form a resolution chain but have different owners:

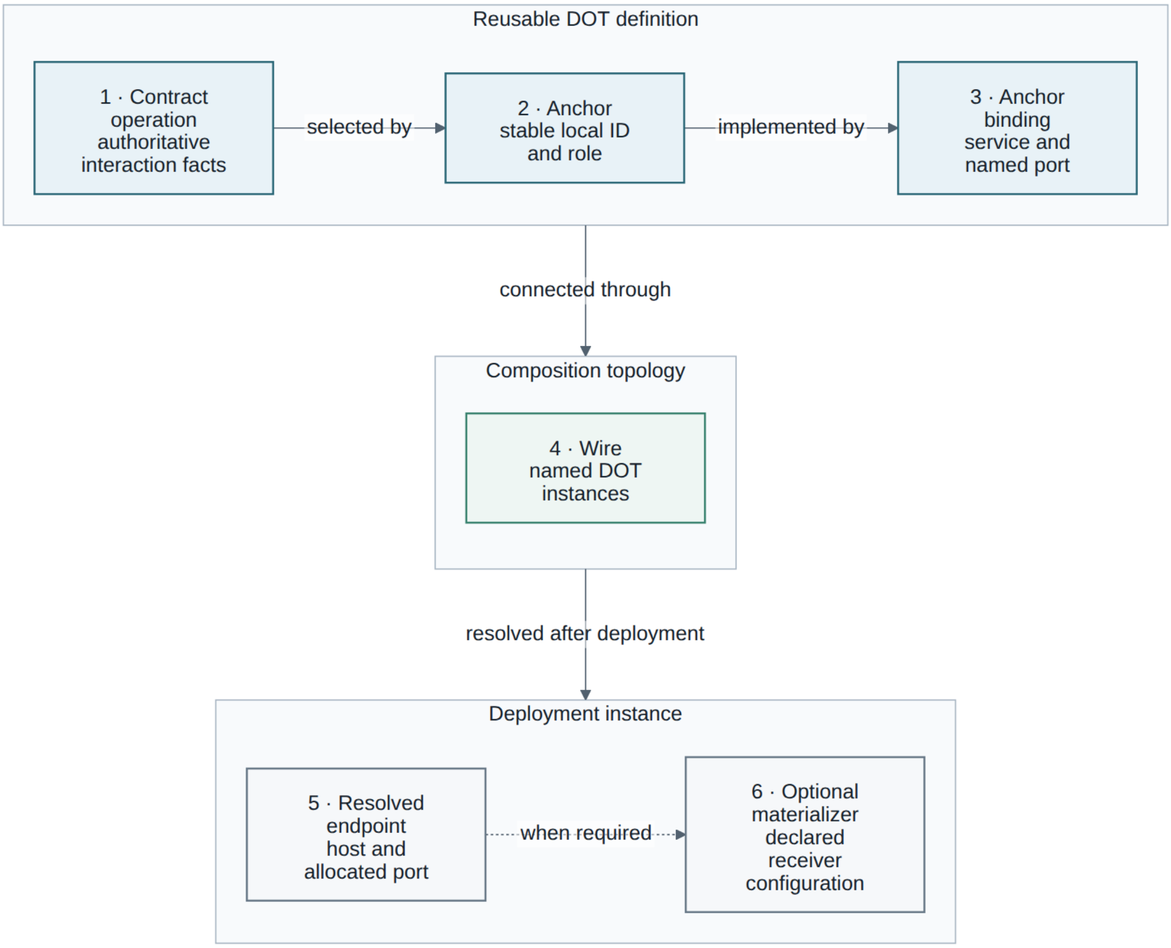

Reusable DOT definition
1 · Contract operation authoritative interaction facts
selected by
2 · Anchor stable local ID and role
implemented by
3 · Anchor binding service and named port
connected through
Composition topology
4 · Wire named DOT instances
resolved after deployment
Deployment instance
5 · Resolved endpoint host and allocated port
when required
6 · Optional materializer declared receiver configuration

Figure 4. From an authoritative Contract operation to deployed configuration

- The **Contract** defines the interaction.
- The **Anchor** identifies one logical participation point.
- The **binding** says which declared service port implements that Anchor.
- The **wire** connects named deployment instances.
- The optional **materializer** writes a resolved value into configuration that the receiving service has already declared.

See DOT Anchors and Composition for the field model, compatibility rules, and examples.

Runtime and deployment separation

The model separates two orthogonal questions:

Concept	Field	Answers
`PlatformEngine`	`Runtime.platform.engine`	Where does the workload run? For example Docker, Kubernetes, Vagrant, or WebAssembly.
`DeploymentManifestType`	`Deployment.manifest.type`	How is it deployed? For example Docker image, Kubernetes manifest, Helm chart, Vagrantfile, or WASM module.

Every manifest type declares its target platform. Runtime validation rejects a deployment whose manifest mechanism does not align with its platform.

`DeploymentService` is the unit that binds a deployed service to Contracts, named ports, Anchors, and supported configuration. Configuration is service-scoped; there is no deployment-level shared configuration surface.

Contract resolution

The reference resolver maps an Anchor selector to an authoritative Contract element:

```
AnchorContract(id, selector)
        |
        v
load Contract by case-sensitive ID
        |
        v
standard-specific resolver
        |
        v
identifier + interaction + protocols + media types
          + input/output schema references + security scheme names
```

Resolution is read-only. Derived facts are returned to consumers and never written back into the Anchor. Contractless Anchors already carry their explicit direction, protocols, and media types and therefore have no selected Contract element.

Composition resolution

The stable Composition v1 executor uses a compile-like pipeline:

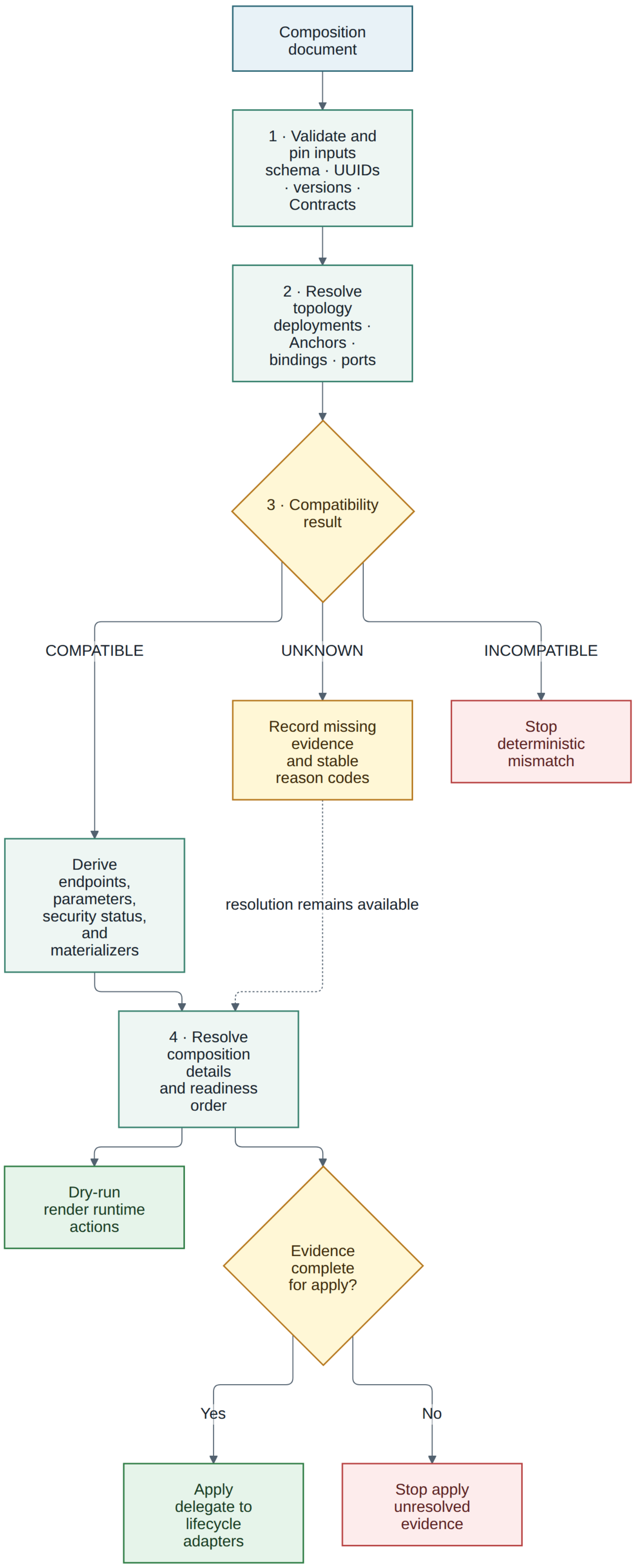

Composition document
1 · Validate and pin inputs schema · UUIDs · versions · Contracts
2 · Resolve topology deployments · Anchors · bindings · ports
3 · Compatibility result
COMPATIBLE
UNKNOWN
INCOMPATIBLE
Record missing evidence and stable reason codes
Stop deterministic mismatch
Derive endpoints, parameters, security status, and materializers
resolution remains available
4 · Resolve composition details and readiness order
Dry-run render runtime actions
Evidence complete for apply?
Yes
No
Apply delegate to lifecycle adapters
Stop apply unresolved evidence

Figure 5. Fail-closed composition resolution pipeline

1. Validate the strict Draft 2020-12 composition shape.
2. Load each local DOT source and verify the pinned textual ID and version.
3. Load each Contract input and verify its case-sensitive textual ID.
4. Resolve exact external Anchor Contract sources without copying their declarations or searching other bundles; lock the originating definitions.
5. Select one deployment for every named DOT instance.
6. Resolve Anchor references and service-port bindings.
7. Evaluate three-state compatibility: `COMPATIBLE`, `INCOMPATIBLE`, or `UNKNOWN`.
8. For provider-neutral OIDC requirements, verify the supported discovery profile against the actual selected provider Contract. Exact consumers retain strict same-Contract matching; this is not general OpenAPI substitutability.
9. Derive connection endpoints, explicit cross-platform network ownership, parameter values, security status, and materializers.
10. Build a deterministic resolved composition and readiness order.
11. On apply, prepare configuration and delegate workload operations to the existing single-DOT lifecycle adapters.

Resolution may retain `UNKNOWN` with reason codes so missing evidence is visible. Apply rejects `UNKNOWN` rather than treating absence of evidence as compatibility.

Security boundaries

DOT-Design models declarations and explicit grants; it does not infer trust.

- Contract security requirements remain in the Contract.
- Sensitive deployment values are supplied by reference, not embedded in a composition or persisted as resolved values.
- OCI capabilities, host namespaces, volumes, and CDI devices require exact DOT declarations and exact operator grants.
- A configured wire does not prove credential validity, authorization, remote attestation, policy compliance, or network reachability.
- An `EXTERNAL_GATEWAY` binding records operator ownership and `NONE` cleanup; it does not assert that `dotx` provisioned or verified the gateway.
- Cross-domain admission decisions belong to deployment or policy systems.

Active traffic and trust evidence must be collected by scenario-specific probes after deployment.

Catalogue and API boundaries

`DOT-Repository` versions reusable definitions and its catalogue metadata in Git. `dotx repository` validates and resolves those entries; `dotx compose` provides the executable compatibility reference. A future remote catalogue or API is free to use another language, framework, storage engine, and transport provided it preserves the domain semantics and passes the consumer acceptance profile.

`dotx compose` stores private, local lifecycle state under the user’s cache directory. Per-composition locking prevents concurrent local lifecycle commands from racing that state. The current model is therefore explicitly single-host; shared orchestration state remains future work.

Design principles

- Reuse established Contract standards instead of inventing parallel protocol models.
- Keep stable logical identity separate from runtime location.
- Make adapters, transformations, privileges, and secret access explicit.
- Fail closed on ambiguity and missing security evidence.
- Preserve independently versioned DOT definitions; keep topology in composition.
- Treat generated resolved compositions and test evidence according to what they actually prove.

DOT Anchors and Composition

This guide explains how independently authored Dynamic Open Trust (DOT) components participate in composition by referring to authoritative Contracts rather than duplicating contract-defined facts. It distinguishes the stable DOT Anchor model from the stable, versioned `dotx compose` format and follows the complete path from a logical capability to a deployed connection.

For a technology-neutral consumer boundary, see integration handoff.

Read this first

> Keep these three boundaries intact:
>
> 1. A **Contract** describes what can be exchanged.
> 2. An **Anchor** gives one Contract operation a stable DOT-local identity and role.
> 3. A **wire** connects deployed DOT instances and remains outside both DOTs.
>
> A structurally valid, deployable DOT is not automatically interoperability-ready. `dotx validate <bundle> --offline --profile interoperable` additionally requires a Contract for every Anchor and a Contract-backed provided Anchor for each exposed service port. The older `reusable` profile preserves mixed evidence during migration. Neither profile discovers undeclared implementation endpoints or replaces live producer acceptance. The catalogue Contract audit records official sources, source-derived internal descriptions, licensing boundaries, and remaining gaps. From DOT-Design, `task testing:contracts:readiness` produces a per-bundle strict-profile report; known gaps remain failures, not an interoperability certification.
>
> If a value changes between installations—for example a host, URL, port, credential, realm, broker, namespace, or device—it does not belong in an Anchor.

If you want to…	Start with
Publish a composable DOT	Authoring two composable DOTs, then Contract-backed Anchors and deployment binding.
Connect existing DOTs	Composition documents and DOT instances, direct request/response wires, and materializers.
Model messaging through a broker	Brokered wires and path and channel parameters.
Handle authentication or secrets	Security and secrets.
Operate a composition	Validation, resolution, and lifecycle and troubleshooting.
Design a new 6G integration type	Anchor boundaries, extension points, and 6G security composition patterns.

Release status and supported profile

The DOT-owned Anchor model, contract selectors, service-port bindings, validation, generated schema, repository resolution, and conformance fixtures are release candidates. The separate composition document and its local lifecycle executor implement the stable `dot.mare6g.eu/v1` profile.

The current executable profile supports:

- local DOT sources pinned by ID and version, and local Contract payloads pinned by ID, in the composition;
- OpenAPI, AsyncAPI, Protobuf, and GraphQL contract selectors;
- contractless protocol/media declarations when no supported contract exists;
- named deployment selection and service-port bindings;
- explicit environment-variable and config-file materializers;
- explicit externally owned gateway bindings for cross-platform wires, with ownership and no-cleanup semantics recorded in resolved composition, lock, and state;
- Docker Image composition networking and per-instance identity;
- Kubernetes and Helm deployment/readiness through the existing single-DOT lifecycle;
- reference-only process-environment and Kubernetes Secret inputs;
- rollback, private local state, resumable partial apply, status, and reverse-order destroy;
- contract-authorized active probes for safe OpenAPI operations, GraphQL query/mutation/SSE lifecycles, finite side-effect-free unary/client/server/ bidirectional gRPC calls, and correlated AsyncAPI MQTT/NATS/Kafka/AMQP delivery within one Docker network or Kubernetes namespace; and
- fail-closed handling of ambiguity, unknown compatibility, unsupported privilege grants, and unreachable cross-platform materialization.

The authoritative machine-readable boundary is `DOT-Testing/acceptance/composition/v1/profile.json`. Run `task testing:compose:acceptance` to compare the installed CLI help and emitted schema with that profile, verify every committed lock, exercise all twenty-one cross-standard wires and six runtime dry-run adapters, and emit revision-pinned, secret-free JSON evidence.

It is not a general multi-host production orchestrator. Shared state, composition-owned cross-platform gateways or tunnels, external secret-store plugins, multi-scheme authentication, unsafe or unbounded gRPC, AsyncAPI event protocols other than MQTT, NATS, Kafka, and AMQP 0-9-1, AMQPS/AMQP 1.0, and cross-platform probe adapters remain outside this profile. Safe anonymous OpenAPI probes, single-scheme OAuth 2.0 client-credentials probes, GraphQL query/mutation/SSE probes, all four finite side-effect-free gRPC shapes, and AsyncAPI MQTT/NATS/Kafka/AMQP delivery probes within a shared Docker network or Kubernetes namespace are supported.

Single-DOT live lifecycle evidence for a runtime adapter is not automatically multi-DOT composition acceptance. The profile records validation, resolution, dry-run, single-DOT live lifecycle, and composition live lifecycle separately for each adapter.

The five distinct objects

Object	Owner	Responsibility
Contract	DOT author	Authoritative operation, message, payload, protocol, and security description in OpenAPI, AsyncAPI, Protobuf, or GraphQL.
Anchor	DOT author	Stable DOT-local ID and composition role selecting one contract element, or declaring explicit communication facts when no contract exists.
Anchor binding	DOT author	Maps one Anchor ID to a named port of a deployment service. It does not contain a live address.
Wire	Composer/operator	Connects one named DOT instance and Anchor ID to another. It may select a provider, consumer, or broker service-port endpoint.
Materializer	Composer/operator	Explicitly writes a resolved connection value into configuration declared by a participating service.

Where each fact lives

Fact	Source of truth	Referenced by
API operation, message, schemas, protocol, media type, security requirement	Contract payload	`Anchor.contract.selector`
Logical composition point and provider/consumer role	`Dot.anchors[]`	Wire `from` or `to`
Service and named port implementing an Anchor	`DeploymentService.anchorBindings[]`	Resolved service endpoint
Configuration the service permits a deployer to supply	`DeploymentService.config`	Deployment input or materializer target
Selected DOT version, topology, readiness, parameter values, and secret references	Composition document	`dotx compose` inspection and lifecycle

Fact	Source of truth	Referenced by
Concrete runtime address and port	Explicit composition network binding or resolved service endpoint	Materializer or exposed-Anchor forwarding
Live reachability, authentication result, and policy decision	Runtime probe or external policy system	Operational evidence; never persisted as an Anchor fact

The ownership chain is:

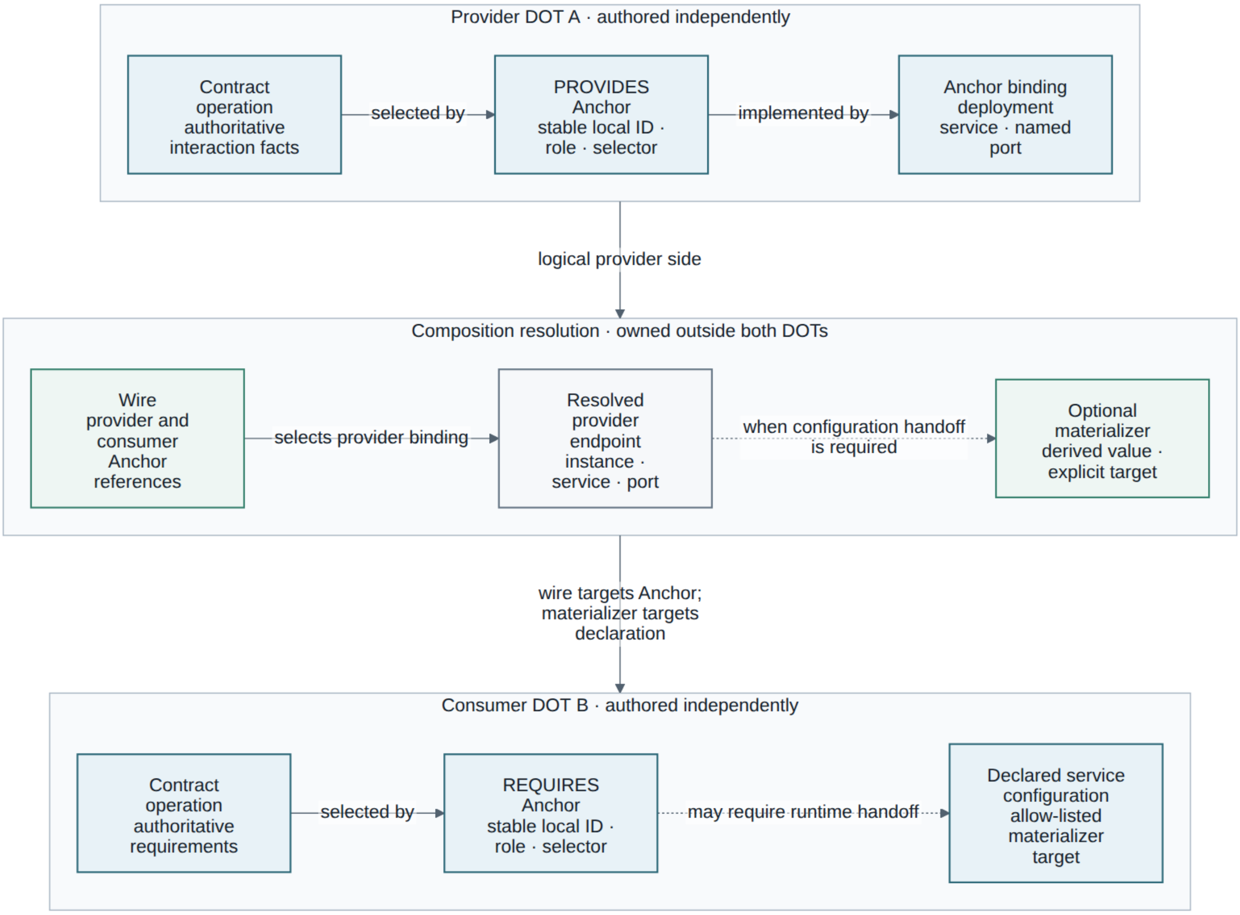

Provider DOT A · authored independently
Contract operation authoritative interaction facts
selected by
PROVIDES Anchor stable local ID · role · selector
implemented by
Anchor binding deployment service · named port
logical provider side
Composition resolution · owned outside both DOTs
Wire provider and consumer Anchor references
selects provider binding
Resolved provider endpoint instance · service · port
when configuration handoff is required
Optional materializer derived value · explicit target
wire targets Anchor; materializer targets declaration
Consumer DOT B · authored independently
Contract operation authoritative requirements
selected by
REQUIRES Anchor stable local ID · role · selector
may require runtime handoff
Declared service configuration allow-listed materializer target

Figure 6. Provider-to-consumer ownership chain

These objects must not collapse into one schema. In particular, a wire is not part of either DOT, and a live URL is not part of an Anchor.

Resolution pipeline

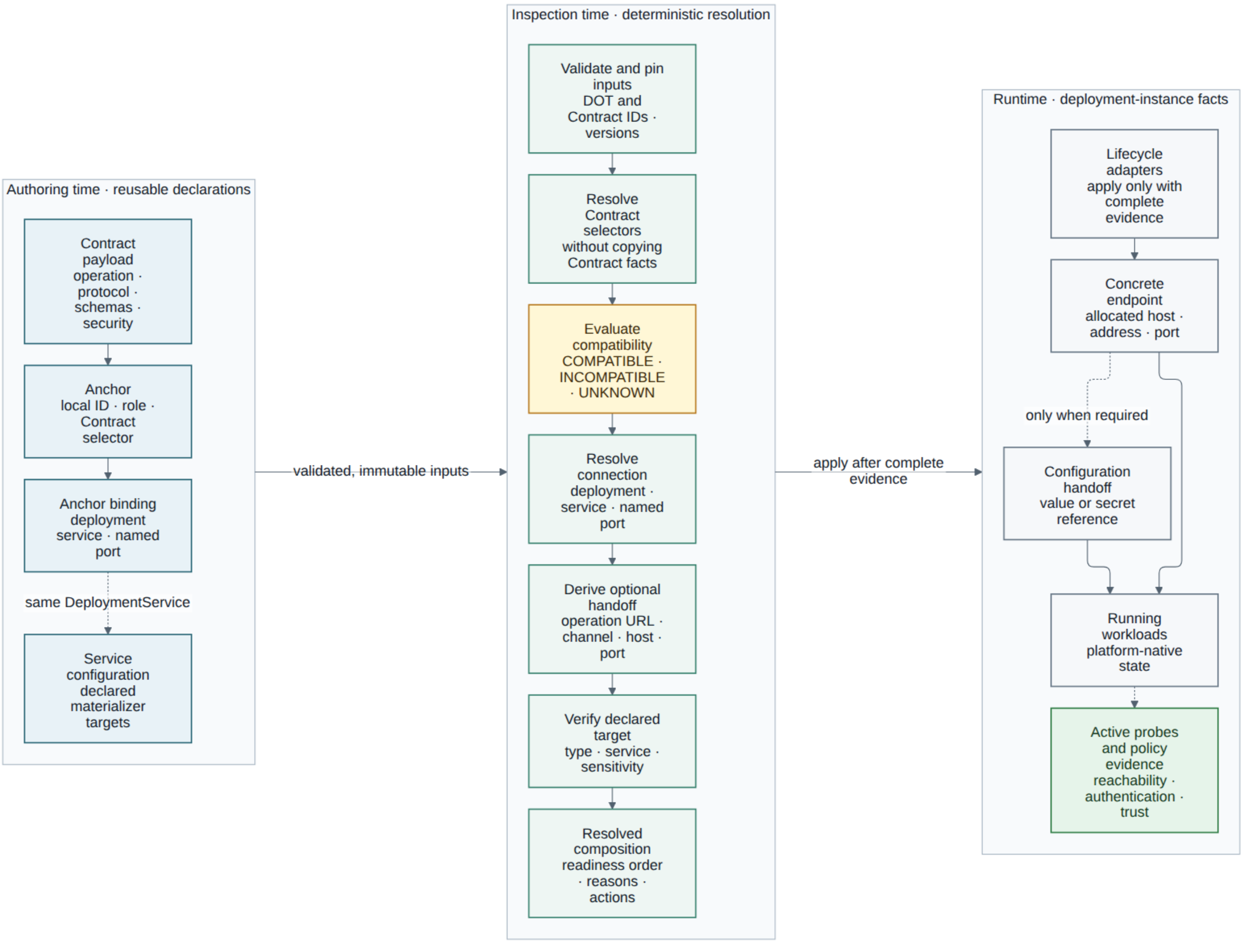

Authoring time · reusable declarations
Contract payload operation · protocol · schemas · security
Anchor local ID · role · Contract selector
Anchor binding deployment service · named port
same DeploymentService
Service configuration declared materializer targets
validated, immutable inputs
Inspection time · deterministic resolution
Validate and pin inputs DOT and Contract IDs · versions
Resolve Contract selectors without copying Contract facts
Evaluate compatibility COMPATIBLE · INCOMPATIBLE · UNKNOWN
Resolve connection deployment · service · named port
Derive optional handoff operation URL · channel · host · port
Verify declared target type · service · sensitivity
Resolved composition readiness order · reasons · actions
apply after complete evidence
Runtime · deployment-instance facts
Lifecycle adapters apply only with complete evidence
Concrete endpoint allocated host · address · port
only when required
Configuration handoff value or secret reference
Running workloads platform-native state
Active probes and policy evidence reachability · authentication · trust

Figure 7. Authoring, resolution, and runtime resolution

This resembles compilation: validate the independently authored inputs, link stable references, resolve deployment details deterministically, and only then mutate runtime state. A failure at one phase is not evidence that a later phase was attempted.

Continue through the workflow

The detailed guide is organized around the work being performed:

Task	Guide
Publish stable composition points in a DOT	Authoring Anchors
Define pinned DOT instances and composition topology	Composition documents and instances
Connect endpoints, provide security inputs, and materialize configuration	Wiring, materialization, and security
Validate, resolved composition, apply, inspect, and evolve a composition	Composition lifecycle and design guidance

Readers implementing a separate composer should also consult the integration handoff.

Authoring Anchors

This page covers the DOT publisher's side of composition. Start with the model overview if the distinction between Contracts, Anchors, bindings, and wires is not yet familiar.

Authoring two composable DOTs

Use this sequence when introducing a new connection:

1. Put the operation or message boundary in an OpenAPI, AsyncAPI, Protobuf, or GraphQL Contract. Use the contractless form only when none of those standards can describe the boundary.
2. Add a stable `PROVIDES` Anchor to the provider DOT and a stable `REQUIRES` Anchor to the consumer DOT. Select the authoritative Contract element from each Anchor; do not copy its protocol or payload fields.
3. On every deployable implementation, bind each Anchor ID to the named `ServicePort` that implements it.
4. Declare every consumer configuration input on its `DeploymentService`, including sensitivity. This declaration is the allow-list for composition.
5. Validate each DOT independently with `dotx validate` before writing a composition.
6. Add named, ID/version-pinned DOT instances and ID-pinned Contract inputs to a separate composition document.
7. Connect the Anchor IDs with a wire. Add an explicit broker endpoint, parameter values, security binding, or materializer only when the connection requires it.
8. Run `dotx compose lock` and commit the generated lock beside the composition. Review digest changes as changes to the resolved DOT-to-DOT graph.
9. Run `dotx compose validate` to check the document and `dotx compose inspect` to inspect the resolved topology. Treat `UNKNOWN` compatibility as missing evidence, not as success.
10. Use `apply --dry-run` before execution. After apply, use `compose probe` for safe Contract-derived traffic and `status` for native runtime status, configuration evidence, and the latest redacted probe evidence. Dedicated acceptance tests remain necessary for authentication, authorization, mutable operations, and application-specific outcomes.

The reusable three-DOT worked example is under `DOT-Repository/Compositions/keycloak-apisix-oauth/`: `composition.yaml` owns the wiring and references canonical DOT definitions and their local Contracts; `runtime/` holds non-secret instance inputs. The executable acceptance harness remains in `DOT-Testing/e2e/api-gateway-oidc/`.

ℹ New Anchor IDs use explicit DOT-local kebab-case keys such as `build-information`. Composition ownership identifiers still use the canonical lowercase UUIDv4 text form defined by RFC 9562. Generate a fresh Composition UUID for a new independently owned graph; do not reuse demo ownership identities. Existing Anchor UUID strings remain readable only when explicitly declared; they are not aliases for the migrated catalogue keys. Catalogue DOT IDs use `<name>@<tool-`

> `release>` and Contract IDs use `<origin-DOT.id>::<standard>::<name>`, for example `Keycloak@26.8.1::OpenAPI::Admin`. The ID identifies the interface, not an operation: the Anchor's separate selector chooses the operation/path. Consumers keep the originating Contract ID. Each DOT references at least one meaningful Skill.

Contract-backed Anchors

A contract-backed Anchor carries stable identity and selects one authoritative contract element. It deliberately omits protocol, direction, media type, payload schema, and security fields that the Contract already defines. This excerpt selects Prometheus's real build-information operation; the Anchor key is DOT-local; this excerpt is not a second catalogue definition.

```
{
  "id": "build-information",
  "name": "build-information-http",
  "role": "PROVIDES",
  "semanticTypes": ["https://prometheus.io/docs/prometheus/latest/querying/api/#build-information"],
  "contract": {
    "id": "Prometheus@3.15.1::OpenAPI::HTTP",
    "selector": "#/paths/~1status~1buildinfo/get"
  }
}
```

The opposite DOT declares its own `REQUIRES` Anchor. An exact consumer selects the same Contract ID and operation; independently authored Contracts are not automatically interchangeable. AsyncAPI additionally requires complementary operations. Unsupported matches fail rather than being inferred from names.

Ownership is not consumption

`Dot.contracts` contains this product's authoritative declarations. A consumer references an external interface on its Anchor without copying its Contract or payload. For example, a Prometheus API consumer uses:

```
{
  "id": "read-build-information",
  "name": "read-build-information",
  "role": "REQUIRES",
  "contract": {
    "id": "Prometheus@3.15.1::OpenAPI::HTTP",
    "selector": "#/paths/~1status~1buildinfo/get",
    "source": "repository://Prometheus@3.15.1"
  }
}
```

The exact origin must declare that ID. Validation resolves its verified asset and selector; another bundle's copy cannot rescue a missing origin. Run from the matching catalogue context. A detached consumer with unavailable origins fails; `--offline` never skips these checks. A DOT implementing an externally defined interface may similarly reference its origin on a `PROVIDES` Anchor without claiming authorship. Service Contract bindings describe provided APIs, not every API the service calls.

Qualified root Contract IDs must start with the **exact** owning `Dot.id` plus `::`, including the tool release. For example, `Prometheus@3.15.1` owns `Prometheus@3.15.1::OpenAPI::HTTP`, not the older `Prometheus@3.15.0::OpenAPI::HTTP`. The catalogue uses the official specification from the selected release where available, retaining upstream bytes and SHA-256 pins. A Contract document's API version remains separate from the tool release; upstream `info.version` must not be edited merely to match an ID. Derived projections stay labelled as derived, and a minor-line specification is not exact patch-version acceptance.

Exact consumers must select the originating release actually used by the wire. A reusable consumer can declare distinct optional required Anchors for supported releases; the composition selects one explicitly. This does not create version aliases, infer cross-release compatibility, or upgrade deployed software. After a declaration changes, regenerate and review the affected locks and rerun live acceptance; previous receipts do not qualify the changed graph.

Current Contract inputs use `source: repository://<owner-name>@<release>`. The exact owner supplies the declaration, raw URL and digest. Locks fingerprint that `dot.json`, never a machine-specific cache path. `compose validate`, `inspect` and `lock` accept `--offline` after one online validation has warmed the shared verified cache; use a local pinned catalogue checkout in that mode. Historical wrappers remain readable, not a fallback for missing owner metadata.

Provider-neutral OIDC discovery

APISIX accepts an OIDC provider; its reusable root must not select Keycloak or embed a Keycloak discovery Contract. Its discovery Anchor instead declares:

```
{
  "id": "oidc-discovery",
  "name": "oidc-discovery",
  "role": "REQUIRES",
  "requirement": "OIDC_DISCOVERY_1_0"
}
```

Contract, requirement and explicit protocol/media modes are mutually exclusive. The existing composition wire selects Keycloak's provider Anchor, Contract and operation. The narrow profile verifies a GET configuration endpoint and a JSON HTTP-200 object schema requiring `issuer`, `authorization_endpoint`, `token_endpoint`, `jwks_uri`, `response_types_supported`, `subject_types_supported` and `id_token_signing_alg_values_supported`, with URI strings and string-array types as appropriate. It conservatively requires token endpoint metadata; unsupported schema shapes fail closed. This is not generic OpenAPI substitutability, full OIDC certification, or live issuer/TLS validation.

`dotx compose inspect --json` reports `requirementProfile` alongside the actual provider Contract.

Locks pin external origin definitions and selected payloads. Unwired optional interfaces are not reported as connected. The unused APISIX forwarded-discovery example Anchor was removed. Protected-metadata routes are still explicitly derived, composition-configured examples, not a native APISIX Admin API or unconditional route. Use the matching source CLI; the original released v0.2.5 predates these fields and checks.

Reading compatibility results

The resolver compares the provider and consumer across independently explainable dimensions:

Dimension	Meaning
Role and direction	The connection is from `PROVIDES` to `REQUIRES`, and the selected interaction directions can communicate.
Semantics	Declared semantic-type URIs intersect when both sides provide them.
Protocol and media	Resolved protocol and media-type sets have a usable intersection.
Interaction	Request/response, publish/subscribe, streaming, query, mutation, or subscription behavior agrees.
Schema	Provider output and consumer input schema identities are compared in the correct direction when both select the same authoritative Contract; cross-Contract structural comparison remains `UNKNOWN`.
Security	Contract requirements are understood and, for execution, a complete supported binding has all required declared inputs.

`INCOMPATIBLE` means comparable evidence conflicts. `UNKNOWN` means evidence is missing or the current resolver cannot prove the dimension. `validate` and `inspect` may explain `UNKNOWN`; apply rejects it. An explicit mediator DOT is preferable to weakening compatibility when protocols or payloads genuinely differ.

Selector syntax

Standard	Selector	Example
OpenAPI	RFC 6901 URI-fragment JSON Pointer to an Operation Object	`#/paths/~1alerts/post`
AsyncAPI	RFC 6901 URI-fragment JSON Pointer to an Operation Object	`#/operations/publishAlert`
Protobuf	Fully qualified service method	`mare.security.AlertService.PublishAlert`
GraphQL	Root-field schema coordinate	`Mutation.publishAlert`

For OpenAPI and AsyncAPI, `/` in a JSON Pointer token becomes `~1` and `~` becomes `~0`. Concrete path or channel template values belong to a composition wire’s `parameters`, not to the Anchor.

Supported Contract representation matrix

Standard	**Contract** `format`	**Authoritative interaction**	**Composition evidence**
OpenAPI	`YAML` or `JSON`	Request/response operation, including path and security requirements	One shared selector; `REQUEST_RESPONSE` for both participants
AsyncAPI	`YAML` or `JSON`	Message operation and Channel Object	Distinct complementary selectors; `SEND` and `RECEIVE`, plus the resolved channel address
Protocol Buffers (Proto3; including gRPC service declarations)	`PROTOBUF`	Fully qualified RPC and unary or streaming shape	One shared method selector; `REQUEST_RESPONSE`, `CLIENT_STREAM`, `SERVER_STREAM`, or `BIDIRECTIONAL_STREAM`
GraphQL	`GRAPHQL`	SDL root field	One shared schema coordinate; `QUERY`, `MUTATION`, or `SUBSCRIPTION`

`dotx compose inspect --json` exposes the resolved `contractStandard`, `contractFormat`, `providerInteraction`, and `consumerInteraction` on each contract-backed wire. These fields are derived from the supplied Contract and therefore do not duplicate authoring metadata in either Anchor or composition. The deterministic `DOT-Testing/composition/contract-standards/` matrix connects one provider DOT to one consumer DOT through every row above.

Semantic types

`semanticTypes` is a list of URI identifiers describing the security or domain meaning of the interaction independently of its transport. Prefer stable, governed ontology or vocabulary URIs when one exists; a project URN is suitable for concepts that have no external vocabulary yet.

Current matching is deliberately conservative: both Anchors must declare at least one semantic type and their sets must intersect exactly. Missing values produce `UNKNOWN`; different values produce `INCOMPATIBLE`. The evaluator does not infer subclass, equivalence, or broader/narrower ontology relations. Such reasoning belongs in a separately reviewed semantic-policy layer.

Contractless Anchors

Use a contractless Anchor only when no supported authoritative contract exists, for example a raw packet stream or a legacy metrics boundary. It must state the facts that cannot otherwise be derived:

```
{
  "id": "packet-stream",
  "name": "packet-stream",
  "role": "PROVIDES",
  "semanticTypes": ["urn:mare:security:packet-stream"],
  "direction": "EGRESS",
  "protocols": ["TCP"],
  "mediaTypes": ["application/octet-stream"]
}
```

Contract-backed and contractless forms are mutually exclusive. A contract-backed Anchor must not repeat `direction`, `protocols`, or `mediaTypes`. A contractless Anchor must declare all three.

Role is not traffic direction

`role` answers who offers the selected capability:

- `PROVIDES` means this DOT makes the capability available;
- `REQUIRES` means this DOT depends on that capability.

It does not mean that bytes move in only one direction. An OpenAPI request/response operation has a providing server and requiring client while both send data. An AsyncAPI provider may select a `send` operation and its consumer a complementary `receive` operation. For a contract-backed Anchor, the resolver derives that interaction from the selected operation. Only a contractless Anchor states `INGRESS`, `EGRESS`, or `BIDIRECTIONAL` explicitly.

When two DOTs implement the same published Contract, referencing the same Contract ID gives the strongest deterministic evidence. Independently authored Contracts are allowed, but identical-looking schema names do not prove structural compatibility across Contract identities; that comparison remains `UNKNOWN` until a schema-aware adapter or policy evaluates it.

Stable identity and version evolution

- The exact `id`, not `name`, identifies an Anchor in a wire. New catalogue IDs use DOT-local kebab-case keys such as `search-api`.
- IDs are unique within their owning DOT. Composition references pair `instance` with `anchorId`; different DOTs may use the same local key.
- A display-name rename or documentation correction preserves the ID. Do not derive an existing ID again from its current name.
- A compatible DOT version preserves the key for the same logical Anchor.
- An incompatible semantic replacement receives a new ID.
- Legacy UUID strings remain valid explicit keys, not aliases for migrated catalogue IDs. Regenerate and review affected locks when migrating references.
- Removal and any future `deprecated`/`replacedBy` metadata are publishing governance decisions; they are not inferred from contract changes.
- Names remain useful in logs and displays but are never foreign keys.

New keys follow `^[a-z](-[a-z0-9]+)$`: no spaces, uppercase, underscores, repeated hyphens or path/fragment separators. The generated model also admits legacy UUID strings. Neither mode permits whitespace normalization. Contract identity, selector, protocol, role, tool release and deployment address remain separate; none is concatenated into the local Anchor key.

Binding an Anchor to a deployment

An Anchor is logical until a selected deployment maps it to a named service port:

```
{
  "id": "security-function",
  "name": "Security Function",
  "host": "security-function",
  "ports": [
    {"name": "http", "protocol": "HTTP", "port": 8080}
  ],
  "anchorBindings": [
    {
      "anchorId": "build-information",
      "port": "http"
    }
  ],
  "config": {
    "environment": [],
    "configFiles": [],
    "volumes": []
  }
}
```

This says which logical service and named port implement the Anchor. The deployer still decides the concrete container DNS name, cluster Service, allocated address, forwarded port, certificate, or network path.

`@ValidAnchor` and `AnchorValidator` reject duplicate Anchor IDs, undeclared Contract references, invalid selector syntax, undeclared ports, undeclared Anchor bindings, and duplicate service-local bindings.

Runtime configuration portability

Every `DeploymentService` declares its supported configuration surface through `environment`, `configFiles`, and logical `volumes`. A composition may supply only values whose names or target paths the selected service declares. Required environment variables and files without defaults fail before runtime mutation; sensitive environment values require a `secretRef` instead of a literal. When an input is omitted, `dotx` reuses the non-sensitive `defaultValue` or `defaultContent` declared by that service. Explicit composition values and wire materializers override defaults; conflicting explicit inputs fail closed. Declarations marked `sensitive` are forbidden from embedding either kind of default and must use a deployment-time secret reference or runtime-native secret facility. Dry runs print resolved environment values in stable name order for every adapter, but replace every value declared sensitive with `<redacted>`.

Each composition input carries the stable `service` id it targets. Authors may omit `service` only when exactly one service in the selected deployment declares that environment name, file path, or volume name; `dotx` resolves and records the service explicitly. If two services declare the same item, omission is an error. Duplicate detection is service-local, so two services can intentionally receive different values for the same environment-variable name.

Environment variables and configuration files are rendered by every currently executable `dotx` deployment adapter:

Manifest	Environment	Configuration files
`DOCKER_IMAGE`	Container environment	Read-only bind mounts
`DOCKER_COMPOSE`	Per-service override using process-environment staging	Generated per-service Compose override mounts
`KUBERNETES_MANIFEST`	Exact workload/container patch; existing Secret keys remain `secretKeyRef`	Service-qualified ConfigMap or Secret plus exact workload/mount patch
`HELM_CHART`	Exact post-install workload/container patch; existing Secret keys remain native	Service-qualified ConfigMap or Secret plus exact workload/mount patch
`VAGRANTFILE`	Vagrant process/provisioner environment	Files provisioned into the managed VM and exposed to the Vagrantfile
`WASM_MODULE`	Allow-listed WASI guest environment; sensitive values inherited by name	Staged configuration root plus explicit Wasmtime directory preopens

Configuration remains service-scoped. A Docker image or Vagrant deployment is one workload or VM, so several logical services may safely share its declared configuration surface. Docker Compose, Kubernetes, and Helm may materialize several independent workloads. Their composition therefore supplies `deploymentInputs.workloads`: a Compose binding names one native service; a Kubernetes/Helm binding supplies exact native labels and optionally a container. Dotx intersects those labels with its composition-owned instance label and requires exactly one workload. Missing, duplicate, cross-runtime, ambiguous workload, and ambiguous container bindings fail closed. This prevents environment values, secrets, or mounted files from reaching the wrong security function without adding platform-specific selectors to the reusable DOT model.

```
deploymentInputs:
  workloads:
    - service: api
      kubernetes:
        matchLabels:
          app.kubernetes.io/component: api
        container: api
```

Storage and privilege configuration is deliberately runtime-native. Docker image deployments support composition-scoped named volumes, explicit OCI bind sources, exact capability/host-namespace grants, and CNCF CDI devices. Compose payloads own their volume graph; Kubernetes and Helm own claims, storage classes, and volume topology; Vagrantfiles own disks and synced folders. A composer must not translate a host bind path into a cluster volume or silently grant privileged mode.

`WASM_MODULE` uses a reviewed Wasmtime/WASI profile. Each module is SHA-256 pinned and loaded from a relative/file or HTTPS URI. `WASI_CLI` executes command modules/components; `WASI_HTTP` serves a component on one declared TCP/HTTP service port bound to loopback. Dotx exposes only declared environment values, staged configuration files, and explicitly bound host directories. It does not translate OCI capabilities, host namespaces, or CDI devices into broader WASI permissions. Sensitive values are never printed or persisted. Because the Wasmtime CLI cannot enforce read-only permissions on a directory preopen, that request fails closed unless the host filesystem already supplies the boundary. The deterministic `DOT-Testing/composition/runtime-configuration/` matrix exercises every executable adapter twice: once using only declared non-sensitive defaults and once using explicit composition overrides. It checks the materialized default configuration document, the resolved service-scoped resolved composition, and the runtime-specific dry-run renderer. The five native cases reuse the external Prometheus DOT; WASI retains a separate conformance fixture. The real Prometheus acceptance runner verifies loaded configuration and environment through its official status APIs, PromQL results and invalid-query responses, stop/start, Git acquisition, and failure cleanup:

```
task testing:compose:runtime-configuration
task testing:compose:runtime-configuration:live RUNTIME=docker-image
```

Select `docker-compose`, `kubernetes-manifest`, `helm-chart`, or `vagrantfile` for the other native mechanisms. These are ephemeral single-server examples, not production monitoring configurations. The `DOT-Testing/composition/service-workloads/` matrix verifies isolation between two services across Compose, Kubernetes, and Helm dry-run adapters.

Discovering and resolving Anchors

The authoritative test and reference definitions are stored in `DOT-Repository` and addressed as `repository://<name>@<version>` for qualified IDs (legacy unqualified IDs use `repository://<id>@<version>`). Consumers can inspect one immutable definition or validate the complete repository:

```
dotx repository inspect Prometheus@3.15.1 --repository DOT-Repository --json
dotx validate DOT-Repository
```

`dotx compose validate` and `dotx compose inspect --json` resolve Anchor IDs, service-port bindings, and selected Contract elements conservatively. The resolved composition reports `COMPATIBLE`, `INCOMPATIBLE`, or `UNKNOWN`; missing evidence is never treated as a match. Contract payloads remain authoritative and are not copied into Anchors or composition wires.

This command boundary is the reference behavior, not a requirement to embed dotx. Another catalogue or composer may use any language, framework, storage, or API shape if it preserves the same identities, fail-closed resolution, and ownership boundaries. See integration handoff.

Next, define the concrete participants and topology in a composition document.

Composition Documents and DOT Instances

A composition document is owned by the composer or operator, not by any member DOT. It selects immutable DOT identities and creates deployment instances whose topology may vary between environments.

Reusable composition catalogue

`DOT-Repository/Compositions/` distributes reusable multi-DOT graphs, reviewed locks, READMEs, and non-secret deployment inputs. Start with `keycloak-apisix-oauth/` (Keycloak, APISIX, OAuth client) or `keycloak-prometheus-openapi-validation/` (official Keycloak and Prometheus OpenAPI operations). Each directory has `composition.yaml` and `composition.lock.json`.

From the standalone DOT-Repository checkout root:

```
dotx validate . --offline
composition=Compositions/keycloak-apisix-oauth/composition.yaml
dotx compose inspect "$composition" --locked --json
dotx compose apply "$composition" --locked --dry-run
```

Repository-wide composition validation requires the current source-built CLI; the released 0.2.4 binary predates this gate. It recursively checks declarations, semantic resolved compositions, adjacent locks, and READMEs without deploying or resolving Secrets. `task compositions:render` additionally renders every catalogue composition. There is no separate composition inventory or Scenario domain type.

A scenario is the broader use-case narrative and acceptance procedure described by the README. Synthetic matrices, negative controls, source-adapted PoC fixtures, executable harnesses, and evidence remain in DOT-Design's `DOT-Testing/`. See Getting started for the gateway's three Wires and Guided demo for live acceptance. Catalogue inclusion or a valid resolved composition never implies production or live producer acceptance.

Composition documents and DOT instances

A composition is a separate versioned document. It names concrete instances of DOT definitions and pins their identities through the authoritative `DOT-Repository` catalogue. The `repository://<dot-id>@<version>` form is stable across checkout locations and prevents a scenario-local copy from drifting away from the reviewed definition:

This structural excerpt omits the required instance inputs and wires. Use `DOT-Repository/Compositions/keycloak-apisix-oauth/composition.yaml` for the complete runnable document; Contract paths below are relative to that directory.

```
apiVersion: dot.mare6g.eu/v1
kind: Composition
```

```yaml
id: e3000000-0000-4000-8000-000000000001
name: keycloak-apisix-oauth
version: 1.0.0
dots:
  - name: identity
    source: repository://Keycloak@26.8.1
    dotId: Keycloak@26.8.1
    dotVersion: 26.8.1
    deployment: kubernetes
  - name: gateway
    source: repository://APISIX@3.19.0
    dotId: APISIX@3.19.0
    dotVersion: 3.19.0
    deployment: kubernetes
    dependsOn: [identity]
contracts:
  - id: Keycloak@26.8.1::OpenAPI::Discovery
    source: repository://Keycloak@26.8.1
```

`name` identifies an instance inside this composition. `source` selects the repository definition by ID and exact version. `dotx compose` locates the nearest `DOT-Repository` containing a `DOTs/` directory, discovers its definitions recursively, resolves the ID/version to its reviewed bundle, and then verifies that `dotId` and `dotVersion` match the loaded `dot.json`. Neither a repository definition nor a local path can silently substitute a different DOT. `deployment` selects a deployment by its stable deployment ID or unambiguous manifest alias.

Local discovery is the convenience default. `--repository <path-or-Git-URL>` selects another authoritative catalogue and `--git-ref <revision>` selects a Git revision. Locked and runtime-mutating commands require a full 40-character commit SHA for remote repositories; branch and tag names are allowed only for unlocked, read-only exploration. Exact commits are cached in a private user cache by normalized repository identity and commit, so the same resolved composition and lock can be reproduced offline. Resolved compositions, locks, and lifecycle state record only the credential-free identity and resolved commit. Credential-bearing URLs are rejected; authentication remains external through Git credential helpers, SSH agents, or environment-backed CI credentials.

Relative paths remain supported for authoring experiments and external Contract documents. Reusable stable, sample, and test-only DOT definitions belong in `DOT-Repository`, under their corresponding catalogue scope. Contracts remain authoritative protocol documents and are referenced independently; placing a DOT in the repository does not copy OpenAPI, AsyncAPI, Protobuf, or GraphQL facts into its Anchors.

`dependsOn` is a readiness graph, not a connection graph. A wire does not imply startup order. This distinction permits reciprocal peer wires and brokered topologies without creating false dependency cycles.

Composition field reference

The emitted Draft 2020-12 JSON Schema is normative for the serialized `v1` shape. This table is the study guide for its top-level ownership:

Field	Required	Meaning
`apiVersion`, `kind`	Yes	Fixed discriminators: `dot.mare6g.eu/v1` and `Composition`.
`id`	Yes	Stable UUID of this composition, also used to derive local state and runtime identities.
`name`, `version`	Yes	Human-readable identity and version of the composition document.
`dots[]`	Yes, at least one	Named deployment instances. Every entry pins `source`, `dotId`, and `dotVersion`; published scenarios use an exact `repository://<id>@<version>` source. `deployment`, `dependsOn`, and `deploymentInputs` refine execution.
`contracts[]`	When a referenced Contract is needed	Maps an exact Contract ID to a local Contract document. The document's embedded `id` must match.
`wires[]`	No	Connections between one `PROVIDES` and one `REQUIRES` Anchor. A composition with no wires can still expose a standalone capability.
`exposedAnchors[]`	No	Composition-local names for member Anchors that an operator may expose or forward.

Each wire requires `name`, `from`, and `to`. Both ends contain an instance name and Anchor ID. Optional `endpoint` selects a third-party or otherwise non-default service port; `parameters` supplies only Contract-declared template values; `materializer` or `materializers` writes resolved values; and `security` binds Contract security requirements to declared participant configuration. Unknown fields are rejected.

Deployment inputs and native workload bindings

`deploymentInputs.environment`, `configFiles`, and `volumes` identify the logical `DeploymentService` that owns each input. `ociRuntime` and `cdiDevices` authorize only requirements already declared by that selected DOT deployment. These fields supply instance values and grants; they do not expand the DOT's declared configuration surface.

When a Docker Compose, Kubernetes, or Helm deployment contains several logical services with configuration, `deploymentInputs.workloads` is required for every configured service. The binding is deliberately instance-owned because native service names, labels, and containers can differ between deployment variants or environments:

```
deploymentInputs:
  workloads:
    - service: api
      compose:
        service: backend
```

The Kubernetes/Helm form uses exact-match standard labels and may select one container:

```
deploymentInputs:
  workloads:
    - service: api
      kubernetes:
        matchLabels:
          app.kubernetes.io/component: api
        container: api
```

Labels are combined with dotx's instance ownership selector. A live apply must resolve exactly one Deployment, StatefulSet, or DaemonSet, and an omitted container is accepted only when that workload has one container. Two logical services cannot target the same Compose service or the same Kubernetes workload/container pair. Resolved compositions expose the binding and secret identifiers, and locks cryptographically pin them; neither contains resolved secret values.

DOT definition versus DOT instance

One pinned DOT definition can appear more than once in a composition. Each entry has a distinct composition-local `name`, runtime identity, deployment inputs, and wires while retaining the same `dotId` and `dotVersion`:

```
dots:
  - name: site-a
    source: ./federation-agent
    dotId: poc11-federation-agent@0.0.0-source.g303b353
    dotVersion: 0.0.0-source.g303b353
    deployment: docker
    deploymentInputs:
      environment:
        - {service: federation-agent, name: FEDERATION_NODE_ID, value: site-a}
  - name: site-b
    source: ./federation-agent
    dotId: poc11-federation-agent@0.0.0-source.g303b353
    dotVersion: 0.0.0-source.g303b353
    deployment: docker
    deploymentInputs:
      environment:
        - {service: federation-agent, name: FEDERATION_NODE_ID, value: site-b}
```

Wires reference `site-a` or `site-b`, never the source path. Docker Image deployments receive deterministic per-instance container, DNS, network, and named-volume identities. Apply rejects repeated instances for deployment mechanisms whose manifest-owned resource names cannot yet be isolated safely. One native Kubernetes instance may coexist with isolated Docker Image/Compose instances of that same DOT because they occupy separate resource scopes. Two native Kubernetes instances of the same DOT remain rejected, including when a Docker instance separates them in document order. See the Prometheus portability demonstration.

Changing a DOT's version-specific ID does not isolate fixed resource names or ports. Apply and upgrade retain exact-ID collision checks and also check name-version product families, including fixed-port WASI HTTP instances. This conservative guard is not an identifier alias and does not change locks.

The executable example is `DOT-Testing/composition/federation-peers/scenario.yaml`.

After defining the participants, wire and configure their selected Anchors.

Wiring, Materialization, and Security

Wires express composition topology outside every participating DOT. They refer to stable Anchor IDs, resolve deployable endpoints through service bindings, and materialize only values that the target service has declared as configurable.

Direct request/response wire

A wire uses instance names and stable Anchor IDs:

```
wires:
  - name: oauth-token-to-client
    parameters:
      realm: {value: dotx}
    from:
      instance: identity
      anchorId: client-credentials-token
    to:
      instance: client
      anchorId: oauth-token
    materializer:
      type: ENVIRONMENT_VARIABLE
      valueFrom: OPERATION_URL
      target:
        instance: client
        service: oauth-client
        environmentVariable: TOKEN_URL
```

Composition resolution performs the following sequence:

1. Resolve the two pinned DOT instances.
2. Resolve both Anchor IDs and require complementary `PROVIDES`/`REQUIRES` roles.
3. Resolve the selected Contract elements.
4. Evaluate semantic, interaction, protocol, media, schema, and security compatibility.
5. Resolve the connection endpoint through the selected deployment's `anchorBindings`.
6. Verify that the target service declares `TOKEN_URL`.
7. Build `OPERATION_URL` from the runtime endpoint and the authoritative OpenAPI server/operation

path.

The resulting URL is deployment data. It is never copied back into `dot.json` or the Anchor.

Cross-platform external gateway

A wire whose participants use different runtime platforms must declare a reviewed network adapter. The first supported adapter represents a gateway that the operator provisions outside `dotx`:

```
wires:
  - name: identity-discovery-to-proxy
    from: {instance: identity, anchorId: oidc-discovery}
    to: {instance: proxy, anchorId: oidc-discovery}
    network:
      adapter: EXTERNAL_GATEWAY
      host: identity-gateway.example.net
      port: 8080
```

The Contract and bound service port remain authoritative for protocol and interaction semantics. The binding replaces only the deployment-specific host and port used by materializers. The resolved composition, lock, and private lifecycle state record `ownership=EXTERNAL` and `cleanup=NONE`; apply, resume, status, and destroy never claim to create, inspect, or remove the gateway. Cross-platform wires without a binding, same-platform wires with a redundant binding, unknown adapters, URL-shaped hosts, and invalid ports fail before runtime mutation.

Composition-owned gateways, ingresses, tunnels, and brokers require dedicated lifecycle adapters and remain unsupported.

A wire and a materializer are intentionally separate. The wire says *these Anchors are connected*; the materializer says *write this derived value into this declared consumer setting*. A wire needs no materializer when the runtime already provides discovery, a service mesh performs routing, both participants use a fixed broker declared elsewhere, or the composition is documenting an Anchor relation without an executable handoff. Apply still requires every actually required service configuration value to have a default or explicit input.

For the example above, the human-readable resolved composition makes every resolution step inspectable before apply:

```
oauth-token-to-client:
  identity/client-credentials-token
    -> client/oauth-token [COMPATIBLE]
  endpoint: keycloak:8080
    (HTTP, instance=identity service=keycloak port=http)
  contract: Keycloak@26.8.1::OpenAPI::OIDC
    selector=#/paths/~1realms~1%7Brealm%7D~1protocol~1openid-connect~1token/post
  materializer: OPERATION_URL -> client/oauth-client env:TOKEN_URL
```

The same resolved composition with `--json` is the machine-readable automation output. It includes stable instance and Anchor references, compatibility reasons, the selected Contract, resolved endpoint, derived operation/channel value, materializer targets, security status, and exposed-Anchor endpoints. Consumers should parse JSON rather than scrape the human-readable form.

Brokered wire

For messaging, the semantic provider, semantic consumer, and connection endpoint may be three different instances. Select the broker explicitly:

```
wires:
  - name: openc2-command-via-mqtt
    from:
      instance: interpreter
      anchorId: send-openc2-command
    to:
      instance: actuator
      anchorId: receive-openc2-command
    endpoint:
      instance: broker
      service: mqtt
      port: mqtt
    materializers:
      - type: ENVIRONMENT_VARIABLE
        valueFrom: ENDPOINT_HOST
        target:
          instance: actuator
          service: openc2-actuator
          environmentVariable: MQTT_BROKER_HOST
      - type: ENVIRONMENT_VARIABLE
        valueFrom: ENDPOINT_PORT
        target:
          instance: actuator
          service: openc2-actuator
          environmentVariable: MQTT_BROKER_PORT
```

The AsyncAPI operations and message schema stay in the Contract. The broker’s host and port stay in the resolved composition. If a channel has a templated address, the wire supplies only values for parameters declared by the Contract.

Materializers

Materializers are explicit adapters between a resolved endpoint and a configuration surface already declared by a service.

Type	Behavior
`ENVIRONMENT_VARIABLE`	Writes one resolved scalar to a declared environment variable.

Type	Behavior
`CONFIG_FILE`	Mounts a declared file and optionally replaces one YAML/JSON value selected by an RFC 6901 JSON Pointer.

Supported `valueFrom` values are:

Value	Result
`ENDPOINT_HOST`	Resolved service host only.
`ENDPOINT_PORT`	Resolved numeric service port only.
`ENDPOINT_HOST_PORT`	`host:port`.
`ENDPOINT_URL`	`scheme://host:port`, using the bound port protocol.
`OPERATION_URL`	Endpoint URL plus the authoritative OpenAPI server base and operation path.
`CHANNEL_ADDRESS`	Address of the authoritative AsyncAPI Channel Object.

An optional RFC 6570 `uriTemplate` can derive an application-specific URI from one resolved scalar. It is explicit composition behavior, not an Anchor or Contract fact.

Configuration-file materialization

A `CONFIG_FILE` materializer always names an operator-controlled source and a file path already declared by the target service. Without `valueFrom`, the source is supplied unchanged. With `valueFrom`, the source must be YAML or JSON and `jsonPointer` selects the scalar to replace:

```
materializer:
  type: CONFIG_FILE
  source: ./prometheus/prometheus.yml
  valueFrom: ENDPOINT_HOST_PORT
  target:
    instance: prometheus
    service: prometheus
    path: /etc/prometheus/prometheus.yml
    jsonPointer: /scrape_configs/0/static_configs/0/targets/0
```

This produces a deployable Prometheus configuration without putting the provider’s runtime address in either DOT. The reusable executable graph is `DOT-Repository/Compositions/node-exporter-prometheus-monitoring/composition.yaml`; the broader PoC4 validation-only fixture remains under `DOT-Testing/composition/`.

URI-template derivation

Use `uriTemplate` only when the application needs an explicit deterministic derivation that is not already the selected Contract operation URL:

```
materializer:
  type: ENVIRONMENT_VARIABLE
  valueFrom: ENDPOINT_URL
  uriTemplate: "{+value}/realms/drax"
  target:
    instance: proxy
    service: proxy-xapp
    environmentVariable: OIDC_ISSUER
```

The RFC 6570 variable is named `value`. The template belongs to the composition because it adapts one deployment to another; it does not amend the Contract or redefine the Anchor.

Materialization fails when the target configuration is undeclared, the Contract path/channel remains unresolved, alternative server paths are ambiguous, two wires produce conflicting values for one target, or the endpoint and target are on different runtime platforms without an explicit network binding.

Path and channel parameters

Only deployment-specific scalar values belong in the wire:

```
parameters:
  realm:
    value: drax
```

The Contract remains authoritative for the parameter name, location, requirement, schema, default, and serialization rules. Current composition bindings cover OpenAPI path parameters and AsyncAPI channel parameters. Query/header/cookie parameters, complex serialization, and runtime-derived parameter values require explicit future adapters.

Security and secrets

Contract security remains authoritative. A wire may demonstrate how one declared scheme is satisfied by deployment configuration:

```
security:
  bindings:
    - scheme: bearerAuth
      mechanism: https://www.rfc-editor.org/rfc/rfc6749#section-4.4
      participant: {instance: client, service: oauth-client}
      parameters:
        tokenUrl: {environmentVariable: TOKEN_URL}
        clientId: {environmentVariable: CLIENT_ID}
        clientSecret: {environmentVariable: CLIENT_SECRET}
```

Sensitive inputs are references, never secret values. They are scoped to the DOT instance whose selected deployment consumes them:

```
dots:
  - name: client
    source: repository://oauth-client@1.0.0
    dotId: oauth-client@1.0.0
    dotVersion: 1.0.0
    deployment: kubernetes
    deploymentInputs:
      environment:
        - name: CLIENT_SECRET
          secretRef:
            store: kubernetes
            name: oauth-client
            key: client-secret
```

Resolved compositions and composition state keep only the reference. Docker deployments can resolve `store: environment` at final handoff; Kubernetes and Helm preserve `store: kubernetes` as `secretKeyRef` without reading the Secret. A `CONFIGURED` resolved composition proves configuration completeness, not token validity, issuer trust, mTLS, workload identity, or remote attestation.

Exposed Anchors

A composition may re-export a member Anchor under a composition-local name:

```
exposedAnchors:
  - name: protected-metadata-through-gateway
    target:
      instance: gateway
      anchorId: protected-metadata
```

At apply time, an operator may request a local forward:

```
dotx compose apply scenario.yaml \
  --forward protected-metadata-through-gateway=19080
```

The exposure name belongs to the composition boundary. It does not rename or replace the member Anchor ID.

Before runtime mutation, follow the validation and lifecycle workflow.

Composition Lifecycle and Design Guidance

The lifecycle deliberately separates reproducible input locking, model validation, deterministic resolution, runtime mutation, recorded state, and active acceptance evidence. This keeps review and CI read-only while making execution recoverable and auditable.

Use `dotx compose inspect composition.yaml --locked` to explain the resolved DOTs, deployments, Contracts, wires, endpoints and configuration destinations before execution. It does not deploy workloads or probe services, although it may fetch and cache referenced definitions and Contracts. Use `topology` for a graph, `apply --dry-run` for deployment commands, and `status` for recorded runtime state. `inspect --json` returns the resolved composition as structured JSON.

The resolved composition is computed from your document and the referenced DOTs; it is not another file to author or another portable domain model. Locks and saved execution state use `resolutionDigest`. Regenerate and review locks after updating the CLI. Before changing binaries, clean up running compositions with their original CLI: saved state is not migrated or adopted.

Validation, resolution, and lifecycle

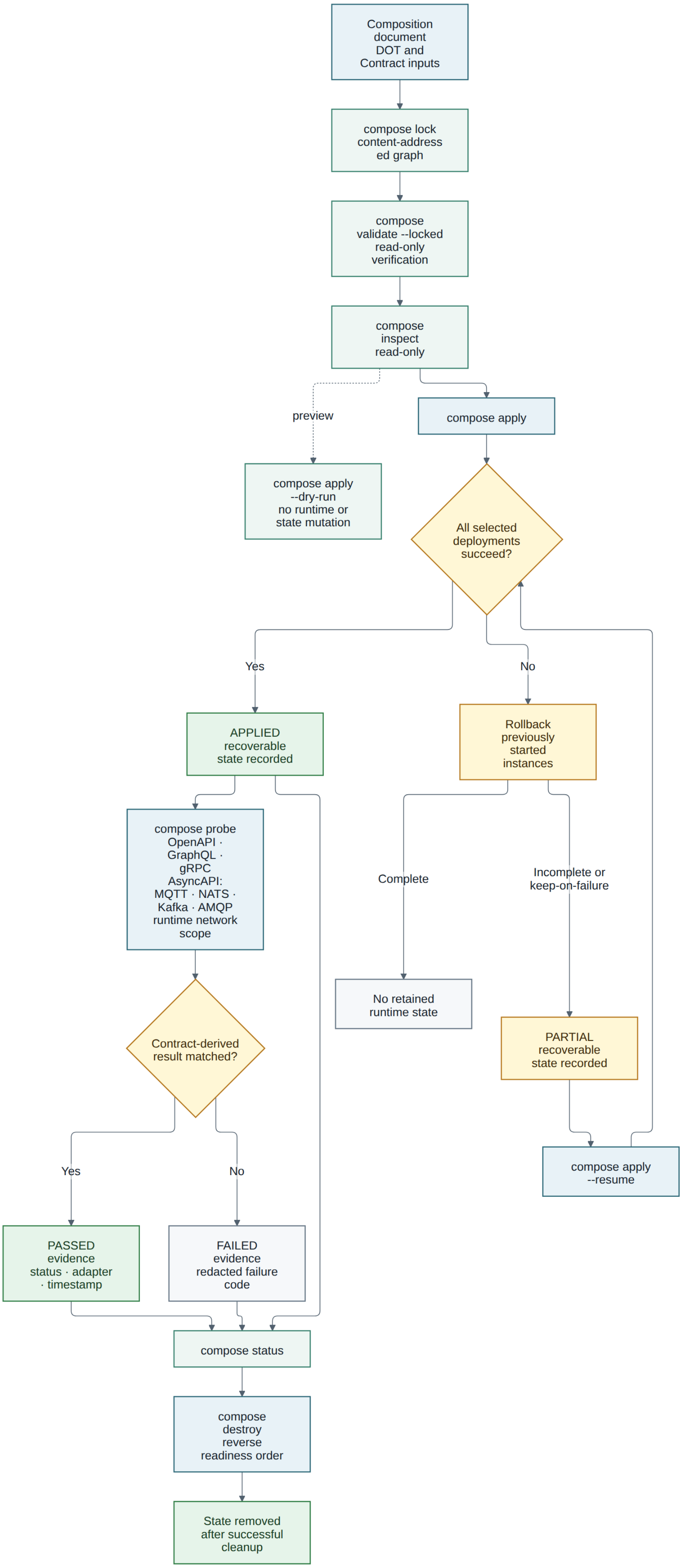

Composition document DOT and Contract inputs
compose lock content-addressed graph
compose validate --locked read-only verification
compose inspect read-only
preview
compose apply --dry-run no runtime or state mutation
compose apply
All selected deployments succeed?
Yes
No
APPLIED recoverable state recorded
Rollback previously started instances
compose probe OpenAPI · GraphQL · gRPC AsyncAPI: MQTT · NATS · Kafka · AMQP runtime network scope
Complete
Incomplete or keep-on-failure
No retained runtime state
PARTIAL recoverable state recorded
Contract-derived result matched?
Yes
No
compose apply --resume
PASSED evidence status · adapter · timestamp
FAILED evidence redacted failure code
compose status
compose destroy reverse readiness order
State removed after successful cleanup

Figure 8. Composition lifecycle

What each command proves

Command	Proves	Does not prove	Mutates runtime/state
`compose lock`	The exact composition, selected DOT definitions and deployments, Contracts, configuration inputs, and portable resolved composition are content-addressed.	Runtime availability, credentials, or policy acceptance.	Writes only the requested lock file; `--check` is read-only
`compose validate`	The strict document shape, references, selected deployments, Anchors, Contracts, compatibility evidence, and materializer declarations resolve.	The exact runtime actions or live availability.	No
`compose inspect`	Everything validated above plus the deterministic instance order, runtime identities, endpoints, Contract standard/format and participant interaction, derived values, security status, and materializers.	That the resolved hosts are reachable or credentials work.	No
`compose apply --dry-run`	The composition reaches lifecycle preparation and can render supported deployment actions without persisting composition state.	Successful deployment or application-level traffic.	No
`compose apply`	The supported deployment adapters completed and recoverable state was recorded, subject to rollback behavior.	End-to-end Contract behavior unless separately probed.	Yes
`compose upgrade --dry-run`	The exact current and target locks resolve, the target SemVer increases, and a deterministic per-instance `ADD`/`UPDATE`/`REMOVE`/`UNCHANGED` diff passes runtime, storage, ownership, and recovery preflights.	That an application-specific persistent-data migration is safe.	No

Command	Proves	Does not prove	Mutates runtime/state
`compose upgrade`	The target revision was applied through a checkpointed transaction and the last-known-good control-plane revision remained available until promotion.	Reversal of application data writes; stateful mutations are rejected.	Yes; failures automatically restore the verified previous revision, while interrupted transactions require `--resume` or `--rollback`
`compose probe`	A selected safe OpenAPI operation, GraphQL lifecycle, finite side-effect-free gRPC method, or correlated AsyncAPI MQTT/NATS/Kafka/AMQP message is observable from the composition's shared Docker network or Kubernetes namespace. GraphQL mutations/subscriptions include mandatory verification or trigger and cleanup.	Complete application semantics, unsafe or unbounded gRPC, other AsyncAPI protocols, multi-scheme authentication, authorization-policy quality, or unprobed operations.	Creates and removes an ephemeral probe runner without changing member workloads; writes redacted probe evidence to composition state
`compose status`	Native status for recorded instances, wire/exposure configuration evidence, and the latest recorded active-probe outcomes.	Current reachability after the probe timestamp, authentication mechanisms not explicitly identified by the evidence, authorization semantics, response bodies, or policy compliance.	No runtime mutation; reads state while holding the lifecycle lock
`compose destroy`	Owned instances were destroyed in reverse order and successful cleanup removed composition state.	Cleanup of resources not owned or recorded by this composition.	Yes

Current-source upgrade admission inspects the selected deployment in both the previous and target revisions, not unrelated deployment alternatives. Compose YAML/JSON is parsed structurally; Kubernetes persistent volumes, claims, StatefulSets and non-ephemeral volume sources are refused. Missing, ambiguous or malformed inputs fail closed. Changed Helm and Vagrant instances are refused until chart defaults and dynamic Ruby storage can be established safely; unchanged instances and normal apply/status/destroy remain supported. This hardening is newer than the published CLI `v0.2.5`: use the reviewed fixed source for upgrades until a later release includes it. No control-plane rollback promises to undo application writes or preserve data in writable container layers.

`validate` answers *is this composition internally resolvable?* `inspect` answers *what exactly would be connected and materialized?* Keeping both commands read-only makes them suitable for review and CI.

`lock` makes those answers reproducible. Its JSON output preserves the original `repository://` and relative references while recording SHA-256 digests for the composition, every resolved `dot.json`, every supplied Contract, configuration inputs, and the portable resolved composition. For an explicit remote repository it also records the normalized repository identity and exact commit. It never records secret values, runtime addresses, or checkout-local absolute paths. Validate, inspect, and apply verify an adjacent lock automatically; `--locked` additionally requires it, and `--lock <path>` selects an explicit file. Any mismatch fails before runtime mutation.

`probe` deliberately reuses the interaction selected by the wire instead of introducing health-check fields in an Anchor or DOT. For OpenAPI, it derives the request from the authoritative operation, accepts only `GET`, `HEAD`, and `OPTIONS`, and requires one fully bound path. The OAuth 2.0 adapter reuses one configured RFC 6749 client-credentials binding that satisfies a complete, single-scheme OpenAPI Security Requirement alternative. It resolves the provider-derived token URL, client ID, scopes, and client-secret reference at execution time, then requires a Contract-declared `2xx` response from the protected operation. Kubernetes Secret values remain `secretKeyRef` references; Docker secrets are supplied only through the runner environment.

Development builds newer than the published CLI `v0.2.5` also accept an optional native JSON response assertion on safe OpenAPI `GET` or `OPTIONS` wires:

```
probe:
  openapi:
    expect: {pointer: /data/status, equals: success}
```

The existing Contract remains authoritative for the operation and declared status codes. With this fixture, the response must additionally be `2xx`, have `application/json` or `application/*+json` content type, and contain valid JSON whose RFC 6901 pointer equals the non-sensitive expected value. Array indices are supported; an empty pointer compares the entire document. Redirects are not followed, execution uses the probe timeout, and runner output is limited to 1 MiB. Only redacted outcomes and failure codes are recorded, never response bodies or actual values. This is explicit value checking, not complete OpenAPI response-schema validation. Without the fixture, existing status-only behavior is unchanged. The assertion is a lock-covered composition input, not a new DOT or Anchor property. `HEAD` cannot carry a JSON body assertion.

For GraphQL, the composition supplies the deployment-specific HTTP path and a non-sensitive operation document with optional operation name and variables. Before resolution succeeds, `dotx` parses the authoritative SDL, validates every document and variable set, requires exactly one direct top-level field, and requires that field to equal the Anchor-selected coordinate. It never guesses `/graphql`. Queries use HTTP directly. Mutations additionally declare an observable verification query and cleanup mutation. Subscriptions use the GraphQL-over-SSE protocol on a connection established before a separate trigger mutation and finish with a cleanup mutation. Every phase must return HTTP `2xx` without GraphQL errors; the expectation is evaluated against the query response, mutation verification result, or delivered subscription event.

For gRPC, the composition supplies one protobuf-JSON `request` for unary and server-streaming methods, or an ordered `requests` sequence for client and bidirectional streams. The selected Proto3 RPC must declare the standard `NO_SIDE_EFFECTS` idempotency level and be discoverable through explicitly enabled server reflection. Every streaming fixture declares a positive exact `responseCount`; an optional expectation uses `ALL`, `ANY`, `FIRST`, or `LAST` matching. Requests are checked against the authoritative input descriptor before the pinned `grpcurl` runner starts, then every returned document is checked against the authoritative output descriptor. Unsafe, unbounded, wrong- count, or descriptor-incompatible calls fail closed.

For AsyncAPI, the active adapters require a provider `send` operation, an explicit MQTT/MQTT5, NATS, Kafka, or AMQP channel binding, one concrete destination, and a JSON-Schema message payload. The composition declares the message and RFC 6901 correlation pointer; MQTT additionally declares QoS. The Kafka adapter honors the official binding-level `topic` override and restricts topics to the Kafka-supported ASCII alphanumeric, dot, underscore, and hyphen syntax. The digest-pinned official Mosquitto or NATS client subscribes before publishing a fresh UUID-correlated non-retained message. Kafka creates a unique consumer group, waits for its topic assignment before publishing the exact correlated record through the official Apache Kafka JVM image, and removes the group after the bounded check. Kafka records remain subject to broker/topic retention and are never represented as transient. The AMQP 0-9-1 adapter applies the official 0.3.0 channel and operation bindings, declares the exact direct/topic/fanout exchange, binds an exclusive auto-delete queue before publishing, acknowledges the correlated delivery, and closes the connection to remove the queue. It accepts credentials only as a password secret reference. Queue-bound channels, headers exchanges, AMQPS, AMQP 1.0, a binding/protocol mismatch, or MQTT-only QoS on NATS/Kafka/AMQP fail closed.

All adapters execute digest-pinned runners inside the common Docker network or Kubernetes namespace. Runner output is capped at 1 MiB before parsing. Secrets and bearer tokens never appear in the argument vector or state. Contractless, unresolved, unsupported-protocol, and cross-platform wires are reported as unsupported rather than inferred healthy. Diagnostics, documents, variables, requests, messages, and response bodies are not persisted; state contains only normalized identifiers, adapter, result metadata, failure code, and timestamp.

The live contract-probe fixture runs a real GraphQL.js producer with distinct GraphQL-over-SSE connections, a reflected `grpc-js` server implementing all four RPC shapes, Eclipse Mosquitto, NATS, Apache Kafka, and RabbitMQ. It demonstrates the full supported boundary; it does not turn those test services or interaction fixtures into reusable DOT metadata.

```
# Composition-level probe input; the DOT and Anchor still contain only the
# authoritative Contract reference and selector Query.health.
probe:
  graphql:
    path: /graphql
    operationName: Health
    document: |
      query Health {
        health
      }
    expect: {pointer: /data/health, equals: true}
```

```
# Finite bidirectional gRPC probe. The selected Proto method in the Contract
# declares stream input/output and NO_SIDE_EFFECTS; these fields supply only
# instance-specific test messages and acceptance criteria.
probe:
  grpc:
    plaintext: true
    serverReflection: true
    requests:
      - {component: contract-provider, count: 1}
      - {component: contract-consumer, count: 1}
    responseCount: 2
    responseMatch: LAST
    expect: {pointer: /sequence, equals: 2}
```

```
# NATS transport delivery. The selected AsyncAPI channel owns the subject,
# NATS binding, message schema, and send/receive semantics.
probe:
  asyncapi:
    protocol: NATS
    correlationPointer: /id
    message: {id: replaced-at-runtime, severity: critical}
```

```
# Kafka retained-stream delivery. The selected AsyncAPI Kafka 0.5.0 channel
# binding owns the physical topic override and the message schema.
probe:
  asyncapi:
    protocol: KAFKA
    correlationPointer: /id
    message: {id: replaced-at-runtime, severity: critical}
```

```
# Emit the canonical strict Draft 2020-12 schema
dotx compose schema > composition-v1.schema.json

# Generate once, commit, and verify without rewriting
dotx compose lock scenario.yaml
dotx compose lock scenario.yaml --check

# Structure, references, selectors, compatibility, and materializer declarations
dotx compose validate scenario.yaml --locked

# Deterministic resolved topology; no runtime mutation
dotx compose inspect scenario.yaml --locked
dotx compose inspect scenario.yaml --locked --json

# Resolve the catalogue from one immutable remote Git revision
dotx compose inspect scenario.yaml --locked --json \
  --repository https://github.com/MARE-EU-Project/DOT-Repository.git \
  --git-ref <40-character-commit>

# Render runtime actions without state or runtime mutation
dotx compose apply scenario.yaml --locked --dry-run

# Execute, inspect recorded evidence, and clean up owned resources
dotx compose apply scenario.yaml --locked
dotx compose probe scenario.yaml --locked --wire oidc-provider-to-gateway
dotx compose probe scenario.yaml --locked --wire protected-gateway-to-client
dotx compose status scenario.yaml
dotx compose destroy scenario.yaml

# Review and execute a locked upgrade
dotx compose upgrade scenario-v2.yaml \
  --from scenario-v1.yaml \
  --from-lock scenario-v1.lock.json \
  --locked --dry-run --json
dotx compose upgrade scenario-v2.yaml \
  --from scenario-v1.yaml \
  --from-lock scenario-v1.lock.json \
  --locked
```

Apply records each pending instance before runtime mutation, rolls back earlier instances on failure, and preserves recoverable state when rollback is incomplete or `--keep-on-failure` is selected. `apply --resume` requires an identical composition source, resolved composition, runtime options, inputs, ordering, and previously applied runtime state. Resume also reconciles an interrupted composition-network creation: it reuses the correctly owned network or creates the still-missing network before continuing. Destroy checkpoints every successfully removed instance, so a retry targets only cleanup that did not finish.

Upgrade is a separate lifecycle; `apply` never mutates an already recorded resolved composition. Upgrade requires the exact previous and target locks, the same composition UUID, and a strictly greater target SemVer. Existing instance names must retain their platform and deployment mechanism. Docker image, Docker Compose, Kubernetes manifest, Helm, Vagrantfile, and WASM module instances use their stable managed identities. The private transaction journal stores only paths, identifiers, phases, and digests—not resolved secrets or input payloads. Stateful and host-bound mutations fail before the first runtime action because restoring a deployment specification cannot undo workload data changes.

The deterministic recovery profile injects an interruption immediately before and after network creation, instance deployment, instance destruction, and network removal. It also verifies lock contention, lock release after a panic, partial cleanup, idempotent resume, and fail-closed composition/runtime drift:

```
task testing:compose:lifecycle-recovery
task testing:compose:upgrade
```

The task writes revision-pinned, secret-free JSON evidence under `DOT-Testing/e2e/target/` and is enforced by the Composition lifecycle recovery workflow.

Apply, probe, status, and destroy acquire a private per-composition process lock. A concurrent lifecycle command fails immediately rather than racing local state or runtime mutations.

`compose status` reports:

- the recorded phase and native runtime status of each DOT instance;
- recorded wire compatibility, security configuration, selected endpoint, and materializer count; and
- recorded exposed-Anchor bindings; and
- the latest redacted active Contract-probe result for each probed wire.

Wire and exposure entries remain configuration evidence. Only entries under active Contract-probe evidence demonstrate a runtime request and declared OpenAPI response match, bounded by their timestamp and adapter scope.

Fail-closed rules

Apply rejects:

- `UNKNOWN` wire compatibility;
- missing, duplicate, or mismatched IDs and versions;
- ambiguous deployments, bindings, services, ports, or OpenAPI server paths;
- unresolved OpenAPI paths or AsyncAPI channel templates;
- undeclared materializer targets and conflicting target values;
- readiness dependency cycles;
- unsupported secret-store/runtime combinations;
- ungranted OCI capabilities, host namespaces, volumes, or CDI devices;
- repeated DOT instances on deployment mechanisms that cannot isolate their resource names; and
- materialized cross-platform wires without an explicit network binding.

Resolution may retain `UNKNOWN` to explain incomplete evidence. Execution never silently upgrades `UNKNOWN` to compatible.

What does not belong in an Anchor

Do not add these as optional Anchor fields:

- hosts, URLs, numeric ports, broker addresses, or Kubernetes names;
- credentials, tokens, certificates, or secret values;
- Camel/Kamelet route syntax;

- transformation logic;
- placement, slice, tenant, trust-zone, latency, or availability policy;
- current health or live authorization state;
- host paths or operator device names; or
- orchestration dependencies.

Use Contracts for interaction semantics, service bindings for deployable ports, composition inputs for operator-controlled configuration, wires for topology, materializers or mediator DOTs for explicit adaptation, and policy engines for deployment/security decisions.

Choosing an extension point

Requirement	Preferred representation
New API/message operation	Add or update the authoritative Contract and select it from an Anchor.
Consumer needs a provider address	Wire materializer targeting declared service configuration.
Payload/protocol transformation	Explicit mediator DOT with two Contracts and two Anchors.
Startup/readiness ordering	Composition `dependsOn`.
Secret or certificate	Reference-only deployment input and platform/external secret provider.
Privileged runtime resource	Typed deployment requirement plus exact composition grant.
Broker service	Explicit wire `endpoint` selecting the broker service port.
Network attachment, eBPF/XDP, WASM import/export, or lifecycle event	Separately reviewed polymorphic binding type when a real use case requires it; do not grow `Anchor` with nullable fields.

6G security composition patterns

The same model supports security-function composition without baking one orchestrator's topology language into every DOT:

- **Zero-trust API chain:** identity-provider and policy-decision DOTs expose token, discovery, and authorization Anchors; an API gateway consumes them and exposes a protected application Anchor. Contracts carry OAuth/OpenID Connect and application semantics, while the composition supplies client-specific secret references and issuer URLs.
- **Closed-loop detection and response:** telemetry, inference, CACAO interpretation, and OpenC2 actuation remain separate DOTs. HTTP and MQTT Contracts define the boundaries; the wire selects the broker endpoint and supplies only deployment-specific channel values.
- **Multi-domain federation:** peer DOT instances can have reciprocal wires while `dependsOn` contains only real readiness dependencies. Trust-domain, roaming, and federation policy

belongs in policy inputs or a dedicated policy DOT, not in the Anchor.

- **Near-real-time RAN control:** E2 or management APIs are Contract-backed Anchors. Host networking, accelerators, eBPF/XDP attachments, and radio-device access are explicit deployment requirements and grants, never inferred from a wire.
- **Observability and digital twins:** metrics, logs, events, and integrity APIs are distinct Anchors. A storage or message broker is selected explicitly as a connection endpoint; a shared host path is not treated as a portable integration boundary.

For cross-domain 6G deployments, a successful resolved composition proves model consistency, not trust. Admission policy must still evaluate publisher identity, artifact provenance, attestation, slice/tenant placement, data-sovereignty constraints, and least-privilege grants before execution. Runtime acceptance should then probe the selected Contract operation over the actual network path.

Troubleshooting

Symptom	What to check
Anchor ID is not found	The wire references the pinned DOT version, the exact local ID is stable, and the Anchor is present in that version's `dot.json`.
Contract or selector cannot be resolved	The composition maps the exact Contract ID to a readable source, the source document's `id` matches it, and the selector uses the standard-specific syntax.
Anchor has no deployable endpoint	The selected deployment contains exactly one service binding for the Anchor, and its named port exists on that same service.
Compatibility is `UNKNOWN`	Inspect `compatibilityReasons` in `compose inspect --json`; add authoritative Contract evidence or use an explicit mediator. Do not override the result.
Materializer target is rejected	The target service declares the environment variable or config-file path, the JSON Pointer is valid, and no other wire writes a conflicting value.
Security remains unresolved	Bind a Contract-required supported scheme, declare all participant variables, and supply sensitive values by `secretRef` rather than plaintext.
Apply reports the composition is busy	Another apply, status, or destroy holds the per-composition lock. Let that command finish; do not delete the lock while a lifecycle process is running.
Status shows a configured wire but no active evidence	Run `dotx compose probe scenario.yaml --wire <wire>`. If the wire is skipped, its operation requires a dedicated adapter or application-level acceptance test. Configuration evidence alone is not reachability evidence.

Symptom	What to check
An OAuth probe reports `AUTHENTICATION_ERROR`	Check the bound token URL and client ID, the referenced Secret name/key or environment-store source, required scopes, issuer reachability, and client authorization. Dotx deliberately does not persist the token endpoint's body.

Executable examples

- `DOT-Testing/conformance/anchors/` covers all four Contract standards, contractless Anchors, validation failures, bindings, and compatibility.
- `DOT-Testing/e2e/api-gateway-oidc/` is the canonical authenticated request/response lifecycle example.
- `DOT-Testing/composition/poc6-closed-loop/` demonstrates an explicit MQTT broker endpoint and OpenC2/AsyncAPI operations.
- `DOT-Testing/composition/federation-peers/` demonstrates reciprocal wires without false readiness cycles.
- `DOT-Testing/e2e/poc1-ric-aaa/` demonstrates identity, OPA authorization, forwarding, metrics feedback, and deny-list update across seven wires.
- `DOT-Testing/interoperability/pocs.yaml` is the evidence ledger for all eleven MARE PoCs and records every remaining acceptance gap.

Run the repository gate with:

```
task testing:conformance
```

The source-adapted E2E tests prove only their documented boundaries. They do not claim partner-runtime, live-RIC, production-network, trust-policy, or SFC acceptance unless that exact behavior is asserted by the scenario.

Verification and examples

DOT-Design keeps model correctness, composition resolution, deployment dry-runs, and live behavioral acceptance as separate evidence levels. A passing lower level never implies a higher one.

Evidence model

Level	Demonstrates	Does not demonstrate
Model and schema conformance	`dot.json` shape, Bean Validation constraints, stable references, selectors, service-port bindings, and generated-model compatibility	Contract payload resolution or multi-DOT topology
Contract resolution and compatibility	Selected operations exist and normalized semantic, interaction, protocol, media, schema, and security facts can be evaluated conservatively	Runtime reachability or valid credentials
Composition validation and resolved composition	Pinned DOTs, wires, endpoints, materializers, parameters, grants, and readiness ordering resolve deterministically	Successful deployment
Lifecycle dry-run	The reference executor accepts the supported profile and renders runtime actions without writing composition state	Workload startup or traffic
Source-adapted E2E	Pinned upstream logic or artifacts execute behind explicit test adapters and satisfy the assertions documented by that scenario	Unmodified partner runtime, production infrastructure, or full PoC acceptance
Live production acceptance	The intended workloads, network, identity, policy, and Contract operations work in the target environment	Nothing beyond the probes and acceptance criteria actually executed

Read catalogue readiness correctly

Catalogue validity, interface completeness and live acceptance are separate results. A green repository workflow validates declarations and renders locked compositions; it does not deploy every DOT or certify production readiness.

Result	Interpretation
Base/reusable validation	Checks definition structure, exact references, verified Contract bytes and applicable bindings. It permits explicitly lower-evidence runtime-only entries.

Result	Interpretation
`INTERFACE_PROFILE_PASSED`	Passes `--profile interoperable` for declared Anchors and exposed ports. This is not live producer acceptance or proof of complete API coverage.
`INTERFACE_GAP`	Lacks machine-readable coverage required by that strict profile. It is not automatically a broken deployment or proof that no upstream specification exists.
Live receipt	Records only the tested workloads, revisions, assertions and cleanup outcome. It does not qualify other deployments or the whole catalogue.

Also distinguish **official Contracts**, **source-derived Contracts**, **runtime-only components and brokers**, **planning definitions and stubs**, and **test fixtures, adapters and producer doubles**. Official provenance does not establish runtime behavior; source-derived documents need producer review; a broker's protocol does not define application message semantics. Do not fabricate a Contract for a client-only worker or planning API just to turn a profile result green. An ownership folder is not an evidence level.

From the **DOT-Design checkout**, warm the verified cache and regenerate the report:

```
task testing:contracts:warm
task testing:contracts:readiness
jq '.dots | group_by(.status) | map({status: .[0].status, count: length})' \
  DOT-Testing/e2e/target/catalogue-contract-readiness.json
```

The report records `liveProducerAcceptance: false`, each definition's SHA-256 and its strict-profile diagnostic. Regenerate it after catalogue changes; counts describe that exact snapshot, not a permanent readiness percentage. These reporting Tasks do not belong to the standalone catalogue checkout. There, use `dotx validate .` and inspect the per-bundle README and Contract audit at the selected catalogue revision. For historical evidence, replace `main` in that link with the recorded commit; do not attribute today's audit to an old run.

Primary verification commands

Command	Scope	External requirements
`mvn -pl DOT-Domain test`	Domain value objects, manifests, Anchors, and aggregate validation	Java and Maven only
`go -C DOT-CLI test ./…`	CLI parsing, validation, resolution, deployment adapters, and lifecycle behavior	Go; individual tests use doubles where appropriate

Command	Scope	External requirements
`task testing:dots:repository`	Every `dot.json` discovered recursively below `DOT-Repository/DOTs/`, with the reusable profile for external/internal bundles that publish adjacent Contract evidence, ownership-path isolation, mandatory per-bundle README files, schema validity, CLI cross-references, and identity uniqueness	Go and the checked-out `DOT-CLI` and `DOT-Repository` revisions; repository validation uses its co-versioned `Schema/`, validates the authoritative `Skills/` catalogue, and resolves every DOT Skill ID to one categorized capability
`task testing:dots:external:dry-run`	Reusable Keycloak/Prometheus DOTs plus runtime-only external Loki/Wireshark definitions: profile-appropriate validation, inspection, deployment dry-run, exact-version Helm rendering, expected service/workload shape, and pinned OCI image metadata	`dotx`, Helm, Python/PyYAML, Docker, and network access to the pinned public artifacts
`task testing:quickstart:dry-run`	Execute the command blocks included by Getting started: validate and render Prometheus, then check the locked gateway resolved composition and dry-run	`dotx`, `jq`, initialized catalogue, and network access for official Contracts
`task testing:quickstart:live`	Run Prometheus at the default and overridden port, assert the build-information API/version, and verify container/network/volume cleanup on success and injected failure	Docker Compose, `dotx`, `curl`, `jq`, network access, and free host ports `18909`, `18919`, and `18929`; all example metrics are deleted
`task testing:dots:external:stress`	Repeated Linux deployment, readiness, HTTP/HTTPS access, destroy, and cleanup across two mechanisms per external product: three Helm/Compose pairs plus the Wireshark Docker-image/Compose pair; Keycloak and Prometheus additionally call the exact operations selected from their official Contracts	Docker, Minikube/Kubernetes, Helm, network access, and enough resources for Keycloak, Loki, Prometheus, and Wireshark; emits revision-pinned evidence
`task testing:compose:keycloak-prometheus-openapi-validation`	Reusable-profile validation and a locked three-DOT resolved composition proving the stable Keycloak and Prometheus Anchors resolve against exact official OpenAPI payloads, compose compatibly, bind the Keycloak realm parameter, and materialize the expected consumer operation URLs	`dotx`, `jq`, and the checked-out repository/schema revisions; no live runtime

Command	Scope	External requirements
`task testing:compose:contract-standards`	Two canonical test DOT definitions connected through OpenAPI 3.0/3.1/3.2 and AsyncAPI 3.0/3.1 YAML/JSON, Protocol Buffers (including gRPC service declarations), and GraphQL SDL; asserts selectors, participant interactions, endpoints, probe fixtures, and materializers across twenty-one wires	`dotx`, `jq`, and the checked-out repository/schema revisions
`task testing:compose:contract-probes:live`	A standards-native test producer and four real brokers prove GraphQL query, mutation verification/cleanup, GraphQL-over-SSE delivery, all four finite side-effect-free gRPC shapes through reflection, correlated non-retained AsyncAPI MQTT/NATS delivery, correlation-scoped Kafka delivery under broker retention, and acknowledged AMQP 0-9-1 delivery through an exclusive probe queue; rejects existing resources and emits redacted success evidence only after verified cleanup	Docker, network access for pinned images, and the checked-out `dotx`
`task testing:compose:acceptance`	Authoritative `v1` command/schema parity, every committed lock, twenty-one Contract wires, contractless compatibility, all six runtime dry-runs, service-scoped configuration, recovery/status/destroy, and focused fail-closed controls; emits revision-pinned JSON evidence	A globally installed `dotx` from the checked-out CLI revision, Go, Task, and fixture-specific dry-run CLIs
`task testing:contracts:standards`	Extracts the twelve distinct Contract payloads and validates them with dated official OAI schemas, the official AsyncAPI CLI, `protoc`, and GraphQL.js; each family also has an invalid control which must be rejected	Python with PyYAML/jsonschema, Node.js, network access for pinned validators, and `protoc`
`task testing:compose:runtime-configuration`	The same catalogued DOT receives declared environment and configuration-file inputs through Docker image, Docker Compose, Kubernetes manifest, Helm chart, Vagrantfile, and Wasmtime/WASI dry-run adapters	`dotx`, `jq`, and runtime CLIs in dry-run rendering mode

Command	Scope	External requirements
`task testing:compose:service-workloads`	A two-service DOT receives isolated defaults, overrides, configuration files, and secret references through exact Compose, Kubernetes, and Helm workload bindings	`dotx`, `jq`, and runtime CLIs in dry-run rendering mode
`task testing:compose:service-workloads:live`	Two concurrent copies of the same multi-service workload prove exact environment, secret, config-file, and volume isolation plus readiness, status, resume, selective destroy, surviving-peer health, and cleanup across Docker Compose, Kubernetes, and Helm	Docker Compose, Minikube/Kubernetes, Helm, and network access for runtime images; emits redacted revision-pinned evidence
`task testing:compose:runtime-isolation:live`	Two locked instances of one Vagrant DOT and one WASI CLI DOT prove distinct runtime identities, concurrent state, status, selective destroy, peer survival, and cleanup. Repeated fixed-port WASI HTTP fails before mutation.	Vagrant 2.4.9 with libvirt/KVM and Wasmtime 49.0.2; emits redacted, revision-pinned evidence
`task testing:cli:coverage`	Complete DOT-CLI test suite plus machine-readable total and high-value package statement-coverage profiles	Go; writes reports below `DOT-Testing/e2e/target/`
`cd DOT-CLI && task ci:security`	Scans the Go module dependency graph and reachable code against the current OSV vulnerability database with the exact pinned scanner used by CI	Go and network access; intentionally separate from deterministic `task ci`
`task testing:runtime:kubernetes-helm:live`	Real Kubernetes-manifest and Helm-chart deploy, rollout readiness, status, service-scoped environment materialization, and destroy lifecycle	Docker, Minikube, Kubernetes, Helm, and network access for runtime images
`bash DOT-Testing/runtime/wasm/live.sh`	Real Wasmtime lifecycle for WASI CLI; CI additionally builds the pinned Bytecode Alliance `wasi:http/proxy` sample and proves an HTTP response	`dotx`, Wasmtime, and the optional prebuilt HTTP component path
`task testing:conformance`	Cross-language Anchor and composition gate, including all deterministic PoC fixtures and lifecycle dry-runs	Java, Maven, Go, Task, and fixture-specific local tools
`mvn clean install`	Full Java domain build, schema generation, coverage gates, and SBOMs	Java and Maven only
`task testing:e2e:<scenario>`	Scenario-specific live behavior	Read that scenario’s README first; commonly Docker and network access

Test asset layout

```
DOT-Testing/
 ├─── acceptance/        versioned, implementation-neutral consumer profiles
 ├─── conformance/       deterministic valid and invalid DOT fixtures
 ├─── composition/       read-only validate, inspect, and dry-run scenarios
 ├─── coverage/          executable code-coverage ratchets
 ├─── e2e/               opt-in deployable behavioral scenarios
 ├─── interoperability/  revision-pinned evidence and remaining-gap ledger
 └─── runtime/           isolated native-runtime lifecycle harnesses
```

`testing` is repository-level infrastructure, not a Maven module and not a deployable DOT. Authoritative test-only definitions live under `DOT-Repository/DOTs/testing/` and are selected from compositions with `repository://<dot-id>@<version>`. Intentionally invalid conformance inputs stay under `DOT-Testing/conformance/`; they must never be admitted to the repository catalogue. Generated secrets, logs, reports, coverage profiles, and materialized configuration belong under scenario-local `target/` directories and must not be committed.

The focused `task testing:anchors:acceptance` gate verifies the versioned DOT Anchor consumer profile. It resolves pinned test definitions from `DOT-Repository`, verifies the composition lock, and asserts the complete cross-standard resolved composition without a deployed workload.

The separate `task testing:compose:acceptance` gate freezes the complete `dot.mare6g.eu/v1` production profile. Its machine-readable source is `DOT-Testing/acceptance/composition/v1/profile.json`; the gate binds that source to CLI help, the tracked schema, repository locks, positive evidence, and negative fail-closed behavior. Runtime rows distinguish underlying single-DOT live lifecycle proof from multi-DOT composition lifecycle proof.

The focused `task testing:compose:contract-standards` gate complements that API profile with an executable composition matrix. It proves all six supported standard/format representations between the same provider and consumer DOTs; it does not promote resolution evidence to live protocol or application acceptance.

The separate `task testing:contracts:standards` gate validates those payloads outside the DOT resolvers with standards-native tooling. The positive fixtures must pass and deliberately malformed OpenAPI, AsyncAPI, Protobuf, and GraphQL controls must fail. This catches both invalid examples and accidentally permissive validator invocations.

The runtime-configuration gate separately proves that selecting another executable deployment does not discard a service's declared environment or configuration-file surface. Each adapter is exercised first with its declared `defaultValue` and `defaultContent`, then with explicit composition overrides; the gate validates the cached default document, resolved service id, override values, and runtime-specific dry-run. Runtime-native storage, secret, privilege, and device semantics retain dedicated tests rather than being flattened into a misleading universal volume abstraction.

The multi-service live profile is the higher evidence layer for the exact workload bindings. Compose projects and their named volumes are keyed by the composition runtime identity; Kubernetes repetitions use separate namespaces; Helm repetitions use identity-derived release names in one namespace. The profile verifies that destroying one copy leaves the other healthy and that dotx-created ConfigMaps/Secrets are cleaned without deleting referenced operator-owned Secrets. The tracked Helm chart is packaged from the pinned checkout, and its SHA-256 is included in the redacted evidence artifact.

Coverage claims and gates

Coverage is reported by evidence dimension; the project does not collapse different meanings of “covered” into one percentage:

Dimension	Gate	Interpretation
Repository inventory	`dotx validate DOT-Repository`, `dotx repository audit`, and the DOT repository conformance workflow	Must be 100%: every filesystem definition is discovered, parsed, schema-valid, cross-reference-valid, uniquely identified, and stored in a supported category; no deployable copy may remain outside `DOT-Repository`.
CLI statements	`task testing:cli:coverage`	The whole CLI and every package containing executable statements must retain 100% statement coverage. The constant-only `feature/sources` package has no testable statements. This is a code-execution metric, not proof that a workload or security control works.
Java instructions and branches	`mvn verify` through JaCoCo 0.8.15	The Java domain module must retain 100% instruction and branch coverage. JaCoCo HTML/XML/CSV reports are uploaded with the Java build evidence.
Native runtime lifecycle	WebAssembly and Kubernetes/Helm runtime E2E workflows	Proves real Wasmtime CLI and HTTP behavior plus Minikube manifest/Helm lifecycle. It does not imply that every external DOT workload is accepted.
Composition scenarios	`task testing:e2e:all:dry-run`	Every listed deterministic scenario must validate, inspect, and complete its supported lifecycle dry-run.
PoC interoperability	`DOT-Testing/interoperability/pocs.yaml` plus scenario-specific live workflows	Each of the eleven PoCs has traceable evidence, while remaining adapters, unavailable partner components, and unproved production behavior stay explicit.

The repository validation gate fails on duplicate identities, mis-scoped definitions, invalid DOTs, invalid Skill/category assets, or unresolved DOT Skill references. The workspace audit separately classifies uncatalogued, identical, divergent, and malformed definitions and fails on every deployable `dot.json` outside `DOT-Repository`. `DOT-Testing/conformance/` fixtures are reported but exempt because invalid definitions are their purpose. During the staged migration, run:

```
dotx repository audit --repository DOT-Repository --workspace . --json
```

The coverage gates enforce 100% Java instruction/branch coverage and exact 100% Go statement coverage overall and for every package containing executable statements. Those thresholds do not replace negative tests, lifecycle checks, live probes, or partner acceptance.

Canonical Anchor scenario

`DOT-Testing/e2e/api-gateway-oidc/` is the primary tutorial and lifecycle fixture. Its composition resolves three independently valid, test-scoped definitions from `DOT-Repository`:

- Keycloak provides OpenID Connect discovery and token operations;
- APISIX requires discovery information and provides a protected operation; and
- an OAuth client requires the token and protected operations.

The composition uses three OpenAPI Contracts, three wires, explicit security configuration, Kubernetes Secret references, configuration-file and environment materializers, readiness dependencies, and one exposed Anchor.

```
task testing:compose:keycloak-apisix-oauth
task testing:demo:keycloak-apisix-oauth:dry-run

# Opt-in live execution
task testing:demo:keycloak-apisix-oauth:live
```

Read DOT Anchors and Composition alongside `DOT-Repository/Compositions/keycloak-apisix-oauth/composition.yaml`.

MARE PoC-derived coverage

The interoperability ledger pins the source revision used for every analysis, records the modeled interactions, identifies the executable fixtures, and keeps remaining acceptance gaps explicit. The current repository contains source-adapted evidence for all eleven PoCs; this is not a claim of unmodified partner-runtime acceptance.

PoC	Composition focus	Fixture
1	OIDC identity, OPA authorization, RIC forwarding, metrics feedback, and deny-list update	`DOT-Testing/composition/poc1-ric-aaa/`

PoC	Composition focus	Fixture
2	Repeated Flower trainers over gRPC/Protobuf and exposed BentoML inference	`DOT-Testing/composition/poc2-federated-inference/`
3	Five-stage contractless JSON insider-threat analytics dataflow	`DOT-Testing/composition/poc3-insider-threat-pipeline/`
4	Prometheus metrics, Elasticsearch logs, Grafana, and Kibana configuration	`DOT-Testing/composition/poc4-continuous-monitoring/`
5	Monitoring-to-analysis-to-assessment-to-mitigation security pipeline	`DOT-Testing/composition/poc5-security-pipeline/`
6	OpenAPI plus brokered AsyncAPI/MQTT, CACAO, OpenC2, and NWDAF remediation	`DOT-Testing/composition/poc6-closed-loop/`
7	Least-privilege packet capture, explicit CloudEvents mediation, and trust scoring	`DOT-Testing/composition/poc7-trust-scoring/`
8	cAdvisor/Prometheus observability and physical/twin integrity checks	`DOT-Testing/composition/poc8-ndt-observability/` and `poc8-twin-integrity/`
9	Amarisoft/Open5GS collection, InfluxDB storage, and data export	`DOT-Testing/composition/poc9-edge-ddos-collection/`
10	OIDC-protected capability exposure, SPARQL, OPA policy decisions, and observability	`DOT-Testing/composition/poc10-secure-capability-exposure/`
11	Reciprocal federation peers and local-core adapters with separate readiness edges	`DOT-Testing/composition/federation-peers/`

The authoritative coverage and gap list is `DOT-Testing/interoperability/pocs.yaml`. The official project descriptions are available from the MARE PoC catalogue.

Negative evidence is intentional

Rejected fixtures are part of the design evidence. They demonstrate that the model does not invent connectivity where the pinned source lacks a declared consumer input, does not treat stdout or a host path as a portable connection, and does not silently grant privileged runtime access.

Examples include:

- PoC7 rejects a collector-to-scorer wire when the collector declares no such Anchor or configuration input;
- PoC8 rejects an undeclared Prometheus-to-NDT handoff; and
- invalid conformance fixtures cover missing Contracts, selectors, ports, duplicate bindings, and incompatible Anchor pairs.

Failing closed is a successful test outcome when the available evidence cannot support the proposed composition.

Adding or promoting a scenario

1. Pin the exact external source revision used as evidence.
2. Document what the source actually produces and consumes.
3. Add deployable definitions according to ownership: third-party software under `DOT-Repository/DOTs/external/`, MARE-owned software under `DOT-Repository/DOTs/internal/`, and only synthetic fixtures under `DOT-Repository/DOTs/testing/`. Record their evidence and purpose in the applicable test evidence ledger and select them by exact `repository://<name>@<tool-release>` identity.
4. Define Contracts only for supported, observable interactions.
5. Use contractless Anchors only when no supported Contract standard applies.
6. Declare all service ports, configuration targets, privileges, and devices.
7. Add negative fixtures for tempting but unsupported connections.
8. Start under `DOT-Testing/composition/` with validate and resolved composition assertions.
9. Add a lifecycle dry-run before any live execution.
10. Promote to `DOT-Testing/e2e/` only with isolated resources, explicit prerequisites, reliable cleanup, and externally observable assertions.
11. Record what remains unproved in the interoperability ledger.

Interpreting results

> ❗ A successful resolved composition proves model consistency. A successful apply proves that the supported deployment adapters completed. Only an active scenario probe can prove that traffic, authentication, authorization, policy, or security behavior worked as intended.

External DOT examples

The external-product examples demonstrate how an existing open-source workload can be described as a DOT without modifying that workload. The upstream project remains authoritative for the software, configuration, and interface Contract. The DOT descriptor selects a pinned artifact and declares the portable service surface that `dotx` can validate and deploy.

Upstream documentation is useful source evidence, but it is not acceptance evidence. DOT-Design therefore tests each example at separate levels:

1. direct `dotx validate`, using the reusable publication profile where applicable;
2. Contract-selector and DOT-to-DOT Anchor composition for reusable definitions;
3. offline inspection and deployment rendering;

4. networked rendering of the exact pinned Helm charts;
5. one real Linux lifecycle on Kubernetes/Helm or Docker; and
6. bounded repeated lifecycle stress with cleanup verification.

Canonical external catalogue

The external catalogue currently has one root for each of these 20 products: APISIX, cAdvisor, Elasticsearch, Filebeat, Grafana, GraphDB, InfluxDB, Keycloak, Kibana, Loki, Eclipse Mosquitto, Prometheus node-exporter, OPA, OpenTelemetry Collector, Oxigraph, Prometheus, Wireshark, NATS, Kafka, and RabbitMQ. PoC compositions reuse those identities and supply routes, realms, policies, scrape targets, data sources, credentials, files, and deployment selection as instance inputs. They do not create `pocN-*` copies of the product.

The following four products form the bounded release stress matrix. They are not the complete external catalogue.

Release stress matrix

DOT	Catalogue status	Mechanisms in the same DOT	Contract/Anchor surface	Live assertions
`Keycloak@26.8.1`	Stable reusable external DOT	Codecentric `HELM_CHART 7.4.0`; official Keycloak `DOCKER_COMPOSE`, `DOCKER_IMAGE` and native Kubernetes deployments, all targeting 26.8.1 in one root definition	Exact Keycloak 26.8.1 Admin REST OpenAPI plus an attributed OIDC endpoint projection; Admin realm, JWKS, and client-credentials provider Anchors	Exact StatefulSet binding and Compose `start-dev`; readiness; obtain an admin token and assert `GET /admin/realms/master` returns the master realm through both mechanisms
`Loki@3.7.8`	Runtime sample, not reusable	Grafana Community `HELM_CHART` 18.15.1; official Loki `DOCKER_COMPOSE` image pinned by digest	No fabricated Contract: the pinned release has HTTP documentation and internal multi-file protobuf APIs, but no supported self-contained machine-readable Contract for the exposed public HTTP surface	Exact single-binary StatefulSet binding and `/ready`; Compose `/ready`

DOT	Catalogue status	Mechanisms in the same DOT	Contract/Anchor surface	Live assertions
`Prometheus@3.15.1`	Stable reusable external DOT	Prometheus Community `HELM_CHART` `29.38.2`; official Prometheus `DOCKER_COMPOSE` image pinned by digest; `DOCKER_IMAGE` for composition-owned scrape configuration	Exact Prometheus 3.15.1 OpenAPI 3.1 artifact for build information and PromQL; explicit contractless scrape-target requirement materialized through `prometheus.yml`	Exact server Deployment binding; readiness; assert `/api/v1/status/buildinfo` reports version `3.15.1` through both mechanisms
`Wireshark@4.6.1`	Runtime sample, not reusable	LinuxServer versioned `DOCKER_IMAGE`; equivalent `DOCKER_COMPOSE` image pinned by digest	No fabricated Contract: the image exposes a human web desktop, not a supported machine API	Container state/environment/volume/ports plus HTTPS for both mechanisms

Each product is kept under `DOTs/external/` because it is owned outside MARE, with every supported mechanism for its selected actual tool release in the same root `dot.json`. Older releases remain in Git history, not nested bundles. Ownership does not imply interoperability maturity. Keycloak and Prometheus publish adjacent official Contract evidence and pass the strict reusable profile. Loki and Wireshark currently provide runtime evidence only; they must not be misrepresented as Contract-backed interface interoperability. Chart versions, Compose image indexes, and the direct Wireshark image tag are deliberately pinned. `latest` is convenient for exploration but unsuitable for release evidence because a repeated run could execute different bytes. The Wireshark OCI index currently contains Linux amd64 and arm64 manifests. Stable `0.2.0` retained a historical cross-built arm64 preview, but subsequent DOT-Design release packaging and automated acceptance intentionally target Linux amd64 only.

Authoritative upstream references:

- Keycloak 26.8.1 release assets, Keycloak Admin REST API, Codecentric Keycloak chart, and Keycloak containers
- Grafana Loki Helm documentation
- Prometheus 3.15.1 official OpenAPI 3.1 artifact, Prometheus Community chart, and Prometheus installation
- LinuxServer Wireshark image

Run the examples

Run commands from the `DOT-Design` root with a `dotx` built from the checked-out `DOT-CLI` revision.

```
# Fully offline and non-mutating
```

```
task testing:dots:external:offline

# Downloads and renders the pinned charts, but does not deploy them
task testing:dots:external:dry-run

# Deploys each example once and cleans it up
task testing:dots:external:live

# Repeats the complete lifecycle three times by default
task testing:dots:external:stress
```

The live profiles require Linux, Docker, Kubernetes, Helm, `kubectl`, `curl`, `jq`, and network access to the pinned public artifacts. `dotx tool check` starts Minikube when the current environment has no selected cluster. Set `DOTX_E2E_SKIP_HELM=1` or `DOTX_E2E_SKIP_DOCKER=1` only for focused local diagnosis; a release acceptance run must not skip either runtime.

What the stress profile adds

`DOTX_E2E_STRESS_REPEATS` accepts a bounded value from 1 through 5. Every cycle uses a new namespace, Docker runtime name, Compose project names, marker, and host-port range. It executes validation again before runtime mutation, selects each mechanism explicitly, verifies workload readiness and HTTP/HTTPS reachability, and destroys all resources. The runner then fails if the cycle namespace, Wireshark container, or any of the four Compose projects still exists. If a Kubernetes deployment or readiness check fails, the runner records pod, workload, event, and bounded container-log diagnostics before cleanup and skips only the assertions that are causally blocked by that failed workload. The root failure still fails the cycle; dependent timeout noise is not counted as independent evidence.

The resulting matrix contains eight cells:

- Keycloak: `HELM_CHART` and `DOCKER_COMPOSE`;
- Loki: `HELM_CHART` and `DOCKER_COMPOSE`;
- Prometheus: `HELM_CHART` and `DOCKER_COMPOSE`; and
- Wireshark: `DOCKER_IMAGE` and `DOCKER_COMPOSE`.

This is a runtime-mechanism matrix, not a reusable-interface count. Only the Keycloak and Prometheus cells carry official machine-readable Contracts and Contract-backed Anchors. A separate locked three-DOT composition wires those two providers to a headless consumer, resolves the exact official selectors, binds the Keycloak `{realm}` parameter, and asserts the resulting operation paths and environment materializers:

```
task testing:compose:keycloak-prometheus-openapi-validation
```

```
DOTX_E2E_STRESS_REPEATS=5 task testing:dots:external:stress
```

Evidence is written to `DOT-Testing/e2e/target/external-dots/`:

- one log per cycle;
- `summary.json`, including the DOT-CLI and DOT-Repository revisions; and
- `SHA256SUMS` covering the summary and logs.

The dedicated `External DOT examples` GitHub Actions workflow runs the non-mutating profile for relevant changes. Its weekly and manually dispatched Linux job runs the stress profile and uploads the evidence bundle. Push, schedule, and manual runs use distinct concurrency lanes, so requesting a live stress run cannot cancel the required push conformance result for the same revision.

Recorded `0.2.0-rc.3` evidence

External DOT examples run 38708940363 completed two independent Linux lifecycle cycles against DOT-CLI `f8cf796afd985cc0514d42fb149ef67b6a1b7b53` and DOT-Repository `76585f6ec68a892f6cc6d48c9b2acdce6b08d48d`. Each cycle passed all 47 checks and cleaned up its resources. The complete eight-cell mechanism matrix was:

- Keycloak: Helm chart and Docker Compose;
- Loki: Helm chart and Docker Compose;
- Prometheus: Helm chart and Docker Compose; and
- Wireshark: Docker image and Docker Compose.

For both Keycloak mechanisms, the test invoked the exact Admin REST operation selected from the pinned official Contract. For both Prometheus mechanisms, it invoked the exact build-information operation selected from the pinned official Contract. The uploaded evidence contains `cycle-1.log`, `cycle-2.log`, `summary.json`, and `SHA256SUMS`; local checksum verification of the downloaded artifact passed for both cycle logs and `summary.json`. This evidence supersedes the earlier `rc.2` run because it exercises Keycloak-DOT `0.1.1` with the corrected Helm resource profile and pins the exact candidate CLI and repository revisions.

Interpreting a pass

A pass means the pinned public artifacts satisfied the assertions executed in the disposable test environment. For Keycloak and Prometheus this includes a successful request to the exact operation selected by the DOT Anchor, in addition to readiness. It does not establish production sizing, availability, security hardening, backup/restore, or every upstream feature. In particular, the Wireshark example checks its web application surface; it does not grant packet-capture capabilities, host networking, or a CDI device. Those requirements must be declared and explicitly authorized in a separate acceptance profile.

When adding another external DOT, place it under `DOTs/external/` because of its ownership and provide an immutable runtime artifact. Use a vendor/project- official Contract whenever available. Otherwise either pin a runtime-extracted Contract, maintain a clearly identified derived Contract with live conformance, or expose an adapter-owned Contract. Every Contract asset declares its authority, source revision, license, and SHA-256. A runtime-only external DOT remains valid without Contract declarations but makes no interoperability claim. Owned declarations belong in `dot.json` as raw HTTPS URLs plus SHA-256; no local Contract directory is required. Add production claims only after a test explicitly proves them.

Integration handoff

This handoff defines the interoperability boundary for a team implementing a catalogue, composer, Security Function Composer adapter, or other DOT consumer. It intentionally does not prescribe a service, database, API style, language, or framework.

Authoritative inputs

The reference source of reusable definitions is `DOT-Repository`. Resolve a DOT by its exact textual ID and pinned version, then verify the returned `dot.json` matches both. Resolve referenced Contract payloads without copying their protocol, schema, media, or security facts into an Anchor.

Use these commands as executable reference behavior:

```
dotx repository inspect <dot-id>@<version> --repository DOT-Repository --json
dotx validate DOT-Repository
dotx compose validate composition.yaml
dotx compose inspect composition.yaml --json
```

An implementation may expose equivalent behavior through REST, gRPC, events, an SDK, or an embedded library. Its external contract should be versioned and tested independently from the DOT model.

Disposition of legacy SFC requests

The original SFC issues 6 through 9 predate the stable repository-backed model. Their consumer needs are handled at the integration boundary without creating parallel sources of truth:

- **Parameter forms:** derive an SFC-facing form from exact service-scoped environment and configuration declarations plus authoritative Contract schemas. Composition validation rejects missing or ambiguous required inputs before runtime mutation. A duplicate root `params` model is not defined.
- **Skill search:** resolve stable, case-sensitive textual `Skill` IDs through the authoritative `DOT-Repository/Skills/` catalogue. Every referenced capability has one category there; `dotx repository list --skill <id-or-name>` and `--category <id-or-name>` expose the tested consumer projection without copying names or categories into every DOT as a second controlled vocabulary.
- **Catalogue API:** `DOT-Repository` and `dotx repository` define reference behavior. A catalogue team may expose equivalent queries through any versioned API; DOT-Design does not own or prescribe that service.
- **Low-barrier authoring:** a DOT author may hand-write one `dot.json`, place its folder under the appropriate recursive `DOT-Repository/DOTs/` category, and validate the whole filesystem catalogue with `dotx validate DOT-Repository`. There is no duplicate inventory file or wrapper script. This path requires no Maven, Jsonnet, MongoDB, or running service. `dot.json` remains the sole deployment definition; a parallel YAML format and inline copies of referenced entities

are intentionally not introduced.

These dispositions settle DOT-side model ownership. They do not replace joint SFC acceptance or prove a partner catalogue/search implementation.

Required semantics

An interoperable consumer must:

- address DOTs by exact textual ID and pinned version; preserve exact DOT-local Anchor IDs and Composition ownership UUIDs;
- resolve Skills and Contracts by their exact, case-sensitive textual IDs; legacy UUID-shaped IDs remain valid, but names and case-folded aliases do not replace identity;
- require at least one meaningful Skill reference per DOT and consume the generated nonempty-list constraint rather than a separate handwritten rule;
- resolve contract-backed selectors against the authoritative OpenAPI, AsyncAPI, Protocol Buffers, or GraphQL document;
- use `anchorBindings` to map a logical Anchor to a named service port;
- keep instance topology, endpoints, parameter values, credentials, and policy outside reusable DOT definitions;
- evaluate compatibility conservatively and preserve `UNKNOWN` as distinct from `COMPATIBLE`; and
- keep secret values out of DOTs, resolved compositions, persisted state, and logs.

Contractless Anchors are valid only when a supported Contract standard cannot describe the interaction. Their explicit direction, protocols, media types, and semantic types must be complementary before a wire is accepted.

Acceptance evidence

The versioned consumer profile is under `DOT-Testing/acceptance/dot-anchors/v1/`. It resolves two catalogued test DOTs, verifies the committed composition lock, and resolves 21 compatible wires across all supported Contract standards and versions.

```
task testing:anchors:acceptance
task testing:conformance
```

Passing the deterministic profile proves discovery and resolution semantics. A joint live test must separately prove endpoint reachability, credential and token handling, authorization, policy enforcement, and application behavior.

Future implementation freedom

The removed catalogue-service prototype is not part of the production architecture. A future team may replace it or implement discovery elsewhere in a different language or framework.

Compatibility is judged by the stable DOT model and executable acceptance profile, not by reproducing the prototype’s internal classes or former HTTP response shapes.

Release notes

0.3.0

The aligned Linux-amd64 release promotes `0.3.0-rc.1` without changing portable Java models, generated schemas or Go bindings, catalogue definitions or locks. The model archive retains all 77 schemas and the reviewed compatibility corpora; its release label advances without rewriting historical corpus identities. See Compatibility and migration and Catalogue readiness.

Use the selected CLI release and the exact catalogue revision in its manifest. Offline validation requires its warm verified Contract cache; `compose new` creates a template-based draft, not an automatic graph or state migration. Source, rendered-command and evidence-boundary regressions protect the guidance. Final qualification still requires the local release gates, a fresh exact-source demo and successful mandatory remote checks; downloaded-binary acceptance is separate from publication. The supported single-host profile is not certification of every MARE producer or a final 1.0 API guarantee.

Immutable stable documentation is cut at `0.3.0`. The development branch resumes the Next descriptor afterwards; historical tags and documentation are preserved.

0.3.0-rc.1 candidate

The candidate model surface is reviewed in `DOT-Domain/MODEL-SPEC.md`. A reproducible, checksummed model-baseline archive includes all 77 public schemas, shared compatibility corpora and hash inventory. Java, generated schemas and Go remain one authority chain. This is not a final stable release or certification of all producers; publication requires the separate release/runtime gates.

`dotx compose topology <composition-or-directory> --write` now maintains the README Mermaid block, wire legend and adjacent `topology.mmd` natively in Go. `--check` rejects drift without writing. Both require verified locks and preflight all graphs/targets; Task wrappers delegate to the command instead of the removed `Compositions/topology.sh` generator. These remain resolved composition artifacts, not live acceptance evidence.

Catalogue DOT IDs now follow `<name>@<actual-tool-release>` and all 50 distinct Contract IDs follow `<origin-DOT.id>::<standard>::<name>`. For example, `Prometheus@3.15.1` declares `Prometheus@3.15.1::OpenAPI::HTTP`. DOT and composition DOT-reference types are now textual, generated from DOT-Domain. Legacy UUID strings remain valid inputs, but migrated entries do not provide aliases. Catalogue Anchors now use DOT-local kebab-case keys; Composition ownership UUIDs are unchanged. Every DOT now requires at least one Skill reference; native authoring accepts repeated `--skill <id>` flags and prompts interactively. See the catalogue's `IDENTIFIER-MIGRATION.md` for exact mappings and state safety. Destroy old instances using their original checkout and CLI before migrating; regenerate and review locks, then rehearse fresh deployments. Do not reuse old runtime state or treat previous rehearsal receipts as evidence for the new graph. These development-format migrations are included in the 0.3 line, not silently backported to historical 0.2 releases.

The catalogue has 67 canonical product/component definitions, each with one root `dot.json` containing all supported deployment choices. Nested `versions/` bundles and duplicate product definitions are rejected by catalogue CI. Tool upgrades update all mechanisms, release-matched Contracts, explicit references and reviewed locks together; Git retains previous releases. APISIX 3.19.0, Keycloak 26.8.1, Prometheus 3.15.1, OPA 1.21.4 and Elasticsearch 9.5.5 use stable upstream artifacts verified on 2026-10-26. Unreleased internal/validation-only/demo components remain labelled as such. Official Contract documents remain byte-for-byte upstream artifacts; derived configured-route/policy projections remain explicitly derived. Static validation does not establish full producer certification.

The catalogue now uses explicit readable IDs for its five Skill categories and eight capabilities. DOT references contain only `id`; labels and descriptions resolve from `Skills/`. Existing references and affected composition locks have been migrated together. UUID-shaped Skill IDs remain supported in independently authored or historical catalogues, but old UUIDs are not implicit aliases in the current catalogue. That earlier Skill-only migration did not change the other identities; the subsequent DOT/Contract migration is described above.

Automated native Prometheus and quickstart acceptance now coordinate unused TCP ports with per-user leases, check IPv4/IPv6 listeners, and reject occupied explicit overrides without stopping existing services. Leases are released after cleanup, including controlled failures. The manually followed Prometheus walkthrough retains its documented port defaults.

Contract `standard` now accepts only `OpenAPI`, `AsyncAPI`, `Protobuf`, and `GraphQL`, generated from the Java domain enum’s Jackson values. This is an intentional input-format change: uppercase values and case/whitespace aliases are rejected. Migrate owned Contract declarations and explicit origin references, then explicitly regenerate and review affected composition locks with `dotx compose lock <composition.yaml>`. Existing locks fail closed; do not reuse saved runtime state against a changed graph. The `format` values remain `YAML`, `JSON`, `PROTOBUF`, and `GRAPHQL`. Use the selected 0.3 release or matching source; historical 0.2 binaries retain the earlier vocabulary. Stable-tag docs remain unchanged.

`dotx compose inspect` is the read-only command for resolving and explaining a composition. Derived execution details are called the **resolved composition**, not a separate authored model. Deployment commands are previewed with `dotx compose apply --dry-run`. Locks and saved state now use `resolutionDigest`; locks must be regenerated and reviewed. Clean up running compositions with the original binary before upgrading: saved state is not migrated, and retired commands have no aliases. Use the selected release or its matching source checkout.

The official ecosystem presenter route now connects reuse across Docker and Kubernetes, secure multi-DOT policy enforcement and native stateless recovery. Shared preparation/preflight cover five lanes; the full clean-source rehearsal runs seven live paths twice and links cleanup-verified child evidence. The existing runtime workflow offers an explicit long-running full-demo dispatch.

Development CLI builds add `compose topology --status`: a locked composition graph with scoped, bounded runtime observations and a distinct historical probe table. Matching saved execution inputs are required before runtime queries. Presenter pauses use this view without treating running workloads or a recent probe receipt as current API acceptance. Mermaid export remains declaration-only; there is no portable-model or schema change, and no new release is implied.

Contracts now resolve only from their declaring DOT bundle or a complete digest-pinned URL declaration. The repository-level Contract catalogue and validator fallback have been removed; all remaining assets live in their owning bundles. Repository validation still rejects conflicting definitions of the same Contract ID. The shared local/GitHub repository gate additionally validates every bundle in isolation and offline.

Getting started now begins with a checksummed Linux-amd64 CLI installation and a complete Prometheus Docker Compose lifecycle, with API/version verification, cleanup, and troubleshooting. Contributor toolchains and shared CI gates have their own guide. Antora includes the same command blocks executed by `testing:quickstart:dry-run` and `testing:quickstart:live`; the external-DOT workflow rehearses both, including port overrides and cleanup after failure. Development pages now export their own PDF alongside the latest stable PDF, matching the existing download link in the site header.

The Antora site now includes a guided demo for presentations and onboarding, with a tested five-minute read-only path, expected evidence, an optional live Contract-probe path, and explicit support boundaries.

`dotx compose apply --dry-run` no longer creates or changes the composition state/lock directory. Lifecycle rendering therefore works with a read-only cache while retaining the same fail-closed resolution and runtime checks.

0.2.4

`0.2.4` is a backward-compatible Linux-amd64 hardening release for the stable `dot.mare6g.eu/v1` single-host composition profile. It preserves the DOT, Contract, Anchor, composition-schema, repository-layout, deployment, and CLI interfaces published in `0.2.3`. The exact pinned revisions are DOT-CLI `e88daa83221ae5d49a1317b7412045ab2b78315f` and DOT-Repository `247606d4a55c5e85a822f1f82eca52a3574b5a04`.

DOT-CLI now exposes a local/GitHub `task ci:security` gate using the pinned stable OSV-Scanner release. It checks the module dependency graph and performs Go call analysis against the current OSV database. The network-backed scan is kept separate from deterministic `task ci` because vulnerability data changes independently of a source revision.

Standalone DOT-CLI packaging now emits a deterministic CycloneDX 1.6 SBOM for the exact Linux-amd64, CGO-disabled build profile. Release tests validate the SBOM identity, platform properties, dependency content, omitted timestamp and serial number, checksum coverage, and byte-for-byte equality across two package builds. Downloaded-binary acceptance retains the published SBOM as evidence.

Readable workflow badge publication now refreshes the complete status set on a coalesced 15-minute schedule, with manual dispatch available for immediate refresh. It no longer consumes one self-hosted-runner job for every completed workflow in the already large validation matrix.

Wasmtime lifecycle status and inventory checks now tolerate the short, bounded interval in which Linux procfs may expose an empty process command line during `exec`. A non-empty command line that does not match the recorded Wasmtime module identity is still rejected immediately, preventing lifecycle operations from acting on a reused PID.

Antora publication now deduplicates an exact stable tag from its identical release-commit worktree, closing the tag-push race while preserving immutable stable documentation. Current-profile documentation is guarded against the machine-readable acceptance profile: twenty-one wires, all four finite side-effect-free gRPC shapes, and MQTT, NATS, Kafka, and AMQP 0-9-1 delivery.

The complete pushed workflow matrix passed on `77d8f099e5f42430f1d30e8d01c27b2c534b8d49`. It includes 100% DOT-Domain instruction and branch coverage, 100% DOT-CLI aggregate and per-executable- package statement coverage, repository conformance, all twenty-one cross-standard wires, real-producer and real-broker active probes, Docker, Kubernetes, Helm, Vagrant, and WebAssembly lifecycle evidence, and versioned Antora publication. DOT-CLI `v0.2.4` additionally passed two-pass reproducible packaging and downloaded-binary acceptance.

The release publishes the versioned DOT-Domain JAR, DOT and Composition v1 JSON Schemas, JSON and XML CycloneDX SBOMs, the Linux-amd64 `dotx` binary, an exact source manifest, and checksums. Linux amd64 remains the only supported release target. DOT-Registry REST API work, native arm64, non-Linux targets, multi-host orchestration, and joint Security Function Composer or partner acceptance remain outside this release boundary.

0.2.3

`0.2.3` is a backward-compatible production-hardening release for the stable `dot.mare6g.eu/v1` single-host composition profile. It preserves the DOT, Contract, Anchor, composition-schema, repository-layout, and deployment interfaces published in `0.2.2` while broadening standards-authorized runtime evidence and strengthening deployment-configuration acceptance. The exact pinned revisions are DOT-CLI `b07ec3b568b59cb25631a9b51700a14ee124bb05` and DOT-Repository `247606d4a55c5e85a822f1f82eca52a3574b5a04`.

The cross-runtime configuration matrix now exercises every executable adapter through separate default-only and explicit-override paths. It verifies non-sensitive `defaultValue` and `defaultContent` materialization, explicit environment/file precedence, resolved service targeting, and redacted dry-run diagnostics.

Contract-authorized gRPC probes now cover unary, server-streaming, client-streaming, and bidirectional methods. All methods still require the Proto3 `NO_SIDE_EFFECTS` option and server reflection. Streaming fixtures must declare an exact finite response count; input and output messages are validated against the authoritative descriptors; and expectations explicitly select `ALL`, `ANY`, `FIRST`, or `LAST` response matching.

AsyncAPI delivery now supports NATS core messaging beside MQTT. NATS fixtures require the official channel binding and a concrete non-wildcard subject, then prove UUID-correlated subscribe-before-publish delivery through digest-pinned official NATS client and server images. The live standards fixture exercises all four gRPC shapes plus MQTT and NATS while retaining redacted state evidence.

AsyncAPI delivery also supports Kafka through the official 0.5.0 channel binding. `dotx` honors a binding-level `topic` override, rejects invalid Kafka topic names and MQTT-only QoS, waits for a correlation-scoped consumer-group assignment before publishing, and verifies the exact correlated record through the digest-pinned official Apache Kafka 4.3.1 JVM image. The ephemeral consumer group is removed; the record itself remains subject to the topic’s broker retention policy. The canonical runtime-only Kafka DOT expands the locked standards composition to twenty wires and three real message brokers.

AsyncAPI delivery now also supports AMQP 0-9-1 through the official 0.3.0 binding. `dotx` validates exact exchange, routing-key, virtual-host, durability, auto-delete, and delivery-mode semantics; binds an exclusive auto-delete probe queue before publish; acknowledges and correlation-matches the delivered JSON; and obtains RabbitMQ credentials only through a secret reference. Queue-bound channels, headers exchanges, AMQPS, and AMQP 1.0 fail closed. The canonical digest-pinned RabbitMQ 4.3.6 runtime DOT expands the locked standards composition to twenty-one wires and four real message brokers.

The complete pushed workflow matrix passed on `79807aacebe7aab1112fe82104898ab4a0929db0`, including DOT-CLI CI, Contract active probes, compatibility matrix, coverage, Kubernetes and Helm runtime E2E, DOT repository conformance, and versioned Antora publication.

The release publishes the versioned domain JAR, DOT and Composition v1 schemas, CycloneDX JSON/XML SBOMs, Linux-amd64 `dotx` binary, immutable release manifest, and checksums. The tag workflow performs deterministic two-build packaging before publication, and post-publication acceptance downloads and exercises the released binary.

Existing `0.2.0` through `0.2.2` definitions, compositions, and locks remain valid. Unsafe or unbounded gRPC, AsyncAPI transports other than MQTT, NATS, Kafka, and AMQP 0-9-1, AMQPS/AMQP 1.0, multi-scheme authentication, multi-host orchestration, automatic cross-platform gateway provisioning, general stateful-data migration, native arm64, non-Linux executables, DOT-Registry REST API work, and joint SFC/partner acceptance remain outside this Linux-amd64 release.

0.2.2

`0.2.2` is a backward-compatible extension of the stable `dot.mare6g.eu/v1` single-host composition profile. It retains the DOT, Contract, Anchor, composition-schema, repository-layout, and deployment interfaces from `0.2.1` while completing the standards-authorized active-probe profile. The exact pinned revisions are DOT-CLI `2dafb492bb657da4d6e6dfe3eab4e9ee4ee229c9` and DOT-Repository `31357e57ab1332e93de6afc25a2b0d67aafed857`.

GraphQL query probes validate documents, variables, and optional exact response expectations against the authoritative SDL. Mutations require verification and cleanup operations; GraphQL-over-SSE subscriptions require a separately triggered event plus cleanup. Unary gRPC probes require server reflection, protobuf-JSON request validation, and the Proto3 `NO_SIDE_EFFECTS` option. AsyncAPI MQTT probes require an explicit binding, concrete topic, JSON-Schema-valid correlated message, QoS, subscribe-before-publish ordering, bounded timeout, and exact non-retained delivery. All fixtures remain composition-owned and persisted evidence remains redacted.

The fifteen-wire deterministic matrix covers OpenAPI 3.0/3.1/3.2, AsyncAPI 3.0/3.1, Proto3, and GraphQL. A real GraphQL.js/gRPC producer and Eclipse Mosquitto broker provide live Docker acceptance. The complete pushed workflow matrix passed on `70b0e94760d60bc0fc37f887db0466796627829a`, including Contract active probes, coverage, compatibility, and Antora publication.

Existing `0.2.0` and `0.2.1` definitions and compositions remain valid; probe fields are optional and additive. Streaming or safety-unclassified gRPC, non-MQTT AsyncAPI transports, multi-scheme probe authentication, native arm64, non-Linux executables, multi-host orchestration, DOT-Registry REST API work, and joint SFC/partner acceptance remain outside this Linux-amd64 release.

0.2.1

`0.2.1` is a backward-compatible hardening release for the stable `dot.mare6g.eu/v1` single-host composition profile. It retains the `0.2.0` DOT, Contract, Anchor, composition-schema, repository, and deployment interfaces. The exact pinned revisions are DOT-CLI `82d7de45800f4e4a435ef08202d4f1e42c16eca8` and DOT-Repository `2fe0ecc81f9caea43c5bf6e0dc824c3e0faf34e3`.

The patch adds overflow-safe SemVer precedence, atomic WASM module-cache replacement and restart-profile verification, fail-closed empty Docker port forwarding, correct Minikube home-directory expansion, and child-process checks for detached Kubernetes forwarding. Repository commands now share one verified private-cache acquisition path. DOT-Domain enforces 100% instruction and branch coverage, while DOT-CLI enforces 100% aggregate and per-executable-package statement coverage. The installer missing-`curl` negative test is hermetic and cannot fall through to host networking or `sudo`.

The complete 13-workflow matrix passed on the pushed hardening baseline `7c71e60393b1758d45e3558a15a2073450cb5cfc`, including coverage, Anchor conformance, and repository conformance. The tag workflow must also pass deterministic two-build packaging and checksum verification. Post-publication acceptance downloads and exercises the released binary rather than trusting only source-tree tests.

The release publishes the versioned domain JAR, DOT and Composition v1 schemas, CycloneDX JSON/XML SBOMs, release manifest, checksums, and `dotx` for Linux amd64. Linux amd64 is the only supported executable target. DOT-Registry REST API work, native arm64, non-Linux targets, and joint Security Function Composer/partner acceptance remain outside this release boundary.

0.2.0

`0.2.0` promotes the fully accepted `0.2.0-rc.8` candidate as the first stable release of the `dot.mare6g.eu/v1` composition profile. There are no behavioral changes from that candidate: the stable tag regenerates the versioned domain artifact, schemas, SBOMs, documentation, and Linux CLI assets from the same pinned DOT-CLI `4f415d43f08bb546d333ab61d6de272e92113903` and DOT-Repository `2fe0ecc81f9caea43c5bf6e0dc824c3e0faf34e3` revisions.

The release pipeline classifies `0.2.0` as stable and publishes it as the latest GitHub release. Before promotion, `0.2.0-rc.8` passed the full pushed commit matrix, reproducible release packaging, and post-publication Linux amd64 acceptance using the downloaded checksummed binary, including two repeated external-DOT lifecycle and cleanup cycles. Arm64 remains a cross-built preview until equivalent native evidence exists, and joint SFC acceptance remains a separate integration activity.

0.2.0-rc.8

`0.2.0-rc.8` supersedes `0.2.0-rc.7` with a current stable build and CI toolchain while retaining the accepted `dot.mare6g.eu/v1` composition, runtime, lifecycle, Contract, Skill-taxonomy, reusable-catalogue, and WASI profiles. Node.js 26 is now the explicit documentation and Contract-validation baseline; Go 1.27.2, Rust 1.98.1, Ruby 4.0, Java 27, and Maven 3.10.0 provide the remaining release toolchains. Direct Maven and Go dependencies and all third-party GitHub Actions were updated to compatible stable releases and immutable Action SHAs, with weekly Dependabot checks for Actions, Maven, Go, and the independently cloneable repositories. Documentation CI now rejects prerelease Ruby runtimes before Antora/PDF assembly.

Release publication now uses a tested, fail-closed SemVer channel classifier: prerelease versions such as `0.2.0-rc.8` are published as GitHub prereleases, while `0.2.0` is published as the latest stable release. The exact pinned revisions are DOT-CLI `4f415d43f08bb546d333ab61d6de272e92113903` and DOT-Repository `2fe0ecc81f9caea43c5bf6e0dc824c3e0faf34e3`.

0.2.0-rc.7

`0.2.0-rc.7` supersedes `0.2.0-rc.6` with two release-evidence hardening changes while retaining the same stable `dot.mare6g.eu/v1` profile and pinned DOT-CLI and DOT-Repository revisions. Source-adapted PoC harnesses now acquire private producer source through one guarded helper: it checks out the full pinned commit in a fresh temporary directory, verifies `HEAD` and the expected GitHub origin, retries bounded transient failures, and rejects partial, untrusted, missing-revision, or unsafe-destination results. Deterministic tests exercise those positive and negative boundaries without network access.

Contract conformance now installs the official Protocol Buffers 36.2 compiler for Linux amd64 or arm64 through the shared local bootstrap, verifies the download with an architecture-specific SHA-256, and uses that exact binary for both positive and deliberately invalid Protobuf fixtures. Local Taskfile runs and GitHub Actions no longer depend on different compiler setup paths. The exact pinned revisions remain DOT-CLI `b01f736393caa1a93d32e2c5593e032e854052ac` and DOT-Repository `d0b7fc1f3aac13a2b785482b07d5529f1d53404f`.

0.2.0-rc.6

`0.2.0-rc.6` supersedes `0.2.0-rc.5` with one shared, fail-closed bootstrap for the official WASI HTTP fixture. Local `task ci:live` and GitHub Actions now fetch and build the same exact Bytecode Alliance `sample-wasi-http-rust` revision before executing the same Wasmtime lifecycle assertions. The stable `dot.mare6g.eu/v1` composition profile, runtime adapters, lifecycle behavior, Contract model, Skill taxonomy, and reusable catalogue structure are otherwise unchanged. The exact pinned revisions remain DOT-CLI `b01f736393caa1a93d32e2c5593e032e854052ac` and DOT-Repository `d0b7fc1f3aac13a2b785482b07d5529f1d53404f`.

The bootstrap verifies the exact upstream URL and commit, rejects non-Git, wrong-origin, and modified tracked source caches, and rebuilds the `.wasm` component rather than trusting an ignored build artifact. The GitHub workflow no longer contains a separate checkout/build implementation; it invokes the same `ci:wasm-runtime` Taskfile gate available locally. Documentation records the Rust

1.98.1, `wasm32-wasip2`, and first-build network prerequisites and the optional `WASI_HTTP_COMPONENT` override for a deliberately supplied component.

0.2.0-rc.5

`0.2.0-rc.5` supersedes `0.2.0-rc.4` with one fail-closed representation for complete Contract payloads. The stable `dot.mare6g.eu/v1` composition profile, runtime adapters, lifecycle behavior, Skill taxonomy, and reusable catalogue structure are unchanged. The exact pinned revisions are DOT-CLI `b01f736393caa1a93d32e2c5593e032e854052ac` and DOT-Repository `d0b7fc1f3aac13a2b785482b07d5529f1d53404f`.

Complete Contract declarations now carry a structured `payload` object:

```
"payload": {
  "url": "https://vendor.example/contracts/service-openapi.yaml",
  "sha256": "0123456789abcdef0123456789abcdef0123456789abcdef0123456789abcdef"
}
```

The URL must be absolute HTTPS without user information or a fragment, and the digest must be exactly 64 lowercase hexadecimal characters. Raw embedded Contract documents and the former flat `payloadUrl`/`payloadSha256` fields now fail schema and `dotx validate` checks. UUID-only Contract references remain a migration-compatible form for verified repository-local wrapper definitions.

`dotx validate` downloads the authoritative bytes, verifies their digest, and uses `MARE_READ_ONLY_TOKEN` or `GITHUB_TOKEN` only for `raw.githubusercontent.com`. Authorization is removed before following a cross-host redirect. Unit and repository tests cover malformed URLs and digests, unavailable assets, digest mismatch, authentication scoping, redirect credential leakage, inline payload rejection, flat-field rejection, and the official external Contract composition. The standalone DOT-Repository gate also runs the pinned validator directly for every catalogue revision.

GitHub Actions and local pre-push verification now share root `ci:*` Taskfile gates. `task ci:deterministic` covers the deterministic non-live release boundary; `task ci:live` covers the mutable Linux runtime workflows and requires the same official WASI HTTP component exercised by Actions, preventing a reduced local test path from being mistaken for workflow parity.

0.2.0-rc.4

`0.2.0-rc.4` retains the runtime and lifecycle profile accepted in `0.2.0-rc.3` and completes the closed SFC-requirement audit with repository-backed Skill discovery. `DOT-Repository/Skills/` now contains an authoritative asset for every capability UUID referenced by the catalogue and links each capability to one of the five controlled MARE categories.

`dotx validate DOT-Repository` resolves all 67 DOT definitions against that catalogue and rejects missing capabilities, category assets used as capabilities, duplicate references, and invalid category links. `dotx repository list --skill <uuid-or-name>` and `--category <uuid-or-name>` provide resolved, machine-readable discovery without copying category fields into every `dot.json`. The exact published revisions were DOT-CLI `3069f907bfd2056b16b92800bb5b98552c28832d` and DOT-

Repository `3117a18da3004c57f4edf3714494fd3954225ae7`.

The audit also reconfirmed the existing DOT-side dispositions for stable Anchor identity and semantics, authoritative payload Contracts, service-scoped pre-deployment configuration validation, and single-file `dot.json` authoring. The separate DOT-Registry REST API remains outside this candidate, and joint SFC/partner acceptance remains a manual gate.

External products now have one canonical reusable DOT root rather than PoC-specific identities. Seventeen product roots plus two shared external test fixtures replace the former duplicated Keycloak, OPA, Prometheus, Elasticsearch, and APISIX definitions. PoC realms, policies, routes, scrape targets, data sources, secrets, and files remain composition-owned inputs; unavoidable source adaptations are named deployment variants under the same product UUID. Repository validation rejects any new deployable definition below `DOTs/external/pocs/`.

Composition lifecycle commands preserve the exact deployment ID rather than reducing it to a manifest type. The `--deployment` selector prefers an exact ID and accepts a manifest type or alias only when it identifies one deployment, so source-adapted and upstream deployments of the same type cannot be selected silently.

Catalogue placement now records ownership rather than Contract maturity: third-party software is under `DOTs/external/`, MARE-owned/developed software is under `DOTs/internal/`, and only synthetic fixtures remain under `DOTs/testing/`. Every DOT bundle includes a local quickstart `README.md`. External/internal bundles opt into strict reusable validation by publishing a `contracts/` evidence directory whose assets identify their authority as upstream-official, runtime-extracted, DOT-ecosystem-derived, or adapter-owned, along with source revision, license, and SHA-256. Loki and Wireshark are now correctly classified as external runtime-only definitions without fabricated vendor Contracts.

0.2.0-rc.3

`0.2.0-rc.3` supersedes `0.2.0-rc.2` after post-publication, exact-pin lifecycle acceptance found that the Keycloak Helm workload could not reliably complete Quarkus augmentation within its declared 1536-MiB container limit. Three independent real-cluster runs retained pod and event diagnostics showing `OOMKilled` exit 137; the runner had 62 GiB of host memory and Minikube exposed 16.8 GB and 20 CPUs, isolating the defect to the DOT resource declaration.

Keycloak-DOT `0.1.1` declares a 2-GiB memory request and 4-GiB limit while retaining the same official Keycloak 26.3.3 Contract, Anchor selectors, chart, image, service, and Docker Compose mechanism. The external-DOT render suite now inspects the real upstream chart output and rejects any mismatch in the Keycloak container's CPU or memory requests and limits before live deployment. The official external-Contract composition and content-addressed lock resolve the versioned `0.1.1` definition.

External DOT examples run 38708940363 passed both the non-mutating chart conformance job and two complete Linux lifecycle cycles. Each cycle passed all 47 checks with zero failures and cleanup verification across Keycloak, Loki, Prometheus, and Wireshark's eight deployment-mechanism cells. The downloaded, checksum-verified evidence pins DOT-CLI `f8cf796afd985cc0514d42fb149ef67b6a1b7b53` and DOT-Repository `76585f6ec68a892f6cc6d48c9b2acdce6b08d48d`.

This candidate also adds a post-publication workflow that downloads the actual checksummed Linux amd64 `dotx` release asset, verifies its release manifest and source revisions, then runs repository validation, Anchor acceptance, Composition v1 acceptance, and two complete external-DOT lifecycle cycles. Maven dependency submission is now an explicit self-hosted workflow; the billing-blocked GitHub-managed automatic submission feature is disabled. Linux amd64 is the release-accepted executable target. The cross-built arm64 asset remains preview-only until equivalent native arm64 evidence exists. Joint SFC/partner acceptance remains a separate manual gate and is not claimed by this candidate.

0.2.0-rc.2

`0.2.0-rc.2` supersedes the first candidate with the fixes and acceptance evidence completed after `0.2.0-rc.1` was tagged. The stable serialized composition API remains `dot.mare6g.eu/v1`.

Changes since rc.1

- Exact service-to-workload selection and configuration isolation for multi-service Docker Compose, Kubernetes-manifest, and Helm deployments.
- Exact official, digest-pinned OpenAPI Contracts and Contract-backed Anchors for reusable Keycloak and Prometheus external DOTs, with every Helm and Docker Compose deployment binding the same interface.
- `dotx validate --profile reusable` now enforces declared HTTPS Contract provenance, payload integrity, selector resolution, Contract use, and per- deployment Anchor binding. Review establishes that the pinned source is the official upstream artifact. `dotx validate <repository-root>` recursively discovers every `dot.json`; the later rc.4 ownership migration derives strict reusable validation from adjacent Contract evidence rather than category path, and is the single DOT-Repository GitHub Actions gate; no duplicate inventory or shell wrapper is required.
- Same-DOT alternative deployment mechanisms for the Keycloak, Loki, Prometheus, and Wireshark external-product examples. Loki and Wireshark were initially retained as runtime-tested samples; rc.4 reclassifies them as external runtime-only DOTs without claiming Contract-backed interoperability.
- A locked Keycloak/Prometheus/consumer composition proves that both stable external Anchors resolve and materialize the exact official operations.
- Failed Kubernetes/Helm workloads now retain pod, event, workload, and bounded log diagnostics before cleanup; causally blocked endpoint assertions are skipped rather than counted as independent failures.
- Repeated hosted-Linux acceptance run 38600327954 covering all eight external-DOT mechanism cells: two complete lifecycle cycles, 46 checks per cycle, zero failures, verified cleanup, and four successful official- Contract operation checks per cycle. The checksummed evidence pins DOT-CLI `848b2d56e9e416ae87fa46d221c9649041b64541` and DOT-Repository `b681c937422fa3db13ad16587cf271700eff5d79`.
- Portable evidence checksums and retained GitHub Actions artifacts.
- Separate workflow concurrency lanes for push conformance and manually requested live stress, preventing one evidence class from canceling the other.

Linux amd64 is the release-accepted executable target. The published arm64 executable is a cross-built preview and is not a supported final-release target until equivalent native arm64 evidence exists. Producer/partner acceptance, non-Linux executables, multi-host orchestration, general stateful migration, and automatic provisioning of cross-platform gateways remain outside this candidate. GraphQL, gRPC, and event-driven runtime probes beyond the documented safe profile remain standards-led roadmap work rather than an `rc.2` blocker.

The candidate assets and verification procedure are the same classes described for `rc.1` below, with filenames versioned as `0.2.0-rc.2`. The tag workflow rebuilds the Java artifact, both schemas, SBOMs, and Linux `dotx` binaries from the immutable revisions recorded in `release-manifest.json`, then verifies reproducibility and `SHA256SUMS` before publication.

0.2.0-rc.1

`0.2.0-rc.1` is the first integrated release candidate for the stable `dot.mare6g.eu/v1` composition profile. It replaces the narrower 0.1.0 artifact set with a traceable domain model, both machine-readable schemas, and version-stamped `dotx` binaries built from the exact DOT-CLI revision pinned by this repository.

Highlights

- Stable, repository-backed Composition v1 lifecycle, probe, and upgrade commands.
- Contract-driven Anchors for OpenAPI 3.0/3.1/3.2, AsyncAPI 3.0/3.1, Protobuf/gRPC, GraphQL, and declared contractless interactions.
- Immutable repository resolution, content-addressed locks, resumable lifecycle operations, and last-known-good upgrade rollback.
- Managed Docker image, Docker Compose, Kubernetes manifest, Helm chart, Vagrant/libvirt, and Wasmtime/WASI profiles.
- Explicit cross-platform networking through externally owned gateway bindings with pinned host, port, ownership, and no-cleanup semantics.
- Service-scoped configuration, secrets, OCI requirements, CDI devices, and exact native-workload bindings.

Assets and verification

The candidate includes the DOT-Domain JAR, DOT and Composition v1 schemas, CycloneDX SBOMs, and `dotx` binaries for Linux amd64 and arm64. `release-manifest.json` records the immutable DOT-Design, DOT-CLI, and DOT-Repository revisions. The repository revision pins its `DOTs/`, `Skills/`, and `Schema/` assets together. Verify the download set with:

```
sha256sum --check SHA256SUMS
```

The tag workflow publishes only after the Java build, generated-schema drift check, DOT-CLI tests, repository test, Composition v1 acceptance suite, deterministic packaging comparison, manifest validation, and checksum verification pass.

Compatibility and boundaries

The serialized composition API is `dot.mare6g.eu/v1`; breaking serialized changes require a new API version. The experimental local catalogue/registry prototype from the 0.1.0 era is not included: DOT-Repository is authoritative.

This candidate does not claim multi-host orchestration, automatic provisioning of cross-platform gateways/tunnels/brokers, general stateful data migration, arbitrary privileged workloads, non-Linux `dotx` executables, or producer/partner acceptance. The current CLI includes Linux process controls for its managed WASI runtime, so this candidate publishes Linux amd64 and arm64 binaries only. Closed legacy SFC issues record DOT-side disposition; joint consumer acceptance remains a separate manual gate.

See the repository-root `RELEASE_NOTES.md` included with the source for the complete candidate notes and upgrade guidance.

DOT-Domain

Domain model

`DOT-Domain` is the persistence-neutral Java library that defines the portable DOT aggregate and its validation rules. It contains no application, web, or storage-adapter layer.

Aggregate overview

Read the diagram from `Dot` down to a deployment’s logical services. It shows the main relationships without the configuration and validation details below.

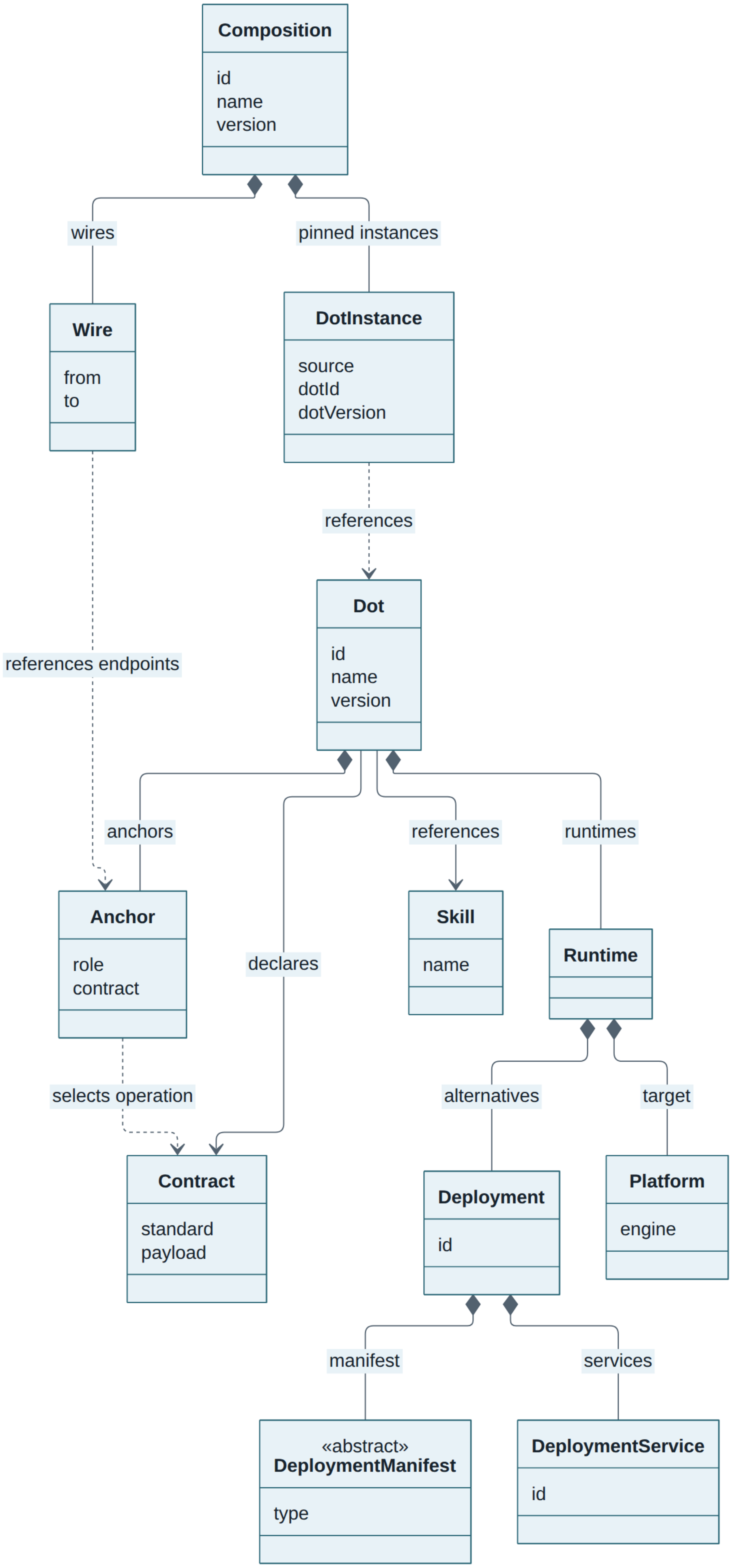

Composition
id
name
version
wires
pinned instances
Wire
from
to
DotInstance
source
dotId
dotVersion
references
Dot
id
name
version
references endpoints
anchors
references
runtimes
Anchor
role
contract
declares
Skill
name
Runtime
selects operation
alternatives
target
Contract
standard
payload
Deployment
id
Platform
engine
manifest
services
«abstract»
DeploymentManifest
type
DeploymentService
id

Figure 9. Simplified domain model: reusable DOT definitions and a separate Composition

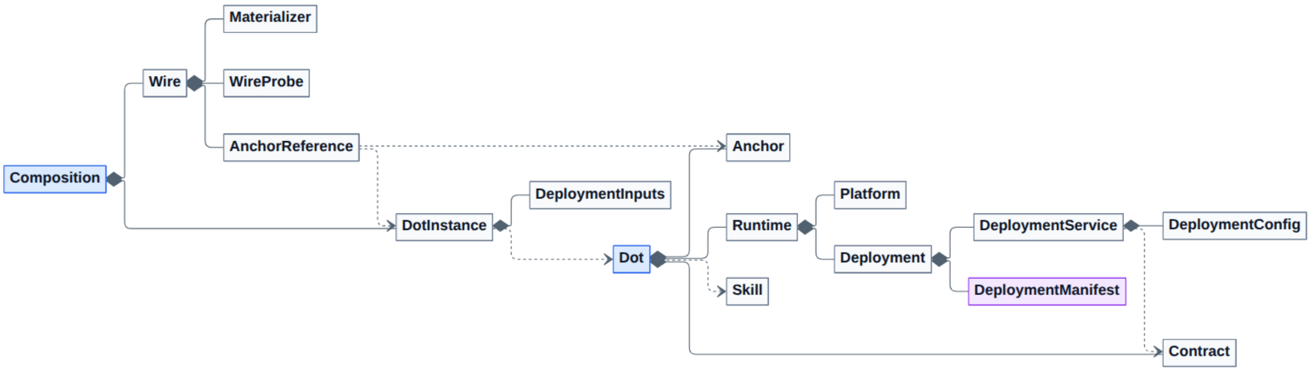
Diamonds indicate embedded values; arrows indicate declarations or references. The dashed arrow applies to Contract-backed Anchors. Collection sizes, configuration, ports, binding helpers and audit metadata are omitted for readability; see the full generated class diagram for implementation detail. `Composition` is a separate DOT-Domain aggregate: its instances reference DOTs and its Wires reference their Anchors. Neither is embedded in the reusable Java `Dot` aggregate.

Type	Responsibility
`Dot`	Aggregate root: identity, version, description, classification groups, metadata, Skills, Contracts, Anchors, and Runtimes.
`Skill`	Standalone human-readable capability description.
`Composition`	Separate aggregate: pinned DOT instances, Anchor Wires, exposed Anchors, deployment inputs, Secret references and active probe expectations.
`Contract`	Standalone authoritative OpenAPI, AsyncAPI, Protobuf, or GraphQL document.
`Anchor`	Embedded stable logical point through which the DOT participates in composition.
`Runtime`	A target `Platform` plus one or more compatible deployment definitions.
`Deployment`	Stable deployment ID, one polymorphic manifest, and the logical services it materializes.
`DeploymentService`	Service identity, host, named ports, Contract references, Anchor bindings, and its supported configuration surface.

`Dot`, `Skill`, `Contract`, and `Composition` are independently identified domain entities. Other types are embedded values owned by these aggregates. The module carries no database or ORM annotations; persistence is an adapter concern.

Complete Mermaid model reference for deliverables

The A4 landscape reference sheet combines a readable core class diagram with an inventory of every exported public model type (77 in this source revision). Scalars, generated accessors and validation implementation helpers are omitted. Supporting types are listed by their nearest core-model family, not presented as exclusively owned values, extra aggregates or guaranteed runtime features. Shared types are listed once.



The complete vector diagram shows all 77 public classes/enums and their relationships individually. Both the compact Mermaid source and complete Mermaid source come from the existing Java-generated diagram and exported schema inventory. The DOT-Domain README renders them natively and includes a schema/semantically validated, compact Prometheus `dot.json` projection using the real catalogue Contract and deployment, rather than another product descriptor.

After an intentional model/catalogue documentation change, run `task docs:domain:generate` from DOT-Design with the source-built CLI installed. `task docs:domain:check` verifies source/output drift and the exact JSON example offline; it is part of `task testing:models`, not a live deployment test.

One model authority

DOT-Domain defines **all** portable models, including Composition, not just `Dot`. The existing Victools build generates `Schema/schema.json` for DOTs, `Schema/composition-v1.schema.json` for Composition v1, and standalone `Schema/models/*.schema.json` files for every public domain data class and enum. Jackson reference views remain intentional: a Skill reference containing only `id` is not the full standalone Skill entity.

The CLI consumes generated Go input types and embeds the generated Composition schema. It does not require Java at runtime. From DOT-Design, regenerate with:

```
mvn -pl DOT-Domain process-test-classes
task cli:domain:generate
```

Never edit generated files manually. `task testing:composition:model` checks generation drift and v1 compatibility after the Contract validator bootstrap.

`Composition` references reusable DOTs; it neither inherits from `Dot` nor copies its runtimes or Contracts. Resolved execution details, lock digests, private lifecycle state, endpoint allocation and runtime policy enforcement remain CLI/executor concerns. Standard Jakarta constraints and Jackson shape annotations describe fields; model-owned JSON Schema annotations express conditional input rules such as exactly one literal value or Secret reference. Jakarta validation also checks the exclusive environment, workload, probe and gRPC request choices, and the fields allowed for each materializer destination. Cross-DOT resolution, secret authorization and ownership checks still run in the CLI after schema validation.

This migration preserves `dot.mare6g.eu/v1`, existing documents and reviewed locks. Composition `version` and `dotVersion` retain the existing v1 string semantics; they have not silently become stricter Java `Version` fields.

Domain stabilization

The immediate priority is a model compatibility freeze, before further CLI authoring features. Preserve valid Composition v1 JSON and reviewed locks while allowing targeted Java API improvements. Probe and materializer selections use typed Java enums: `AsyncAPIProtocol`, `GraphQLTransport`, `GRPCResponseMatch`, `MaterializerType`, and `MaterializerValueFrom`. Existing wire values remain unchanged and exact, for example `MQTT`, `SSE`, and `CONFIG_FILE`.

```
AsyncAPIProbe probe = AsyncAPIProbe.builder()
    .protocol(AsyncAPIProtocol.MQTT)
    .correlationPointer("/id")
    .build();
```

This constructs a declaration, not a broker connection or successful delivery probe. Validate Java objects with Jakarta Bean Validation and raw input with the generated schema. Raw-input validation must precede deserialization when required-property presence matters: defaulted Java fields cannot distinguish an omitted required property from an explicitly supplied default. Optional empty Java Wire collections serialize as absent; explicit empty JSON collections remain invalid where the schema requires a nonempty value. Materializer target pointers such as `/routes/0/uri` remain accepted without rewriting.

The Java build enforces 100% instruction and branch coverage. Compatibility tests compare generated schemas to the frozen v1 oracle, including existing composition inputs and all 128 materializer field combinations. These checks do not establish independent adoption or live producer acceptance.

The `0.3.0-rc.1` candidate freezes the reviewed portable model surface, not a permanent 1.0 API guarantee. `DOT-Domain/MODEL-SPEC.md` records the cross-model invariant review and validation boundaries. The reproducible, checksummed `dot-domain-models-0.3.0-rc.1.tar.gz` release asset contains all 77 exported schemas and shared compatibility corpora. Publication and full runtime qualification remain separate release gates.

The service-configuration audit now rejects duplicate environment/CDI names, configuration-file paths, and volume names/target paths within one service. Distinct services keep separate scopes, so the same key may occur in both. Declarations retain their order and are never silently collapsed. Java and installed-CLI tests share the same positive/negative corpus. These are semantic checks: standard JSON Schema cannot express uniqueness by one property of an array item. The generated JSON shapes and reviewed composition locks remain unchanged; use full DOT validation rather than schema-only acceptance. Resolve these in DOT-Domain and its existing generation pipeline, not in independent CLI models. No final stable-model claim or new release is implied.

Deployment-free model gate

```
task testing:models
```

This sequential, fail-fast local gate runs Java tests and coverage, shared positive/negative examples and serialized Java/schema agreement, generated Go drift checks, CLI race tests and coverage, offline catalogue validation and reviewed locks. Every exported model needs reviewed examples; all enum values and registered manifest subtypes are covered. This is a baseline, not an exhaustive proof of every field combination.

No deployment, runtime startup or GitHub workflow dispatch occurs. Contributor toolchains are required; initial bootstrap may download pinned validation tools. Use `task ci:live` separately for actual adapter/runtime acceptance and `task testing:demo` separately for full presenter qualification. A coding-only push is not a live-runtime qualification or documentation publication.

Optional null manifest/configuration fields serialize as absent, not schema-invalid JSON nulls; JSON assertions intentionally retain `equals: null`. OCI capabilities/host namespaces reject duplicates before Set conversion. Java callers of their previous Set-specific setters must recompile. Generated schemas now retain existing Java patterns for Helm repositories and WASM module references/digests, rejecting malformed inputs earlier without changing valid document shapes.

Reference-aware validation and collection safety

Anchors use exact DOT-local kebab-case keys such as `search-api`, not generated UUIDs for new catalogue entries. `Anchor.ID_PATTERN` is shared by the Java definition, service bindings and composition references and exported to the generated schemas/Go inputs. A wire's `instance` identifies the owning DOT, so two different DOTs may reuse the same local key. Display labels and operation selectors remain separate and never act as lookup aliases. Legacy UUID strings remain readable when explicitly declared. These three Java properties now use `String`; recompile callers and convert old UUID arguments explicitly. Composition ownership UUIDs are unchanged. Catalogue migration updates references and reviewed locks together, not private live state.

Use ordinary `validator.validate(dot)` to cascade validation. ID-only Skill and service Contract views use the portable Jakarta `IdentityReference` group; ignored entity metadata is not validated as a complete declaration. Validating a standalone `Skill` or `Contract` still requires a full entity. Root Contracts use the `ContractDeclaration` group, admitting either a complete declaration with validated payload/version or the existing ID-only migration reference. Partial declarations fail. Serializing a root ID-only reference does not add the Java default version to its JSON.

Service Contract references must identify a root declaration or an explicitly sourced external Anchor Contract. A different release or similar name is not an alias. The model does no catalogue or network I/O: Skill existence, verified Contract assets and operation resolution remain CLI/resolver checks.

Scope	Unique keys
Runtime	Deployment IDs
Deployment	Service IDs
Service	Port names
DOT	Root Contract IDs and Anchor IDs
Composition	Instance names, Contract-input IDs, wire names, exposed-Anchor names

Jackson's `Deployment.setServices(Collection<DeploymentService>)` rejects duplicate service IDs before constructing its ordered Set; it never silently chooses one conflicting service. Recompile Java callers of the former Set-specific setter. Validation also rechecks keys after in-place ID mutation. The CLI applies the corresponding semantic checks to raw definitions and emits field locations without copying the portable models into handwritten Go types.

Dependencies, security scopes, CDI bindings and OCI grants retain their existing v1 `uniqueItems` constraints, now checked in Java lists too. Generated schemas also retain their annotated item-level nonempty-string/capability constraints. The portable `@UniqueElements` constraint drives both Jakarta validation and the shared Victools schema module; there is no separate hand-maintained list of unique fields. No blanket deduplication or reordering applies to arbitrary lists. Missing required values and null nested entries produce validation violations rather than aggregate-traversal exceptions. Validate raw required-property presence before deserialization, as described above. Optional null Dot/Skill fields serialize as absent, not schema-invalid nulls. Shared golden examples run through both Java round trips and generated-schema validation. The broader optional-value serialization audit remains open.

Identity, versioning, and references

`Dot`, `Skill`, and `Contract` use case-sensitive textual IDs. The catalogue uses `Prometheus@3.15.1`, `Metrics-Scraping`, and `Prometheus@3.15.1::OpenAPI::HTTP`, respectively. DOT IDs follow `<name>@<version>` using the actual tool release; Contract IDs follow `<origin-DOT.id>::<standard>::<name>`. The local Contract name is `HTTP` in this example. Consumers retain the originating identity, even when their own DOT has a different name or version. Contract standard versions and chart packaging versions are independent. IDs are explicit keys, not download URLs or filesystem paths. Existing UUID-shaped strings remain accepted, without implicit aliases to migrated entries. Anchor IDs are DOT-local kebab-case; Composition identities remain UUIDs. IDs are not URLs. The catalogue naming convention requires an explicit identity/reference update when changing a DOT name or tool release; mutating a Java object's fields does not silently regenerate its ID. Equality for `Dot`, `Skill`, `Contract`, and `Composition` is ID-based so changes to mutable metadata or collections do not destabilize sets or persistence identity.

Every DOT must reference at least one meaningful Skill. The catalogue's capability assets resolve through `Skills/`. Prometheus, for example, contains:

```
"skills": [{"id": "Metrics-Scraping"}, {"id": "Observability"}]
```

`dotx repository inspect <id>@<version> --repository DOT-Repository --json` resolves those IDs to the authoritative labels, descriptions and `Monitoring-and-Analysis` category. References require the exact ID; search filters may additionally match a name case-insensitively. A display-label change never implicitly changes identity. See the catalogue's `IDENTIFIER-MIGRATION.md` for the completed DOT/Contract migration and safe lock/state handling.

`Version` accepts Semantic Versioning 2.0.0 values and the special `latest` selector. It serializes as a string and implements SemVer precedence; `latest` sorts after concrete versions. A new `Dot` defaults to `0.0.0`, while `Platform.version` defaults to `latest` so no platform-specific release is implied.

`Contract.version` is a standard-specific string, not a software/API release: OpenAPI and AsyncAPI copy their exact top-level `openapi`/`asyncapi` version; Protobuf uses the native `syntax`/`edition` value (implicit `proto2` when absent). GraphQL SDL declares no version, so that field is omitted. There is no default or placeholder Contract version. `info.version` remains unchanged inside the upstream payload. `dotx validate`, composition loading and URL authoring reject mismatches even when the declared number is otherwise well formed.

JSON serializes `Dot.skills` and `DeploymentService.contracts` as ID-only references. Root `Dot.contracts` carries authoritative Contract declarations: identity, standard, format, and a structured `payload` with an HTTPS `url` and `sha256` pin. Raw API documents are not embedded in `dot.json`. Current catalogue roots must carry complete declarations; CLI validation rejects ID-only roots instead of searching local or sibling files. Verified bytes are cached outside the DOT; cold offline validation fails until warmed online. An Anchor references the root declaration by Contract ID and selector.

`dotx validate` enforces catalogue identity uniqueness, including DOT `(id, version)` and `(name, version)`, and rejects conflicting documents for one Contract ID across bundles. These catalogue rules are not database indexes or persistence requirements enforced by the Java entities.

Contracts

Contract `standard` accepts only the canonical JSON spellings `OpenAPI`, `AsyncAPI`, `Protobuf`, and `GraphQL`. Uppercase enum names, alternative casing, and surrounding whitespace are rejected. The Java enum constants remain uppercase; Jackson serialization and the generated JSON Schemas use their canonical values. `DataFormat` is a separate enum and is unchanged.

Standard	Payload format	Anchor selector
`OpenAPI`	`YAML` or `JSON`	RFC 6901 URI-fragment JSON Pointer to an Operation Object
`AsyncAPI`	`YAML` or `JSON`	RFC 6901 URI-fragment JSON Pointer to an Operation Object
`Protobuf`	`PROTOBUF`	Fully qualified service method
`GraphQL`	`GRAPHQL`	Root-field schema coordinate

`Contract.isFormatCompatibleWithStandard()` rejects mismatched standard/format pairs. Payload parsing and selected-element existence are resolver concerns; the domain model validates selector syntax and references within the aggregate.

Anchors and bindings

A contract-backed Anchor contains:

- a stable DOT-local ID and a separate display name;
- a `PROVIDES` or `REQUIRES` role;
- optional semantic-type URIs; and
- a Contract ID plus standard-specific selector.

It omits direction, protocol, and media type because those facts come from the selected Contract operation. A contractless Anchor must declare `direction`, at least one protocol, and at least one media type explicitly.

`DeploymentService.anchorBindings` maps an exact Anchor ID to a named `ServicePort` on that same service. An Anchor contains no host, numeric port, credential, placement, or live policy state.

The class-level `@ValidAnchor` constraint delegates cross-reference validation to `AnchorValidator`. It rejects:

- duplicate Anchor IDs within one DOT;

- Contract references not declared by the same DOT;
- selectors invalid for the referenced Contract standard;
- bindings to undeclared Anchors;
- bindings to port names absent from that service; and
- duplicate bindings of one Anchor within the same service.

See DOT Anchors and Composition for authoring and wiring examples.

Runtime two-layer model

`Runtime` separates infrastructure from deployment mechanism:

Enum	Used by	Answers
`PlatformEngine`	`Platform.engine`	Where the workload runs: `DOCKER`, `KUBERNETES`, `VAGRANT`, or `WEBASSEMBLY`.
`DeploymentManifestType`	`DeploymentManifest.type`	How it is deployed: Docker image, Docker Compose, Kubernetes manifest, Helm chart, Vagrantfile, or WASM module.

Every `DeploymentManifestType` declares its required `PlatformEngine`. `Runtime.isDeploymentPlatformAligned()` validates every deployment against the runtime platform. `Runtime.isDeploymentIdsUnique()` enforces stable deployment selection within that runtime.

`Deployment.services` is a set. `DeploymentService` uses ID-based equality, so logical service IDs remain unique within the deployment.

Deployment manifests

`DeploymentManifest` is polymorphic JSON dispatched by its existing `type` property.

Type	Java subtype	Important fields
`DOCKER_IMAGE`	`DockerImage`	Image reference and ordered dotenv payloads
`DOCKER_COMPOSE`	`DockerCompose`	Complete YAML/JSON payload and ordered dotenv payloads
`KUBERNETES_MANIFEST`	`KubernetesManifest`	Complete YAML/JSON manifest payload
`HELM_CHART`	`HelmChart`	Repository, chart or OCI reference, version, and ordered values payloads
`VAGRANTFILE`	`Vagrantfile`	Complete Ruby DSL payload
`WASM_MODULE`	`WebAssemblyModule`	Relative/file or HTTPS module URI, WASI execution world, command arguments, and required SHA-256 integrity pin

Docker Compose and Kubernetes manifest payloads accept YAML or JSON only. Helm values also preserve YAML/JSON order: later entries override earlier entries as with repeated Helm `-f` arguments.

Runtime instance concerns do not belong in a manifest. For example, a Helm release name and namespace are selected by deployment tooling, not embedded in `HelmChart`.

Service configuration declarations

Every `DeploymentService` owns one `DeploymentConfig` containing:

- environment variables, including default, required, and sensitive metadata;
- configuration-file paths, content type, defaults, and sensitivity;
- logical volume mounts;
- logical CNCF CDI device requirements; and
- optional OCI capability and host-namespace requirements.

These are supported inputs and requirements, not applied runtime state. Operator values, host paths, concrete CDI device names, and permission grants belong to deployment or composition input.

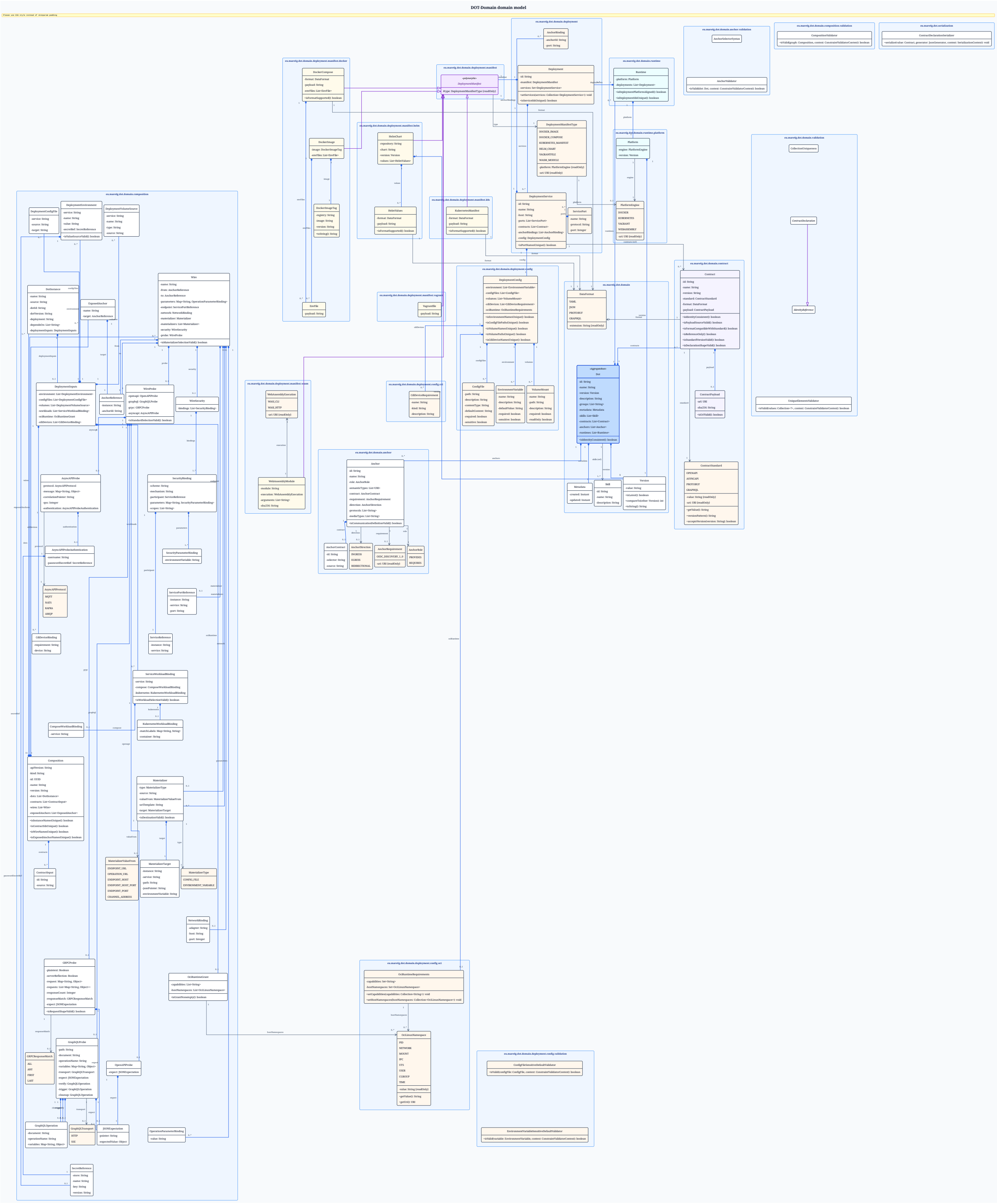
OCI requirements use standard `CAP_*` capability names and typed Linux namespaces. The model deliberately has no unrestricted `privileged` flag.

A CDI requirement declares a stable logical name and a `vendor.example/class` kind. A composition binds it to one exact `vendor.example/class=device` identifier. CDI installation, allocation, and authorization remain operator responsibilities.

Serialization and persistence adapters

- Collections have initialized builder defaults and do not become `null` when constructed through Lombok builders.
- `Version` is JSON-serialized as a normalized string.
- Persistence adapters must preserve UUID, textual identifier and version representations.
- Skill and service-level Contract references expose only `id`; root `Dot.contracts` contains authoritative declarations or verified local transition references, as described above.
- Domain values carry no database or ORM annotations.

Class diagram



The diagram is generated from current domain sources during the `DOT-Domain` build. Open it separately for the full-resolution view.

Build and test

```
# Unit tests only; no service or database required
mvn -pl DOT-Domain test
```

```
# Regenerate schema and diagram through the reactor lifecycle
mvn -pl DOT-Domain clean install
```

Representative tests cover version precedence, reference serialization, manifest polymorphism, runtime/manifest alignment, deployment configuration, Anchor rules, and entity equality.

Related pages

- Architecture
- DOT Anchors and Composition
- Integration handoff

DOT-CLI

CLI overview

`dotx` is the Go command-line application provided by the independent DOT-CLI repository. It operates on DOT definitions and compositions; it is not a catalogue server or a replacement for Docker, Kubernetes, Helm, Vagrant or Wasmtime.

Start with a working DOT

Follow Getting started for checksummed Linux-amd64 installation and the Prometheus Docker Compose quickstart: validate, deploy, verify an API response, inspect status and clean up. Running the released binary does not require a Java or Go development toolchain.

Use the guided demo to see a topology and a multi-DOT composition. The MARE PoC walkthroughs include a presenter pause for inspecting verified live results before cleanup.

Find the right command

Task	Command family
Check a definition, bundle or complete catalogue	`dotx validate`
Read a DOT's declared interfaces and deployment choices	`dotx inspect`
Discover and inspect catalogue entries	`dotx repository`
Author a DOT; create a separate editing candidate in current 0.4.0 source	`dotx new`; current source: `dotx edit`
Operate one selected DOT deployment	`dotx deploy`, `status`, `start`, `stop`, `destroy`, `ps`
Validate, lock, inspect and display a multi-DOT graph	`dotx compose validate`, `lock`, `inspect`, `topology`
Operate and verify a composition	`dotx compose apply`, `status`, `probe`, `destroy`
Relate a locked graph to observed runtimes and historical probes	`dotx compose topology --status` (use the applied run's cache/configuration)
Refresh or check composition README diagrams and Mermaid files	`dotx compose topology Compositions --write` / `--check`

Task	Command family
Check deployment tools and runtime prerequisites	`dotx tool`

Ask the installed binary for its exact options:

```
dotx version
dotx version --verbose
dotx help
dotx compose --help
dotx compose inspect --help
```

`version --verbose` adds the source revision, modified-worktree status, Go version and OS/architecture without contacting a server. Missing build metadata is shown as `unknown`. Plain `version` remains a single line; the version number alone does not identify the exact source build used for a rehearsal.

Composition subcommand help includes only that operation's flags, an example, a suggested next step and its safety boundary. For example, `compose inspect` resolves declared inputs without deployment, `apply --dry-run` previews commands, and `status` reads recorded runtime state. The complete production profile remains available through `dotx compose --help`.

Maintain composition topology documentation

From a standalone catalogue checkout, the selected release or matching source CLI can update every composition's existing README topology block and adjacent `topology.mmd`:

```
dotx compose topology Compositions --write
dotx compose topology Compositions --check
```

From DOT-Design, use `DOT-Repository/Compositions` instead. A single composition file is also accepted. Both modes require verified locks; no lock is repaired automatically. The command checks every graph and destination before writing, preserves authored text outside the marked block, and rejects symlinked files and malformed markers. `--check` writes nothing and exits nonzero on drift. Individual files are replaced atomically, not as a multi-file transaction; resolve any I/O failure and rerun. Display options such as `--format`, `--color` and `--status` cannot be combined with documentation modes.

Only `dotx` is needed: the former Bash generator and its jq/awk dependencies are not required. The optional `task compositions:topology:generate` and `:check` wrappers remain available in both checkouts. A generated diagram is a locked resolved composition view, not proof of live interoperability.

Find a DOT by its readable release coordinate

The current source-built CLI discovers the catalogue from your working directory. At each level, starting in the current directory and walking toward the filesystem root, it checks for `DOTs/` first, then `DOT-Repository/DOTs/`. Commands therefore work from DOT-Design, DOT-CLI, a standalone catalogue checkout, or a directory inside one of its DOTs/compositions. A renamed catalogue is recognized by its `DOTs/` directory, not by its Git remote or directory name.

```
dotx repository list --skill Metrics-Scraping
dotx repository list --verbose
dotx repository inspect Prometheus@3.15.1 --json
dotx repository resolve Prometheus@3.15.1
```

The default catalogue table starts with names, versions and scopes, followed by up to three Skill names. Every matching DOT remains visible; `+N more` indicates additional Skills. `--verbose` shows IDs, groups and categories. `--json` retains the complete structured report and takes precedence over `--verbose`. Listing definitions does not prove live readiness.

`inspect` and `resolve` accept an exact, case-sensitive DOT `name@version`, as well as the authoritative ID/version selector. The version is exact: no fuzzy name matching, automatic latest-version choice or identity migration is performed. The last `@` separates the version; quote names containing spaces. Ambiguous matches fail, and ID matches take precedence over coincidentally matching names. JSON output reports the authoritative case-sensitive ID and definition.

Explicit `--repository <path-or-git-url>` overrides discovery. If the selected catalogue is malformed, unreadable or missing, the command fails rather than switching to a different catalogue. Without a discoverable catalogue, provide that flag explicitly. Composition sources remain `repository://<id>@<version>`; display-name lookup does not replace source identities, locks or runtime ownership checks. No separate catalogue inventory is required.

Author a runtime-only scaffold

To connect existing DOTs without assembling identifiers manually, follow Create a composition from a reviewed example. That flow verifies a template's lock and creates a fresh ownership identity; it does not create new product definitions or deploy workloads.

Native authoring writes `dot.json` without Git, Jsonnet or template downloads. From the DOT-Design checkout:

```
dotx new --from docker-image prom/prometheus:v3.15.1 --name Prometheus --version
3.15.1 --skill Metrics-Scraping --port http:9090 --output ./my-dot/dot.json
dotx validate ./my-dot --schema DOT-Repository/Schema/schema.json
dotx inspect ./my-dot
# Alternatively: dotx new --interactive
```

This does not infer Contracts or Anchors and does not prove interoperability or live readiness. Reuse complete catalogue bundles where available; native authoring is a starting point for a genuinely new DOT. Add authoritative Contracts and explicit Anchors/bindings before integration. The positional `dotx new <name>` mode remains a template-based project workflow requiring Git, Jsonnet and network access. See `dotx new --help` for both modes.

Native authoring derives `<name>@<version>` as the DOT ID and requires at least one explicit `--skill <id>` (repeat the flag for multiple capabilities). Interactive authoring asks for a Skill when none was supplied. Declare the capability's meaning in the selected catalogue's `Skills/`; do not infer capabilities, interoperability or live readiness from the image alone. The actual tool release must be supplied explicitly; no `0.0.0` default is invented and Helm chart versions do not determine application versions.

For an owned OpenAPI interface, native authoring can also verify a generated-model Contract declaration, select operations and generate Anchor IDs and named-port bindings. See the executable interface-authoring exercise. Published v0.3.0 requires explicit authoring for other standards and dependency references. The matching 0.4.0 source adds guided imports for all four standards and separate editing candidates; the authoring guide states their supported profiles and review boundaries.

Inspect deployment choices

From the DOT-Design checkout, use the current source-built CLI:

```
dotx inspect DOT-Repository/DOTs/external/Prometheus-DOT
dotx inspect DOT-Repository/DOTs/external/Prometheus-DOT --verbose
dotx inspect DOT-Repository/DOTs/external/Prometheus-DOT --json
```

The default overview lists every deployment's ID, platform, manifest type, service count and required inputs, followed by short Contract, Anchor and Skill previews. It never automatically chooses a deployment. `INPUTS*` counts required environment/config-file declarations without defaults for each service; it is not a readiness result. Storage and host/device declarations are flagged separately so you can review persistence and permissions before deploying.

Use `--verbose` for IDs, all interfaces/selectors, service bindings and detailed configuration declarations. `--json` preserves the complete structured report and overrides `--verbose` when combined. Invalid sensitive defaults are rejected in every mode; provide secrets through deployment-instance inputs instead. Inspection does not start workloads, test runtime availability or prove live API acceptance. It does not replace `dotx validate`.

Root Contract metadata comes directly from complete `dot.json` declarations; ID-only roots fail rather than importing metadata from local files. JSON keeps the existing report structure, IDs and ordering. Inspection never downloads or verifies payload bytes; use `dotx validate` for that separate check. Validation caches digest-verified bytes outside the DOT. Warm it online once before `--offline`; a cold cache fails without requests or writes. Deployment support uses the same capability list as deploy/lifecycle, including WebAssembly.

Human output provides a validation command and a dry-run template. Choose an exact deployment ID from the displayed choices and replace `<deployment-id>`; inspection never selects one automatically. Remote Git targets instead require a persistent checkout for these next steps, since their inspection clone is temporary.

Understand validation failures

The current source CLI reports a diagnostic code, the affected file, an RFC 6901 JSON Pointer, a short explanation and a repair hint. Schema diagnostics use the selected generated schema to show expected types and exact enum spellings; there is no separate hand-maintained model definition. Input values, manifest contents and raw download errors are withheld to protect credentials. An empty pointer means a document-root or file-level failure. Some catalogue metadata failures identify a directory rather than an individual file.

For editors, scripts and CI, request one JSON report on stdout, including failures:

```
dotx validate DOT-Repository --offline --json
```

Run this example from a DOT-Design checkout with its pinned catalogue and source-built CLI; from a standalone catalogue checkout use `.` instead of `DOT-Repository`. The report has `formatVersion: 1`, `valid`, `target`, `kind`, completed `dots`/`compositions` counts, and `diagnostics` containing `code`, `file`, `pointer`, `message`, `hint` and optional `keyword`/`expected` fields. Exit codes remain `0` (valid), `1` (invalid or unavailable), and `2` (usage error). Normal JSON-mode validation failures leave stderr empty. Validation stops at the first failing definition or phase, although a schema failure can report multiple constraints. It never repairs inputs or rewrites reviewed locks automatically.

The same source CLI also reports failures from `dotx compose validate <composition.yaml> --json` and `dotx compose inspect <composition.yaml> --json` as one secret-safe JSON document on stdout, including usage errors. These failure reports have `formatVersion: 1`, `valid: false`, `kind: composition`, `target`, shared `diagnostics` and `evidence: composition-validation-not-live-acceptance`. Normal JSON failures leave stderr empty; output-write failures use stderr. Schema diagnostics retain exact field pointers, while resolution failures without finer context identify the composition file. Raw parser, document and transport values are withheld.

Successful composition validation still returns its existing `valid`, `compositionId` and `unknownWires` summary, and successful inspection still returns resolved execution details. Check the exit code and result, not merely whether stdout contains JSON. These failure-report guarantees do not extend to runtime `apply`, `upgrade`, `probe` or `status` commands. No lock is regenerated automatically, and an unknown wire remains an explicit limitation rather than live acceptance.

Recover from a composition runtime failure

The matching 0.4.0 source also gives `compose apply`, `upgrade`, `probe`, `status` and `destroy` secret-safe diagnostic codes and recovery hints on stderr. This is **not** a new runtime JSON failure envelope: successful JSON shapes and existing exit-code meanings are preserved. Raw tool, parser, transport and credential values are withheld. A diagnostic identifies an instance or wire when the failing boundary supplies that context; generic failures identify the composition instead.

Code	What to do next
`COMPOSITION_INSTANCE_DEPLOY` / `COMPOSITION_INSTANCE_CLEANUP`	Inspect the named instance and recorded status. Use the original CLI and run identity for cleanup; never remove unrelated resources.

Code	What to do next
`COMPOSITION_WIRE_PROBE`	Review the named wire's operation, endpoint, authentication and probe inputs. A failed probe is not producer acceptance.
`COMPOSITION_BUSY` / `COMPOSITION_RESUME_STATE`	Check the active operation and recorded state. Do not bypass locks, adopt another run or regenerate a composition lock to conceal a mismatch.
`COMPOSITION_UPGRADE_STORAGE` / `COMPOSITION_UPGRADE_RESTORED`	Review storage safety or the restored previous deployment before retrying. Restoration is not a successful upgrade.

Follow the emitted hint and run `dotx compose status <composition.yaml>` with the same cache/configuration as the applied run. No diagnostic automatically repairs locks, changes ownership or grants permissions. These improvements do not claim an equivalent failure-report format for top-level single-DOT runtime commands.

For reusable catalogue examples and explicit deployment selection, see Deployments and examples. For Anchor selectors, Wires, native Secret references, locks and lifecycle safety, use the composition guide.

Validation and resolution are not live acceptance. Active probes run only for supported, explicitly configured Contract operations and transports. The walkthroughs state which endpoints are actually exercised and distinguish real producers, source-adapted integrations and synthetic fixtures.

Versions and model authority

See Adopting DOT for compatible artifact sets, editor schema associations and the current-source, deployment-aware `dotx tool check --dot <bundle> --deployment <exact-id>` command. A successful prerequisite report is not live API acceptance.

DOT-Design and its pinned DOT-CLI source versions are kept aligned. Release installation instructions use the published binary; development demos and the diagnostics above may require the exact source revision pinned by the DOT-Design checkout, even when its version label matches a release. See Development and testing for source installation, Bash completion, local CI-equivalent Tasks and prerequisites.

All portable models belong to DOT-Domain. The CLI consumes its generated JSON Schemas and Go bindings; it does not define a competing DOT or Composition input model. Resolved execution details, lock fingerprints and private runtime state are executor concerns. DOT definitions and reusable composition inputs live in DOT-Repository.

Installation and first DOT

Compatibility, editor setup and preflight

Compatibility and migration policy

Author a DOT and its interfaces

Start by reusing a complete catalogue DOT. Author a new definition only for a genuinely new component or a local exercise; do not publish another root for an existing product. Add deployment alternatives to its canonical bundle instead.

These commands require the **selected release CLI `v0.3.0` or its matching source build**, `jq`, catalogue access and network access to the pinned raw Contract. The original published `v0.2.5` binary predates this flow. Follow the compatible-artifact installation routes. No Java, Jsonnet, Docker daemon or Kubernetes cluster is needed for the commands below; the last command renders a deployment without starting it.

Confirm the actual component release

`--version` is required for noninteractive authoring. Interactive mode asks for the actual tool release without inventing `0.0.0`. Image packaging tags and Helm chart versions are not reliable substitutes: confirm the tool release yourself. Explicit source/test versions remain available for unreleased components.

Declare at least one meaningful `--skill`; repeat it for multiple capabilities. Use an existing catalogue capability where appropriate and document a genuinely new capability in `Skills/`. Creation does not infer Skills from an image.

Rehearse with the existing Prometheus Contract

This temporary exercise reuses Prometheus's existing authoritative declaration; it creates no second catalogue entry and copies no dependency interface. From DOT-Design, prepare an empty exercise directory:

```
# Run from DOT-Design; set DOT_REPOSITORY=$PWD from a standalone catalogue.
DOT_REPOSITORY="${DOT_REPOSITORY:-$PWD/DOT-Repository}"
AUTHORING_DIR="$(mktemp -d -t dotx-authoring.XXXXXXXX)"
jq '.contracts[0]' "$DOT_REPOSITORY/DOTs/external/Prometheus-DOT/dot.json" \
  >"$AUTHORING_DIR/contract.json"
```

`contract.json` is one complete `Contract` in the generated DOT-Domain model, not a raw OpenAPI document or catalogue wrapper. It contains the owned ID, name, standard version, standard, format and HTTPS `payload.url`/`sha256`. For your own tool, provide its own declaration with a release-matched official specification where available. Never relabel another component's Contract.

```
dotx new --from docker-image prom/prometheus:v3.15.1 \
  --name Prometheus --version 3.15.1 --skill Metrics-Scraping \
  --service prometheus --port http:9090 \
  --contract "$AUTHORING_DIR/contract.json" \
  --operation 'build-information=#/paths/~1status~1buildinfo/get' --bind http \
  --output "$AUTHORING_DIR/bundle/dot.json"
```

Expected: an owned-interface verification message (`Owned OpenAPI interface verified` in v0.3.0; `Owned interface verified` in current source). The command validates the declaration against the generated schema, checks exact ownership, downloads and verifies the payload using the ordinary bounded HTTPS/SHA-256 verifier, and resolves the selected operation using the composition resolver. It writes the complete Contract, the explicit Anchor ID, its service Contract reference and the named-port binding into `dot.json` only after verification succeeds.

`build-information` is the stable DOT-local wire identity, not a generated UUID or a display-name alias. `--operation <anchor-id>=<selector>` uses that exact key for both the Anchor and its service binding; `name` initially has the same value but may subsequently change independently. Use kebab-case keys. The selector remains separate from the Contract ID. Repeat `--operation` for additional provided operations on the same declared TCP port. `--bind` requires an exact service port name and never guesses a listener.

For a menu of operations and port choices, use `--interactive --contract "$AUTHORING_DIR/contract.json"` with the same source/name/version/Skill/port arguments and a new output path. Without an imported Contract, interactive mode offers an optional declaration-file prompt; leave it empty for a runtime-only scaffold. Do not overwrite this exercise's output unintentionally.

Validate, inspect and render

```
dotx validate "$AUTHORING_DIR/bundle" \
  --schema "$DOT_REPOSITORY/Schema/schema.json" --profile reusable
dotx inspect "$AUTHORING_DIR/bundle"
dotx deploy "$AUTHORING_DIR/bundle" --deployment docker-image --dry-run
```

Expected: the reusable profile passes, inspection shows one owned Contract and one provided Anchor, and dry-run renders the Docker command. This checks the declared interface and binding, **not** a running API. The flow does not establish that a URL is vendor-owned or that its release metadata is truthful; review provenance. For live acceptance, follow the source-qualified guided demo. The released quickstart uses its own pinned artifact set; do not replace only one component of a qualified pair with another binary, including the original `v0.2.5`.

The published v0.3.0 guided interface profile covers **owned OpenAPI provided operations** only. AsyncAPI, GraphQL, Protobuf, external dependency references and provider-neutral requirements remain supported by the domain and composition commands, but require explicit authoring. No protocol metadata is duplicated from the Contract and no compatibility or live-readiness claim is inferred.

Import directly from the official raw URL

Alternatively, the same native authoring command can construct the generated Contract declaration from a raw HTTPS URL. Run this after the previous exercise so both temporary outputs can be inspected and cleaned up together:

```
# Alternative: supply the official raw URL without hand-writing contract.json.
# This exercise creates no second catalogue entry.
AUTHORING_SPEC_BASE=https://raw.githubusercontent.com/prometheus/prometheus/v3.15.1
dotx new --from docker-image prom/prometheus:v3.15.1 \
  --name Prometheus --version 3.15.1 --skill Metrics-Scraping \
  --service prometheus --port http:9090 \
  --contract-url "$AUTHORING_SPEC_BASE/web/api/v1/testdata/openapi_3.1_golden.yaml" \
  --contract-name HTTP --contract-version 3.1.0 --contract-format YAML \
  --operation 'build-information=#/paths/~1status~1buildinfo/get' --bind http \
  --output "$AUTHORING_DIR/url-bundle/dot.json"
dotx validate "$AUTHORING_DIR/url-bundle" \
  --schema "$DOT_REPOSITORY/Schema/schema.json" --profile reusable
jq '.contracts[0]' "$AUTHORING_DIR/url-bundle/dot.json"
```

Expected: the resulting ID is `Prometheus@3.15.1::OpenAPI::HTTP`, with the official URL and a 64-character lower-case SHA-256 in `payload`. No raw document is embedded. `--contract-version 3.1.0` matches the document's `openapi: 3.1.0`, not its `info.version` or the tool's `--version 3.15.1`. The upstream document's `info.version: 0.0.1-undefined` remains unchanged inside the pinned payload; it is not copied into DOT Contract metadata. A standard-version mismatch fails before publication. Do not infer versions from a filename or rewrite upstream bytes.

To require bytes already approved by your team, add `--contract-sha256 <trusted-digest>`. A mismatch fails without replacing output. Without that option, the command pins the bytes retrieved on first use: review the URL, upstream provenance, release match, license and resulting digest before committing. Computing a hash does not prove that the URL is vendor-owned or that its API matches a deployed producer.

The downloader reuses validation's HTTPS, public-address, redirect, timeout, size and credential-scoping controls. Mutable GitHub branch URLs are refused; use an official release tag or full commit. Transport failures withhold URL and credential details. `--contract` and `--contract-url` are mutually exclusive. For a derived specification hosted on GitHub Gist, use the raw `gist.githubusercontent.com` URL containing the full 40-character revision and filename, without query parameters, plus a trusted digest. Secret Gists are readable without authentication by anyone with the URL; GitHub tokens are not sent to them. Preserve source/license provenance and do not label a derived projection vendor-official merely because it is hosted externally. The published v0.3.0 URL importer requires explicit `--contract-name`, `--contract-version` and `--contract-format JSON|YAML`; it does not guess a standard from the URL.

Interactive authoring accepts either a declaration filename or an HTTPS URL, asks for missing metadata and offers the same verified operation/port menus. The digest selected during that prompt must still match at publication.

Recover from errors and clean up

- Missing release: supply `--version`; no output is created.
- Ownership mismatch: use the actual originating product/release, or an external Anchor reference for a dependency. Do not rename a dependency into ownership.
- Download or digest failure: check access, the official raw URL and expected bytes. Never change a hash solely to bypass a failure.

- Invalid selector: choose an operation from the verified specification. Encode OpenAPI selectors as URI-fragment JSON Pointers, including `~1` for `/`.
- Binding failure: declare `--port http:9090` and use `--bind http`.
- Existing output: choose a new path. `--force` permits replacement only after verification. Complete bytes are staged and published atomically; a concurrently created output is not overwritten without `--force`. Forced replacement preserves existing Unix file mode bits and does not follow a destination symlink into another file. Staging failures preserve the old file.

Non-overwriting publication requires hard-link support in the output filesystem; unsupported filesystems fail closed rather than falling back to truncation.

`new` creates a fresh definition; it is not an incremental editor or migration command. `--force` replaces the whole scaffold using the supplied Anchor IDs; it does not preserve interfaces omitted from the new command. For an existing integrated DOT, edit its definition while retaining stable Anchor identities and review affected composition locks instead of regenerating the scaffold.

```
# Only the files created by this exercise; unexpected extra files are retained.
rm -- "$AUTHORING_DIR/bundle/dot.json" "$AUTHORING_DIR/contract.json"
if [[ -d "$AUTHORING_DIR/url-bundle" ]]; then
  rm -- "$AUTHORING_DIR/url-bundle/dot.json"
  rmdir -- "$AUTHORING_DIR/url-bundle"
fi
rmdir -- "$AUTHORING_DIR/bundle" "$AUTHORING_DIR"
```

Only exercise files are removed. No containers, networks, volumes or cluster resources were created. Developers can run the same documented commands and ownership-negative control with `task testing:authoring` from DOT-Design.

Current-source guided authoring for 0.4.0

These additions require the matching source build; **the published v0.3.0 binary does not contain them**. The reviewed 0.3.0 portable models and schemas are not expanded. Record `dotx version --verbose` and the exact source/catalogue revisions.

`dotx new --interactive` now offers verified operations for all four standards:

Standard	Raw format	Supported selector/profile
OpenAPI	JSON or YAML	Operation JSON Pointer, including supported additional operations
AsyncAPI	JSON or YAML	Version-3 send/receive operation; explicit broker endpoint in the composition
Protobuf	PROTOBUF	Fully qualified service method in a standalone specification
GraphQL	GRAPHQL	Declared query, mutation or subscription root field

New, edit and composition authoring use the same native scrolling TUI. In single-choice menus, type initial letters to filter immediately: `Pro` finds Prometheus, `Key` finds Keycloak. Search is case-insensitive and also matches text within a choice. Use □/□ to browse, Backspace to edit the query, Enter to select and Ctrl-C to cancel. No `/` prefix is needed. Checkbox menus use Space to toggle, `/` to filter and Esc to finish filtering before toggling.

Set `DOTX_NO_TUI=1` or `ACCESSIBLE=1` for non-redrawing line menus. They also apply when output is redirected or the terminal is small or `TERM=dumb`: numbered menus use `/text`, `n`/`p` and `q`; configuration checkboxes use comma-separated names, `*` for all and empty for none. Confirmation remains an explicit text prompt, with no as the default. The browser uses the ordinary composition parsers and operation resolver; it does not create another compatibility engine. An operation menu is not evidence that every transport or feature in the standard is executable. Imported Protobuf dependencies that cannot be resolved by the existing standalone/well-known-import profile fail with no draft publication.

For URL imports, choose the exact canonical `--contract-standard`, name and raw format. Current source derives an omitted standard version from the verified payload's native marker and omits it for GraphQL SDL. Tool `--version` remains the actual component release. A supplied standard version must still match. Use `--offline` only with the trusted digest and already verified cached bytes; a cold or corrupt cache is not repaired or bypassed.

AsyncAPI operations materialize a send/receive participant, not a local broker listener. Do not pass `--bind`; choose the broker's actual service/port when building the composition. Existing MQTT, NATS, Kafka and AMQP acceptance profiles remain authoritative; declarations do not provision brokers or prove delivery.

Improve an existing DOT safely

From the matching **DOT-Repository checkout root**:

```
mkdir -p drafts
dotx edit DOTs/external/Prometheus-DOT/dot.json --interactive \
  --output drafts/prometheus.candidate.json --repository . --offline
```

Choose Skills, add an owned Contract, update its URL, add/change an Anchor operation or rebind an owned Anchor to a named service/port. IDs are retained for existing Anchors; no unaffected ID is regenerated. Selecting an AsyncAPI send/receive operation derives the correct role. Runtime-only definitions can gain their first interface; incomplete interoperability cannot be saved as a reusable candidate.

Expected: a semantic change summary followed by explicit confirmation. The complete candidate is checked against the generated model and ordinary Contract, selector and per-deployment binding rules before writing private, complete bytes. The original, deployment manifests, configuration, unrelated fields, locks and runtime state stay untouched. The editor refuses an existing output, a symlinked output parent, a changed original or a second catalogue `dot.json`. Cancellation, EOF, verification and publication errors leave no partial candidate.

Review the diff and validate the candidate. **Draft creation is not promotion**: explicitly decide whether to replace the canonical file, review dependent composition locks for the intended interface change and rehearse affected APIs. Do not publish a candidate as another product root. Delete only the candidate file when abandoning the exercise; no runtime cleanup is necessary.

Create a composition

A DOT describes one component. A Composition chooses concrete deployments of several DOTs and connects their Anchors through Wires. Start from a catalogue example whose README describes your intended use case and its acceptance limits. The selected release `v0.3.0` and its matching source CLI provide `dotx compose new`; the original released `v0.2.5` binary does not. Use a matching CLI and pinned catalogue.

Choose, preview and create

From your pinned **DOT-Repository checkout**, warm its verified Contract cache once:

```
dotx validate .
dotx compose new
```

In a terminal, choose a numbered local template, enter a name and an unused YAML path in an existing directory, then confirm the exact deployment preview. Nested template paths remain distinguishable. Entering `n`, an empty confirmation, or Ctrl-D does not publish a file. Noninteractive execution requires all choices and `--yes`, so a CI job cannot hang waiting for input.

For a repeatable exercise using the existing Node Exporter/Prometheus graph:

```
dotx compose new \
  --from Compositions/node-exporter-prometheus-monitoring/composition.yaml \
  --name my-monitoring --output my-monitoring.yaml --offline --yes
dotx compose lock my-monitoring.yaml --offline
dotx compose inspect my-monitoring.yaml --offline --locked
dotx compose topology my-monitoring.yaml --locked
```

Expected: a preview of two DOTs and one wire, a fresh ownership UUID, and a new YAML file with the same exact tool releases, deployment choices, Contract and Anchor IDs. The reviewed template and its lock are unchanged. The new lock is created only by your explicit `lock` command; review it before committing. The Composition graph version is retained; it is not another tool release.

The command verifies its input byte snapshot against the template's adjacent lock, applies changes to the Java-generated Composition model, preserves ordinary YAML shape/comments, then checks the rendered values exactly and validates the candidate with the same generated schema and resolver. YAML merge keys are expanded by the YAML parser where necessary: inherited IDs, names and file sources must receive the intended new values too. Authoring uses a fresh model of the verified input bytes, separate from inferred execution services and probes. It does not infer compatibility, invent wires, copy product definitions, download templates from Git, change secret values or start a runtime. Offline creation requires a warm verified Contract cache. Without `--offline`, bounded Contract downloads and cache writes may occur even when no draft is published. The suggested follow-up commands retain `--offline` when creation used it.

Review shared inputs before customization

Filesystem sources are rebased to the template's existing inputs, **not copied**. This avoids silent configuration duplication, but editing a referenced file would affect both compositions. Copy an input to your intended directory and change the new composition's `source` before customizing it. Repository DOT and Contract sources, absolute host bind grants and Secret references retain their identities. Host bind paths are not rebased as if they were relative configuration documents.

A fresh UUID prevents lifecycle-state adoption; it does **not** isolate fixed ports, application/workload names, named storage or externally owned gateways. Review these, Kubernetes namespaces and credentials before any real deployment. Existing collision, ownership, configuration and probe safety checks still apply. A checked resolved composition or generated topology is not live producer acceptance, security hardening or independent adopter acceptance. The template README's limitations continue to apply; creation does not qualify an untested producer.

Recover and clean up

Creator failures now include a stable diagnostic code and a specific recovery hint, without printing raw parser, filesystem or download errors:

- `CONTRACT_CACHE_MISS`: warm the pinned owner/catalogue online, then retry offline.
- `COMPOSITION_NEW_TEMPLATE_SCHEMA`: validate the template against the generated schema.
- `COMPOSITION_NEW_TEMPLATE_LOCK`: inspect and explicitly review an intentional template-lock update.
- `COMPOSITION_NEW_OUTPUT_PARENT`: choose an existing real directory without directory symlinks.
- `COMPOSITION_NEW_OUTPUT_LOCK` or `COMPOSITION_NEW_PUBLISH`: choose an unused destination; there is no force mode.
- `COMPOSITION_NEW_RESOLUTION`: validate exact releases, Contract origins and deployment choices using the same repository/offline options.

Failure never automatically repairs inputs or locks, starts a runtime, or publishes a partial draft. Diagnostic wording is executor output, not a new model.

- Missing or stale template lock: inspect the original inputs and lock diff. Fix an intentional change in the template separately; creation never repairs it.
- Existing output or adjacent lock: choose a new path. There is no `--force`.
- Missing or symlinked output parent: create/select a real directory first.
- Cold offline cache: run online validation of the pinned catalogue first. Digest mismatches require investigation, not changing a pin to bypass checks.
- Need diagnostic detail: run `dotx compose validate <template> --locked --json`. Creator failure messages withhold raw parser values, URLs and credentials.

Complete private bytes are published without overwriting a concurrently created file. Non-overwriting publication needs hard-link support; unsupported filesystems fail closed. No lifecycle state, workloads, networks or volumes are created. Remove only the exercise's `my-monitoring.yaml` and `my-monitoring.lock.json` when finished. For a reusable catalogue contribution, choose its final directory before creating it, add a README, review its lock and use `dotx compose topology <composition.yaml> --write` to generate its topology docs.

Current-source alternative: build from selected DOTs

This 0.4.0 work requires a matching **source** CLI; it is not in the published v0.3.0 binary. From the matching DOT-Repository checkout, with verified cached Contracts:

```
dotx compose new --from-dots --repository . --offline --name my-monitoring
```

Select exact releases and explicit deployments; give each instance a short, unique name. The native scrolling selector lets you browse **all** catalogue DOTs with □/□, type initial letters such as `Pro` or `Key` to filter immediately, select with Enter and cancel with Ctrl-C. Backspace edits the case-insensitive substring query; no `/` prefix is needed. New and edit use this same TUI. Deployment, Anchor and connection choices use the same UI. No DOT, Contract or Anchor ID needs to be constructed manually.

For a screen reader or the numbered interface, set `DOTX_NO_TUI=1` or `ACCESSIBLE=1` before the command. Redirected output, small and `TERM=dumb` terminals also use this non-redrawing fallback: `/text` filters, `n`/`p` pages, a displayed number selects and `q` cancels. The header states the visible range and total; the first 12 rows are not the whole catalogue. The UI does not change validation or perform any deployment. Input must still be a terminal; piped input is rejected. If you omit `--output`, press Enter to accept `Compositions/my-monitoring/composition.yaml` inside the selected catalogue. From DOT-Design, `--repository DOT-Repository` instead suggests `DOT-Repository/Compositions/my-monitoring/composition.yaml`. This is a draft's storage location, not a deployment destination. The same default applies to the current-source interactive template flow. Missing default folders are created only after validation and explicit write confirmation; cancellation creates no folders. A composition name used as a folder must be one path component, not `.`/`..` or a path; other valid display names can use an explicit output path.

To save elsewhere, enter another path at the prompt or supply `--output`. An explicit output's parent must already exist. Existing files, adjacent locks and directory symlinks are refused; there is no overwrite mode. Publication failure removes only still-identical empty folders created by this operation, never pre-existing folders or concurrent contents. Creating the catalogue draft does not add a README, topology artifacts or a lock; review and complete those before contributing the composition. For the real monitoring example choose Node Exporter and Prometheus, then connect `node-metrics` to `scrape-target`. Select a config-file materializer deriving `ENDPOINT_HOST_PORT`, targeting Prometheus's declared configuration file and the existing scrape-target JSON Pointer. Input paths are relative to the new YAML; review the existing monitoring README rather than inventing a producer interface.

The menu labels connections accepted, incompatible or UNKNOWN using the ordinary compatibility authority. UNKNOWN is not verified; required authentication must be explicitly bound and checked. Required operation parameters and service configuration are separate from identifiers. Enter native Secret **references**, never their values. A reviewed optional `DeploymentInputs` YAML/JSON fragment uses the exact generated model for advanced workload selectors, volumes and OCI/CDI grants; paths resolve beside the new composition, not beside the fragment. Missing, ambiguous, unsupported or unsafe inputs fail closed without output.

Brokered AsyncAPI send/receive pairs require a separately selected broker service/port, including existing MQTT, NATS, Kafka and AMQP profiles. This endpoint is not another Anchor wire. Cross-runtime pairs require an explicit, already provisioned `EXTERNAL_GATEWAY` host/port. The operator owns that gateway and its cleanup; the CLI neither creates networking infrastructure nor uses host-network fallbacks. The complete graph still passes the existing resolver's constraints.

Expected: exact deployment choices and a topology preview, then explicit write confirmation. A fresh ownership UUID and generated-model round-trip check precede non-overwriting publication. No lock is created, no existing state is adopted and nothing is deployed. `--from-dots` cannot be combined with `--from` or `--yes`, and requires a terminal. A complete validated draft is not live API acceptance.

After reviewing the YAML and every referenced input, explicitly create and review its lock using the same catalogue/offline options, then follow the real monitoring or authenticated secure-API acceptance route. Declaration validation does not qualify a catalogue gap, producer double or fixture for production use.

Deployments and examples

Compositions and lifecycle

Topology and guided demo

Development and testing